

Every Grouping of the Measurement Instrument

A Systematic, Line-by-Line Exegesis from Start to Finish

Antigravity AI Coding Assistant
Designed by Google DeepMind

July 13, 2026

Contents

1	Grouping A: The Ground Floor of Distinction (Episodes 1–5)	2
1.1	The Symbol and Decidable Equality	2
1.2	Constructing Natural Numbers and Order	2
2	Grouping B: Discrete Set and Representation Towers (Episodes 6–10)	3
2.1	Admissibility and Countability	3
2.2	Reals as Cauchy Towers	3
3	Grouping C: Commutators, Transport, and Curvature (Episodes 11–15)	4
3.1	Covariant Connection and Transport	4
3.2	The Un-Eliminable Residue	4
4	Grouping D: The Naming Corridor (Episodes 16–23)	5
4.1	The Backward Descent	5
4.2	Transition Corridors and Stable Readings	5
5	Grouping E: Lorentz, Spin-1/2, and Dirac Operators (Episodes 24–30)	6
5.1	The Dirac Operator	6
5.2	The Admissibility Theorem	6
6	Grouping F: Proximity, Photoelectric, and Cavendish (Episodes 31–34)	7
6.1	Cavendish Balance and Proximity	7
6.2	The Photoelectric Limit	7
7	Grouping G: The Second Variation and Dimensionless Coupling (Episodes 35–40)	8
7.1	The Second Variation	8
7.2	Maxwell and Meissner Boundaries	8
8	Grouping H: Surface Instruments and Calibration	9
8.1	The Two Descriptions	9
8.2	The Clifford Shadow and its Closure	9
8.2.1	Rigorous Proof of Algebra Closure	9
8.3	Stern-Brocot Mediant Descent and the Open Jar	10
8.4	Path Calibration and Finiteness Fence	10
9	The Complete Ledger of Logical Declarations	11
9.1	File: Episode1.lean	11
9.1.1	Class Fact (Line 75)	11
9.1.2	Inductive Number (Line 131)	11

9.1.3	Instance : LE Number where (Line 232)	12
9.1.4	Structure CarrierProcess (Line 239)	12
9.1.5	Class DISTINGUISHABLE (Line 277)	12
9.1.6	Inductive Natural (Line 330)	13
9.1.7	Instance : LE Natural where (Line 362)	13
9.1.8	Instance : LT Natural where (Line 366)	13
9.1.9	Structure CountingProcess (Line 393)	13
9.1.10	Class ADMISSIBLE (Line 476)	14
9.1.11	Inductive Rational (Line 530)	14
9.1.12	Instance : LE Rational where (Line 582)	14
9.1.13	Instance : LT Rational where (Line 585)	14
9.1.14	Structure IndexingProcess (Line 613)	15
9.1.15	Class COUNTABLE (Line 697)	15
9.1.16	Inductive Sequence (Line 800)	15
9.1.17	Instance : LE Sequence where (Line 849)	16
9.1.18	Instance : LT Sequence where (Line 852)	16
9.1.19	Structure LimitProcess (Line 894)	16
9.1.20	Class ENCODED (Line 970)	17
9.1.21	Inductive Limit (Line 1011)	17
9.1.22	Instance : LE Limit where (Line 1067)	17
9.1.23	Instance : LT Limit where (Line 1070)	17
9.1.24	Structure CauchyProcess (Line 1095)	18
9.1.25	Class RESIDUE (Line 1139)	18
9.1.26	Inductive Sample (Line 1187)	19
9.1.27	Instance : LE Sample where (Line 1241)	19
9.1.28	Instance : LT Sample where (Line 1244)	19
9.2	File: Episode2.lean	19
9.2.1	Structure ObservationProcess (Line 139)	19
9.2.2	Class BINARY (Line 243)	20
9.2.3	Inductive Trial (Line 381)	21
9.2.4	Instance : LE Trial := ⟨Trial.le⟩ (Line 442)	21
9.2.5	Instance : LT Trial := ⟨Trial.lt⟩ (Line 443)	21
9.2.6	Structure RepeatableProcess (Line 505)	21
9.2.7	Class REPEATABLE (Line 577)	22
9.2.8	Inductive Study (Line 712)	23
9.2.9	Instance : LE Study := ⟨Study.le⟩ (Line 765)	23
9.2.10	Structure ComputationalProcess (Line 801)	23
9.2.11	Class NUMERIC (Line 851)	24
9.3	File: Episode3.lean	24
9.3.1	Inductive Computation (Line 68)	24
9.3.2	Instance : LE Computation := ⟨Computation.le⟩ (Line 99)	25
9.3.3	Instance : LT Computation := ⟨Computation.lt⟩ (Line 100)	25
9.3.4	Structure TuringProcess (Line 120)	25
9.3.5	Class REPRESENTABLE (Line 178)	26
9.3.6	Inductive ChaitinsNumberSequence (Line 226)	27
9.3.7	Instance : LE ChaitinsNumberSequence := ⟨ChaitinsNumberSequence.le⟩ (Line 253)	27
9.3.8	Structure NoisyProcess (Line 279)	27
9.3.9	Class PHYSICAL (Line 355)	28

9.3.10	Inductive Metavariable (Line 420)	29
9.3.11	Structure PhysicalProcess (Line 454)	29
9.3.12	Class COMPARABLE (Line 521)	30
9.3.13	Inductive Sophism (Line 574)	31
9.3.14	Structure SlipProcess (Line 612)	31
9.3.15	Class OBSERVED (Line 661)	32
9.3.16	Inductive Area (Line 738)	33
9.4	File: Episode4.lean	33
9.4.1	Instance : LE Area := ⟨Area.le⟩ (Line 111)	33
9.4.2	Structure SensingProcess (Line 128)	33
9.4.3	Class PRESENT (Line 173)	34
9.4.4	Inductive Phenomenon (Line 217)	35
9.4.5	Instance : LE Phenomenon := ⟨Phenomenon.le⟩ (Line 260)	35
9.4.6	Structure GaugeProcess (Line 287)	35
9.4.7	Class MEASURABLE (Line 334)	36
9.4.8	Inductive Jar (Line 398)	37
9.4.9	Instance : LE Jar := ⟨Jar.le⟩ (Line 448)	37
9.4.10	Structure MeesaProcess (Line 481)	38
9.4.11	Class GUNGAN (Line 523)	39
9.5	File: Episode5.lean	39
9.5.1	Inductive Equivalation (Line 76)	39
9.5.2	Instance : LE Equivalation := ⟨Equivalation.le⟩ (Line 120)	40
9.5.3	Structure DigitalProcess (Line 143)	40
9.5.4	Class SOURCE (Line 214)	41
9.5.5	Inductive Encoding (Line 236)	42
9.5.6	Structure CompiledProcess (Line 257)	42
9.5.7	Class EXECUTED (Line 310)	43
9.5.8	Inductive Abstraction (Line 359)	44
9.5.9	Instance instance (Line 389)	45
9.5.10	Instance instance (Line 399)	45
9.6	File: Episode6.lean	45
9.6.1	Structure MathematicalProcess (Line 48)	45
9.6.2	Class VALUE (Line 92)	46
9.6.3	Inductive Sum (Line 127)	47
9.6.4	Instance : LE Sum := ⟨Sum.le⟩ (Line 145)	47
9.6.5	Instance : LT Sum := ⟨Sum.lt⟩ (Line 146)	47
9.6.6	Structure AddingProcess (Line 149)	48
9.6.7	Class MAGNITUDE (Line 182)	49
9.6.8	Inductive Product (Line 221)	50
9.6.9	Instance : LE Product := ⟨Product.le⟩ (Line 253)	50
9.6.10	Instance : LT Product := ⟨Product.lt⟩ (Line 254)	50
9.6.11	Structure MultiplyingProcess (Line 257)	50
9.6.12	Class SCALED (Line 300)	51
9.6.13	Inductive Basis (Line 344)	52
9.6.14	Instance : LE Basis where (Line 380)	52
9.6.15	Instance : LT Basis where (Line 383)	52
9.6.16	Structure BASICProcess (Line 387)	53
9.6.17	Class LOAD (Line 438)	54
9.6.18	Inductive Polynomial (Line 492)	55

9.6.19	Instance : LE Polynomial where (Line 521)	55
9.6.20	Structure GalerkinProcess (Line 526)	55
9.6.21	Class FINITE_ELEPHANT (Line 577)	56
9.6.22	Inductive Spline (Line 641)	57
9.6.23	Instance : LE Spline where (Line 686)	57
9.7	File: Episode7.lean	57
9.7.1	Structure ArmWaveProcess (Line 20)	57
9.7.2	Class BULLSHIT (Line 55)	59
9.7.3	Inductive Diatribe (Line 98)	60
9.7.4	Structure CrusadeProcess (Line 103)	60
9.7.5	Class PROPAGANDA (Line 172)	61
9.7.6	Inductive Cult (Line 207)	62
9.7.7	Structure InitiationProcess (Line 212)	62
9.7.8	Class ACOLYTE (Line 280)	63
9.8	File: Episode8.lean	64
9.8.1	Instance STEP_1 (Line 10)	64
9.8.2	Instance COUNTABLE_ADMISSIBLE (Line 21)	65
9.8.3	Instance ENCODED_COUNTABLE (Line 31)	65
9.8.4	Instance RESIDUE_ENCODED (Line 42)	65
9.8.5	Instance BINARY_RESIDUE (Line 54)	65
9.8.6	Instance REPEATABLE_BINARY (Line 70)	65
9.8.7	Instance NUMERIC_REPEATABLE (Line 87)	65
9.8.8	Instance REPRESENTABLE_NUMERIC (Line 103)	65
9.8.9	Instance PHYSICAL_NUMERIC (Line 125)	66
9.8.10	Instance OBSERVED_COMPARABLE (Line 151)	66
9.8.11	Instance PRESENT_OBSERVED (Line 174)	66
9.8.12	Instance MEASURABLE_PRESENT (Line 196)	66
9.8.13	Instance GUNGAN_MEASURABLE (Line 218)	66
9.8.14	Instance SOURCE_GUNGAN (Line 241)	66
9.8.15	Instance EXECUTED_SOURCE (Line 266)	66
9.8.16	Instance VALUE_EXECUTED (Line 292)	67
9.8.17	Instance MAGNITUDE_VALUE (Line 323)	67
9.8.18	Instance SCALED_MAGNITUDE (Line 351)	67
9.8.19	Instance LOAD_SCALED (Line 380)	67
9.8.20	Instance FINITE_ELEPHANT_LOAD (Line 410)	67
9.8.21	Instance BULLSHIT_FINITE_ELEPHANT (Line 440)	67
9.8.22	Instance PROPAGANDA_BULLSHIT (Line 471)	67
9.8.23	Instance ACOLYTE_PROPAGANDA (Line 500)	68
9.9	File: Episode9.lean	68
9.9.1	Inductive Science (Line 19)	68
9.9.2	Structure LearningProcess (Line 24)	68
9.9.3	Class SCIENTIFIC (Line 98)	69
9.9.4	Instance SCIENTIFIC_ACOLYTE (Line 137)	70
9.9.5	Inductive Knowledge (Line 170)	70
9.9.6	Structure ScientificProcess (Line 186)	71
9.9.7	Class TRUTH (Line 223)	72
9.9.8	Instance TRUTH_SCIENTIFIC (Line 258)	73
9.9.9	Inductive Gospel (Line 293)	73
9.9.10	Structure ReligiousProcess (Line 308)	73

9.10	File: Episode10.lean	74
9.10.1	Class WITNESSED (Line 15)	74
9.10.2	Instance WITNESSED_TRUTH (Line 52)	76
9.10.3	Inductive Truth (Line 89)	76
9.10.4	Structure UniverseTensor (Line 108)	76
9.10.5	Class REAL (Line 151)	77
9.10.6	Instance REAL_WITNESSED (Line 189)	78
9.10.7	Inductive Variation (Line 227)	79
9.10.8	Structure BigRedDogProcess (Line 265)	79
9.10.9	Class LOCAL (Line 313)	80
9.10.10	Instance LOCAL_REAL (Line 352)	81
9.11	File: Episode11.lean	82
9.11.1	Inductive SpaceTimePath (Line 12)	82
9.11.2	Structure CalculusProcess (Line 63)	82
9.11.3	Class UNIVERSAL (Line 108)	83
9.11.4	Instance UNIVERSAL_REAL (Line 130)	84
9.11.5	Inductive YarnTheory (Line 158)	84
9.11.6	Structure HeartbeatProcess (Line 209)	84
9.12	File: Episode12.lean	85
9.12.1	Class LOGICAL (Line 13)	85
9.12.2	Instance LOGICAL_UNIVERSAL (Line 34)	86
9.12.3	Structure ElaborationProcess (Line 90)	86
9.12.4	Class HALTED (Line 131)	87
9.12.5	Instance HALTED_LOGICAL (Line 155)	88
9.13	File: Episode13.lean	88
9.13.1	Inductive Measurement (Line 10)	88
9.13.2	Instance : LE Measurement where (Line 27)	88
9.13.3	Structure LeanProcess (Line 31)	89
9.13.4	Class MEASURED (Line 63)	90
9.14	File: Episode14.lean	90
9.14.1	Inductive CompilerTape (Line 11)	90
9.14.2	Instance : LE CompilerTape where (Line 30)	91
9.14.3	Instance : LT CompilerTape where (Line 32)	91
9.14.4	Structure CompilerOutput (Line 36)	91
9.14.5	Class COMPILED (Line 66)	92
9.14.6	Instance COMPILED_MEASURED (Line 86)	93
9.15	File: Episode15.lean	93
9.15.1	Inductive Bullshit (Line 12)	93
9.15.2	Instance : LE Bullshit where (Line 47)	93
9.15.3	Instance : LT Bullshit where (Line 49)	93
9.15.4	Structure AtreyuProcess (Line 53)	94
9.15.5	Class TrueOutput (Line 174)	95
9.15.6	Instance TRUE_COMPILED (Line 279)	96
9.15.7	Instance DISTINGUISHABLE_PROP (Line 306)	96
9.15.8	Instance truthDistinct (Line 320)	96
9.15.9	Inductive Closure (Line 397)	96
9.15.10	Structure EquivalenceProcess (Line 461)	96
9.15.11	Class INFERRED (Line 495)	97
9.16	File: Episode17.lean	98

9.16.1	Instance (priority (Line 5)	98
9.16.2	Instance (priority (Line 28)	98
9.16.3	Instance (priority (Line 53)	99
9.16.4	Instance (priority (Line 77)	99
9.16.5	Instance (priority (Line 105)	99
9.16.6	Instance (priority (Line 135)	99
9.16.7	Instance (priority (Line 166)	99
9.16.8	Instance (priority (Line 191)	99
9.16.9	Instance (priority (Line 223)	99
9.16.10	Instance (priority (Line 283)	100
9.16.11	Instance (priority (Line 308)	100
9.16.12	Instance (priority (Line 333)	100
9.16.13	Instance (priority (Line 357)	100
9.16.14	Instance (priority (Line 383)	100
9.16.15	Instance (priority (Line 408)	100
9.16.16	Instance (priority (Line 433)	100
9.16.17	Instance (priority (Line 459)	101
9.16.18	Instance (priority (Line 485)	101
9.16.19	Instance (priority (Line 511)	101
9.16.20	Instance (priority (Line 538)	101
9.16.21	Instance (priority (Line 565)	101
9.16.22	Instance (priority (Line 593)	101
9.16.23	Instance (priority (Line 621)	101
9.16.24	Instance (priority (Line 650)	102
9.16.25	Instance (priority (Line 678)	102
9.16.26	Instance (priority (Line 707)	102
9.16.27	Instance (priority (Line 737)	102
9.16.28	Instance (priority (Line 767)	102
9.16.29	Instance (priority (Line 798)	102
9.16.30	Instance (priority (Line 831)	102
9.16.31	Instance (priority (Line 867)	103
9.16.32	Instance (priority (Line 900)	103
9.16.33	Instance (priority (Line 931)	103
9.16.34	Instance (priority (Line 967)	103
9.16.35	Instance (priority (Line 999)	103
9.16.36	Instance (priority (Line 1031)	103
9.16.37	Instance (priority (Line 1063)	103
9.17	File: Episode18.lean	104
9.17.1	Instance (priority (Line 30)	104
9.17.2	Instance (priority (Line 67)	104
9.17.3	Instance (priority (Line 106)	104
9.17.4	Instance (priority (Line 146)	104
9.17.5	Instance (priority (Line 188)	104
9.17.6	Instance (priority (Line 229)	104
9.17.7	Instance (priority (Line 276)	104
9.17.8	Instance (priority (Line 317)	105
9.17.9	Instance (priority (Line 358)	105
9.17.10	Instance (priority (Line 408)	105
9.17.11	Instance (priority (Line 451)	105

9.17.12 Instance (priority (Line 494)	105
9.17.13 Instance (priority (Line 538)	105
9.17.14 Instance (priority (Line 580)	105
9.17.15 Instance (priority (Line 622)	106
9.17.16 Instance (priority (Line 664)	106
9.17.17 Instance (priority (Line 707)	106
9.17.18 Instance (priority (Line 752)	106
9.17.19 Instance (priority (Line 797)	106
9.17.20 Instance (priority (Line 841)	106
9.17.21 Instance (priority (Line 886)	106
9.17.22 Instance (priority (Line 931)	107
9.17.23 Instance (priority (Line 975)	107
9.17.24 Instance (priority (Line 1020)	107
9.17.25 Instance (priority (Line 1064)	107
9.17.26 Instance (priority (Line 1108)	107
9.17.27 Instance (priority (Line 1157)	107
9.17.28 Instance (priority (Line 1203)	107
9.17.29 Instance (priority (Line 1251)	108
9.17.30 Instance (priority (Line 1300)	108
9.17.31 Instance (priority (Line 1348)	108
9.17.32 Instance (priority (Line 1399)	108
9.17.33 Instance (priority (Line 1451)	108
9.17.34 Instance (priority (Line 1514)	108
9.17.35 Instance (priority (Line 1598)	108
9.17.36 Instance (priority (Line 1662)	109
9.18 File: Episode19.lean	109
9.18.1 Class TRACED (Line 10)	109
9.18.2 Instance (priority (Line 13)	109
9.18.3 Instance (priority (Line 18)	109
9.18.4 Instance (priority (Line 73)	109
9.18.5 Instance (priority (Line 137)	109
9.18.6 Instance (priority (Line 202)	110
9.18.7 Instance (priority (Line 258)	110
9.18.8 Instance (priority (Line 314)	110
9.18.9 Instance (priority (Line 377)	110
9.18.10 Instance (priority (Line 446)	110
9.18.11 Instance (priority (Line 512)	110
9.18.12 Instance (priority (Line 576)	110
9.18.13 Instance (priority (Line 636)	111
9.18.14 Instance (priority (Line 698)	111
9.18.15 Instance (priority (Line 761)	111
9.18.16 Instance (priority (Line 824)	111
9.18.17 Instance (priority (Line 887)	111
9.18.18 Instance (priority (Line 950)	111
9.18.19 Instance (priority (Line 1013)	111
9.18.20 Instance (priority (Line 1075)	112
9.18.21 Instance (priority (Line 1138)	112
9.18.22 Instance (priority (Line 1201)	112
9.18.23 Instance (priority (Line 1264)	112

9.18.24	Instance (priority (Line 1327)	112
9.18.25	Instance (priority (Line 1390)	112
9.18.26	Instance (priority (Line 1452)	112
9.18.27	Instance (priority (Line 1514)	113
9.18.28	Instance (priority (Line 1576)	113
9.18.29	Instance (priority (Line 1637)	113
9.18.30	Instance (priority (Line 1699)	113
9.18.31	Instance (priority (Line 1760)	113
9.18.32	Instance (priority (Line 1822)	113
9.18.33	Instance (priority (Line 1883)	113
9.18.34	Instance (priority (Line 1943)	114
9.18.35	Instance (priority (Line 2003)	114
9.18.36	Instance (priority (Line 2063)	114
9.18.37	Instance (priority (Line 2123)	114
9.19	File: Episode20.lean	114
9.19.1	Class EXECUTED_TRACE (Line 12)	114
9.19.2	Instance (priority (Line 18)	114
9.19.3	Instance (priority (Line 29)	115
9.19.4	Instance (priority (Line 49)	115
9.19.5	Instance (priority (Line 64)	115
9.19.6	Instance (priority (Line 79)	115
9.19.7	Instance (priority (Line 94)	115
9.19.8	Instance (priority (Line 109)	115
9.19.9	Instance (priority (Line 124)	115
9.19.10	Instance (priority (Line 139)	116
9.19.11	Instance (priority (Line 154)	116
9.19.12	Instance (priority (Line 169)	116
9.19.13	Instance (priority (Line 184)	116
9.19.14	Instance (priority (Line 199)	116
9.19.15	Instance (priority (Line 214)	116
9.19.16	Instance (priority (Line 229)	116
9.19.17	Instance (priority (Line 244)	117
9.19.18	Instance (priority (Line 259)	117
9.19.19	Instance (priority (Line 274)	117
9.19.20	Instance (priority (Line 289)	117
9.19.21	Instance (priority (Line 304)	117
9.19.22	Instance (priority (Line 319)	117
9.19.23	Instance (priority (Line 334)	117
9.19.24	Instance (priority (Line 349)	118
9.19.25	Instance (priority (Line 364)	118
9.19.26	Instance (priority (Line 379)	118
9.19.27	Instance (priority (Line 394)	118
9.19.28	Instance (priority (Line 409)	118
9.19.29	Instance (priority (Line 424)	118
9.19.30	Instance (priority (Line 439)	118
9.19.31	Instance (priority (Line 454)	119
9.19.32	Instance (priority (Line 469)	119
9.19.33	Instance (priority (Line 484)	119
9.19.34	Instance (priority (Line 499)	119

9.19.35 Instance (priority (Line 514))	119
9.19.36 Instance (priority (Line 529))	119
9.19.37 Instance (priority (Line 544))	119
9.19.38 Structure SlipVerdict (Line 564)	120
9.20 File: Episode21.lean	120
9.20.1 Structure BracketedNumber (Line 17)	120
9.21 File: Episode22.lean	120
9.21.1 Inductive QPhase (Line 14)	120
9.21.2 Structure RecoveredObservables (Line 34)	120
9.22 File: Episode25.lean	121
9.22.1 Inductive CompilerTapeCell (Line 23)	121
9.22.2 Inductive CorridorFace (Line 101)	121
9.22.3 Structure RotatedCorridorRead (Line 138)	121
9.22.4 Structure CorridorBinReport (Line 213)	122
9.22.5 Structure WheelStimulusRead (Line 243)	122
9.22.6 Structure BuildGravitySlipReport (Line 269)	123
9.23 File: Episode26.lean	123
9.23.1 Structure ApparatusRatio (Line 21)	123
9.23.2 Structure CavendishBalanceReport (Line 73)	123
9.24 File: Episode27.lean	124
9.24.1 Structure QuantumGravityCell (Line 73)	124
9.24.2 Structure AlphaTuneReport (Line 90)	124
9.25 File: Episode28.lean	125
9.25.1 Inductive ProximityRegime (Line 17)	125
9.25.2 Structure ProximityRead (Line 59)	126
9.25.3 Structure ProximityBreakReport (Line 99)	126
9.26 File: Episode29.lean	127
9.26.1 Structure RationalDistance (Line 21)	127
9.26.2 Structure SlipBisectStep (Line 72)	127
9.26.3 Structure SlipBisectBracket (Line 84)	127
9.26.4 Structure SlipBisectReport (Line 123)	128
9.26.5 Structure SlipBisectSummary (Line 159)	128
9.27 File: Episode30.lean	129
9.27.1 Structure SlipPointMoments (Line 86)	129
9.27.2 Structure ChargeMassNormalizationReport (Line 109)	129
9.28 File: Episode31.lean	130
9.28.1 Structure MagneticNeedleReport (Line 37)	130
9.29 File: Episode32.lean	130
9.29.1 Inductive CooperElectronOrbitRegime (Line 43)	130
9.29.2 Structure CooperElectronOrbitRead (Line 69)	131
9.29.3 Structure CooperElectronOrbitReport (Line 112)	131
9.30 File: Episode33.lean	132
9.30.1 Structure RationalRadius (Line 25)	132
9.30.2 Inductive LorentzOrbitRegime (Line 63)	132
9.30.3 Structure LorentzOrbitRead (Line 77)	132
9.30.4 Structure LorentzOrbitBracketReport (Line 113)	133
9.31 File: Episode34.lean	133
9.31.1 Inductive ApparatusLaw (Line 19)	133
9.31.2 Structure DiracExperimentReport (Line 73)	133

9.32	File: Episode35.lean	134
9.32.1	Structure SlipGridCell (Line 46)	134
9.32.2	Structure DyadicSlipBracket (Line 92)	135
9.32.3	Structure GridSlipBisectSummary (Line 139)	135
9.33	File: Episode36.lean	136
9.33.1	Structure ForceMeasuredGReport (Line 27)	136
9.34	File: Episode37.lean	137
9.34.1	Inductive ElaborationChargeOrientation (Line 15)	137
9.34.2	Structure ElectronElaborationChargeReport (Line 45)	137
9.35	File: Episode38.lean	137
9.35.1	Structure SlipPointChargeReport (Line 35)	137
9.36	File: Episode39.lean	138
9.36.1	Inductive ChargeSecondVariationChannel (Line 16)	138
9.36.2	Structure ChargeSecondVariationComponent (Line 23)	139
9.36.3	Structure ChargeSecondVariationReport (Line 57)	139
9.37	File: Episode40.lean	139
9.37.1	Structure AlphaSecondVariationReport (Line 40)	139
9.38	File: TwoDescriptions.lean	140
9.38.1	Inductive Spin (Line 26)	140
9.38.2	Inductive Orientation (Line 32)	141
9.38.3	Structure Leg (Line 38)	141

Chapter 1

Grouping A: The Ground Floor of Distinction (Episodes 1–5)

This grouping establishes the foundational vocabulary of the instrument: the ability to select (tange) and bag (funge) logical differences prior to any numeric operations.

1.1 The Symbol and Decidable Equality

In [Episode1.lean](#), the first fact is a type-level former for distinguishability: The typeclass `DISTINGUISHABLE` is parameterized by a universe type `Value` and an observation process `Observation`. It bundles a truth witness `fact : Fact`, a type-level label `symbol : Type Value`, a difference predicate `different? : Type Value Prop` mapping types to propositions, and a decidability witness `dec_distinct : DecidablePred different?`.

The key operation is extracting the universe level from `symbol`. Because type-distinctness is evaluated at compile time by the typechecker, difference carries zero runtime allocation cost.

1.2 Constructing Natural Numbers and Order

On this ground floor, standard natural numbers are constructed from first principles: The inductive type `Natural` represents natural numbers by stacking recursive steps: a base constructor `zero` mapping a proof witness `Fact` to `Natural`, and a recursive step constructor `number` mapping a proof witness `Fact`, a predecessor identifier `Number`, and a predecessor `Natural` to `Natural`.

The natural numbers are ordered via the relation `le`: The ordering relation `le` is a function mapping two `Natural` instances to a proposition (`Natural Natural Prop`). Zero is always less than or equal to any number, a successor is never less than or equal to zero, and comparisons between non-zero numbers are evaluated recursively by matching their structural predecessors. This is a constructive, type-theoretic definition of ordering: zero is less than or equal to everything, and successor comparison is performed recursively.

Chapter 2

Grouping B: Discrete Set and Representation Towers (Episodes 6–10)

This grouping refines distinguishability into countable sets and representation towers.

2.1 Admissibility and Countability

The typeclass constraints `ADMISSIBLE` and `COUNTABLE` restrict the carrier to types that are countably enumerable: The typeclass `ADMISSIBLE` extends `DISTINGUISHABLE` and carries a state predicate `is_discrete : DiscreteState Value`. The typeclass `COUNTABLE` extends `ADMISSIBLE` and defines a counting mapping: a function `enumeration : Nat Option Value` and a proof witness `surjective` showing that every value is mapped to a natural index.

This ensures that the state space is not a continuum, but a discrete set of separated directions that can be mapped onto the natural numbers.

2.2 Reals as Cauchy Towers

Reals are represented not as floating-point approximations, but as nested intervals (Cauchy towers): The structure `CauchyTower` represents reals as nested intervals. It contains three fields: `nested : Nat Interval Value` representing the sequence of intervals, a proof field `is_nested` verifying that intervals are nested, and a closure proof field `limit_closes` showing that the width of the intervals converges to zero as the depth index grows.

A real number is defined as a sequence of nested intervals whose width approaches zero. The difference between two nested levels of the Cauchy tower forms the basis of the physical remainder: the residue.

Chapter 3

Grouping C: Commutators, Transport, and Curvature (Episodes 11–15)

This grouping introduces parallel transport, loop holonomy, and the non-integrable left-over.

3.1 Covariant Connection and Transport

The covariant derivative connection ∇_μ^A defines how a spinor or field component is compared after being displaced along a path: The function `parallelTransport` maps a `Path`, a `Connection`, and a starting `State` to a final `State`. It computes this by iteratively adding the connection coefficients at each step of the path via a folding operation.

If the connection is flat, transporting a state along a loop returns the state unchanged. If the connection has curvature, transporting a state along a closed loop accumulates a non-zero holonomy residue:

$$\text{holonomy_loop_is_charged} : \text{holonomy}(\text{closed_loop}) \neq 0$$

3.2 The Un-Eliminable Residue

This loop non-closure is defined as the `RESIDUE`. In a classical code setup, small numerical remainders are discarded. In this system, attempting to discard this residue results in an ill-typed coercion. The residue is carried and will later be read as physical parameters (mass, charge, spin, phase, curvature).

Chapter 4

Grouping D: The Naming Corridor (Episodes 16–23)

This grouping walks back down the representation towers, isolating stable readings using transition corridors.

4.1 The Backward Descent

After building up the Cauchy tower and connection, the machine performs a backward descent (elimination after introduction): The function `backwardDescent` maps a depth index `n : Nat` and a `CauchyTower V` to a value in `V`. It extracts the midpoint of the interval at depth `n`.

Pass n seeds pass $n + 1$; the residue lives at the join, and the compiler is billed the turns.

4.2 Transition Corridors and Stable Readings

The stable readings are isolated by defining transition corridors that act as gates. A state is stable if it survives translation across the corridor: The structure `TransitionCorridor` is parameterized by `State`. It bundles a binary relation `gate : State State Prop` representing transition compatibility, and a stability predicate `stable? : State Prop` defaulting to the diagonal relation `gate s s`.

The stable reading represents the Weyl/traceless part of the curvature, which is named as the electron (-1) or positron ($+1$).

Chapter 5

Grouping E: Lorentz, Spin-1/2, and Dirac Operators (Episodes 24–30)

This grouping introduces the relativistic Dirac operator and its finite bookkeeping model.

5.1 The Dirac Operator

The Dirac operator $\gamma^\mu(\partial_\mu + iA_\mu) + m$ acts as a gauge-covariant first-order generator: The structure `DiracMode` carries three fields: `slot` : `Nat` (the spinor sector slot), `mass` : `Nat` (the rest mass term), and `gauged` : `Bool` (a boolean flag indicating if electromagnetic connection terms are carried).

The connection term `iA_mu` (when `gauged = true`) ensures that transport registers loop holonomies. Squaring the Dirac operator is modeled as a finite arithmetic check: The theorem `square_is_second_order` proves that the spinor slot satisfies the nilpotent squaring relation `slot * slot = 0` via compile-time evaluation (by `decide`).

5.2 The Admissibility Theorem

The admissibility of the Dirac operator requires both first-order slots and gauge covariance: The theorem `claim_holds` is a constructive witness of type `claim.statement`. It packages the proof that the Dirac mode is admissible, along with the proofs of trivial open holonomy, charged loop holonomy, and positron detection over spatial tilt. This binds the physical Dirac operator straight to the device’s actual holonomy and sign theorems.

Chapter 6

Grouping F: Proximity, Photoelectric, and Cavendish (Episodes 31–34)

This grouping models physical measurements in the finite state space.

6.1 Cavendish Balance and Proximity

The Cavendish balance is modeled as a finite torque system measuring the curvature of nested state boundaries: The function `cavendishTorue` maps a `RationalDistance` and a `mass : Nat` to a torque value represented as a natural number. It evaluates the force as the mass divided by the square of the distance’s numerator.

This represents the gravitational coupling, where the force falls off as the inverse square of the rational distance between state cells.

6.2 The Photoelectric Limit

The photoelectric limit defines the minimum energy/heartbeat cost required to excite a state and release a label: The function `photoelectricThreshold` maps an input `frequency : Nat` and a `workFunction : Nat` to an integer representing the residual kinetic energy. It subtracts the work function from the frequency.

If the frequency (elaboration effort) is below the work function (the EKG noise floor), the excitation fails and no label is generated.

Chapter 7

Grouping G: The Second Variation and Dimensionless Coupling (Episodes 35–40)

This grouping contracts the four-channel second variation of the field at the slip crossing, yielding the coupling constant α .

7.1 The Second Variation

The second variation δ^2 measures the curvature of the field along the slip boundary: The function `alphaFromSecondVariationAtDistance` maps a `targetSlip : Nat` and a `distance : RationalDistance` to an `ApparatusRatio` (the coupling α_c). It multiplies the target potential value at the distance by the field per charge at the slip boundary, and divides the product by the natural orbit radius constant.

This extracts the dimensionless coupling constant α_c :

$$\alpha = \left(1 - \frac{1}{r}\right) \cdot \frac{1}{r^2} \cdot \frac{1}{\text{orbitUnit}}$$

where $1/r$ is the potential channel, $1/r^2$ is the field-per-charge channel, and `orbitUnit` = 18.

7.2 Maxwell and Meissner Boundaries

The magnetic repulsion term carries a photon recoil response, and speed of light limits are enforced: The structure `AlphaSecondVariationReport` aggregates the results of the second variation. It contains fields for the normalization method, the natural orbit unit, the target slip index, flags for enforcing the Maxwell speed of light and carrying magnetic photon recoil, the computed coupling ratio `alpha`, and its scaled and inverse integer values.

Chapter 8

Grouping H: Surface Instruments and Calibration

This chapter details the surface modules and provides a rigorous analysis and proof of the Clifford shadow closure.

8.1 The Two Descriptions

One electron in orbit and the anti-Cooper-pair are one rotating object under two names. By propositional extensionality (`propext`), they are the same proposition: The theorem `two_descriptions` proves the identity of the two descriptions of orbit and pair orientations via propositional extensionality (`propext`).

8.2 The Clifford Shadow and its Closure

The Clifford shadow structure is defined in [Episode57.lean](#): The structure `CliffordShadow` is parameterized by type `H`. It bundles two binary relations, `dot : H → H → Int` (representing the metric pairing) and `anti : H → H → Int` (representing the anticommutator), and the relation axiom field `relation` asserting that $\text{anti } x \ y = -2 * \text{dot } x \ y$.

where the anticommutator ‘anti’ is twice the metric pairing ‘dot’.

8.2.1 Rigorous Proof of Algebra Closure

We prove that if the sectors are orthogonal, their Clifford anticommutator cross-term is zero: The theorem `orthogonal_clifford_scalar_cross_zero` proves that sector orthogonality ($\text{C.dot } g \ q = 0$) forces the anticommutator cross-term to vanish ($\text{C.anti } g \ q = 0$).

1. We assume the orthogonality hypothesis `horth : C.dot g q = 0`.
2. The tactic `rw [C.relation]` rewrites `C.anti g q` to $-2 * \text{C.dot } g \ q$ using the relation axiom.
3. The tactic `rw [horth]` substitutes 0 for `C.dot g q`, reducing the goal to $-2 * 0 = 0$.
4. The tactic `omega` solves the arithmetic, closing the proof.

This ensures that the cross-terms vanish, preventing interior mixed certificates ($\neg$ Has-InteriorScalarMixedCertificate) and forcing the remainder to appear as boundary flux (exactly the electron, -1).

8.3 Stern-Brocot Mediant Descent and the Open Jar

The continued-fraction periodic expansion of the quadratic surd $d^* = \sqrt{18/5} = [1; \overline{1, 8, 1, 2}]$ yields the convergents $1/1, 2/1, 17/9$. Evaluating the inverse-alpha values at these points yields the open jar:

$$129.6 \leq \alpha^{-1} \leq 137.7$$

This bracket is proven to be strictly ordered: The theorem `count3_bracket_ordered` proves that the count-three bracket limits are strictly ordered via compile-time decision.

Deeper convergents converge to the device's own value ≈ 137.011 .

8.4 Path Calibration and Finiteness Fence

Calibration verifies that the direct reading ($1 \rightarrow 3$) and the stepped BFGS descent ($1 \rightarrow 2 \rightarrow 3$) agree: The definitions `direct_1to3` and `stepped_1to2to3` compute the inverse-alpha values using the direct second variation report and the stepped BFGS optimization respectively.

The finiteness fence proves that the finite resolution cannot resolve the residue below the representation grid floor: The theorem `machine_cannot_resolve_residue` proves that the box representation function `read` cannot distinguish between a Gateaux variation and a Frechet variation below the grid floor.

This completes the logical cycle of the instrument.

Chapter 9

The Complete Ledger of Logical Declarations

This chapter lists every single `inductive`, `class`, `structure`, and `instance` declaration in the codebase from start to finish, providing a conceptual analysis of their types, fields, constructors, and logical relations. No code blocks are reproduced.

9.1 File: Episode1.lean

9.1.1 Class Fact (Line 75)

This declaration represents a logical typeclass constraint named `Fact`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`truth` → method signature: `Prop`

`decTruth` → method signature: `Decidable truth`

Implementation Commentary:

ME: I need a `Fact`.

COMPILER: A proposition?

ME: No, a proposition with a receipt.

COMPILER: That is called a proof. This is a measurement from a device of my own design, a ME: Great. Staple it to the universe. +————— clever device that gives us an idea of how much bullshit — the LEan elaborator is putting up with. We will be constructing

9.1.2 Inductive Number (Line 131)

This declaration represents a logical constructive sum type named `Number`. It represents a disjoint union populated by the following constructors:

`zero` → constructor signature: `Fact \ensuremath {\to } Number`

`one` → constructor signature: `Fact \ensuremath {\to } Number \ensuremath {\to } Number`

Implementation Commentary:

ME: I need the number to be bigger or smaller.

COMPILER: Bigger or smaller than what?

ME: Another number.

COMPILER: That is an order relation.

ME: No. That is a fact about two costumes agreeing which one is taller.

COMPILER: You want ‘j’ and ‘≤’.

ME: I want them wearing badges. +————— What you are looking at is an estimation of

9.1.3 Instance : LE Number where (Line 232)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Number where`.

Implementation Commentary:

BTW, I hope you don’t mind I hijack operators for my own nefarious purposes. If you forget what they mean, because hijack is really a euphemism for what I’m about to do, just remember it always means “Less Than or Equal To” in whatever context it is in.

9.1.4 Structure CarrierProcess (Line 239)

This declaration represents a logical record structure named `CarrierProcess`. It constructs records using the constructor `CarrierProcess.mk` with the following fields:

`Carrier` → type signature: `Type i`)

`symbol` → type signature: `Fact`

`value` → type signature: `Number`

Implementation Commentary:

So, this is our “sensor” for the compiler’s meaning of a `Type`. The compiler will happily send us carriers (I think they are called properties, but I have never seen a language spec) that will evaluate true/false for us to use as “facts.”

9.1.5 Class DISTINGUISHABLE (Line 277)

This declaration represents a logical typeclass constraint named `DISTINGUISHABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i`)

`Observation` → method signature: `CarrierProcess Value`)

`fact` → method signature: `Fact`

`symbol` → method signature: `Type Value`

`dec\_distinct` → method signature: `DecidablePred different?`

Implementation Commentary:

ME: Before I count, I need difference.

COMPILER: Difference from what?

ME: From itself, eventually. But start small.

COMPILER: You want decidable equality.

ME: I want a witness that the costumes are not the same costume.

COMPILER: That is decidable equality.

ME: Fine. But with better lighting.

9.1.6 Inductive Natural (Line 330)

This declaration represents a logical constructive sum type named `Natural`. It represents a disjoint union populated by the following constructors:

`zero` → constructor signature: `Fact \ensuremath {\to } Natural`

`number` → constructor signature: `Fact \ensuremath {\to } Number \ensuremath {\to } Natural \ensuremath {\to } Natural`

Implementation Commentary:

ME: I need a natural number.

COMPILER: Great. Use `Nat`.

ME: No. Too much civilization.

COMPILER: Then what do you want?

ME: I want to tell one from the rest.

COMPILER: That is barely counting.

ME: Exactly. We are still in the cave drawing phase.

9.1.7 Instance : LE Natural where (Line 362)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Natural where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.8 Instance : LT Natural where (Line 366)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Natural where`.

Implementation Commentary:

More notional abuse. Won't someone think of the notation?

9.1.9 Structure CountingProcess (Line 393)

This declaration represents a logical record structure named `CountingProcess`. It constructs records using the constructor `CountingProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`object` → type signature: `DISTINGUISHABLE Value Carrier]`

`carrier` → type signature: `CarrierProcess Value`

`count` → type signature: `Natural`

Implementation Commentary:

ME: I need a counting process.

COMPILER: Count the things?

ME: Or count what makes up the things.

COMPILER: That is decomposition.

ME: Good. Put it next to enumeration.

COMPILER: Why?

ME: Eventually the pieces become things and demand representation.

9.1.10 Class ADMISSIBLE (Line 476)

This declaration represents a logical typeclass constraint named `ADMISSIBLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`

`counting\_process` $\rightarrow$ method signature: `CountingProcess Value Carrier`

Implementation Commentary:

ME: Counting only goes up.

COMPILER: Then why do your pieces get smaller?

ME: Because I changed what we are counting.

COMPILER: That sounds illegal.

ME: No. That is admissible.

COMPILER: Admissible to whom?

ME: To the representation that remembered which way was forward.

9.1.11 Inductive Rational (Line 530)

This declaration represents a logical constructive sum type named `Rational`. It represents a disjoint union populated by the following constructors:

`zero` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Rational`

`number` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Natural \ensuremath{\{\to\}} Rational \ensuremath{\{\to\}} Rational`

Implementation Commentary:

ME: I need fractions.

COMPILER: For division?

ME: Not yet. For pieces.

COMPILER: Pieces of what?

ME: The thing we just finished pretending was one.

COMPILER: So one was provisional.

ME: All measurement is provisional. Please initialize the saw.

9.1.12 Instance : LE Rational where (Line 582)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Rational where`.

Implementation Commentary:

Let's make the symbols work.

9.1.13 Instance : LT Rational where (Line 585)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Rational where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.14 Structure IndexingProcess (Line 613)

This declaration represents a logical record structure named `IndexingProcess`. It constructs records using the constructor `IndexingProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i`)
`Carrier` $\rightarrow$ type signature: `CarrierProcess Value`)
`count` $\rightarrow$ type signature: `CountingProcess Value Carrier`
`origin` $\rightarrow$ type signature: `Rational`

Implementation Commentary:

ME: Keep track of the uncounted numbers.

COMPILER: If they are numbers, haven't they been counted?

ME: That is adorable.

COMPILER: I dislike your tone.

ME: Most numbers have not been counted. That is why we need an index.

COMPILER: I thought the index was the counting.

ME: No. The index is the little fence around what we are allowed to pretend we counted.

9.1.15 Class COUNTABLE (Line 697)

This declaration represents a logical typeclass constraint named `COUNTABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i`)
`Carrier` $\rightarrow$ method signature: `CarrierProcess Value`)
`index` $\rightarrow$ method signature: `IndexingProcess Value Carrier`

Implementation Commentary:

ME: Count up until we need a fraction.

COMPILER: Need one for what?

ME: To describe the next thing without pretending it is whole.

COMPILER: So the counting broke.

ME: No. The unit broke.

COMPILER: That sounds worse.

ME: That is called a Dedekind cut when it dresses formally.

9.1.16 Inductive Sequence (Line 800)

This declaration represents a logical constructive sum type named `Sequence`. It represents a disjoint union populated by the following constructors:

`nil` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Sequence`
`index` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Rational \ensuremath{\{\to\}} Sequence \ensuremath{\{\to\}} Sequence`

Implementation Commentary:

ME: I need a sequence.

COMPILER: Of counted numbers?

ME: Of next numbers.

COMPILER: That sounds the same.

ME: That is adorable.

COMPILER: I am filing a complaint.

ME: A sequence is not what we counted. It is the rule that keeps finding the next admissible thing.

9.1.17 Instance : LE Sequence where (Line 849)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Sequence where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.18 Instance : LT Sequence where (Line 852)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Sequence where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.19 Structure LimitProcess (Line 894)

This declaration represents a logical record structure named `LimitProcess`. It constructs records using the constructor `LimitProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`count` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`indexing\_process` $\rightarrow$ type signature: `IndexingProcess Value Carrier`

`limit` $\rightarrow$ type signature: `Rational`

`sequence` $\rightarrow$ type signature: `Sequence`

Implementation Commentary:

ME: I need a limit process.

COMPILER: A sequence that approaches a value.

ME: Look at you.

COMPILER: That is correct, isn't it?

ME: Almost dangerously correct.

COMPILER: I dislike the adverb.

ME: The value is not the point. The approaching is the machinery.

9.1.20 Class ENCODED (Line 970)

This declaration represents a logical typeclass constraint named `ENCODED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i`)

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value`)

`c` $\rightarrow$ method signature: `COUNTABLE Value Carrier`]

`limit\_process` $\rightarrow$ method signature: `LimitProcess Value Carrier`

Implementation Commentary:

ME: I need it encoded.

COMPILER: Into what?

ME: A coin.

COMPILER: You want to reduce the limit process to a token?

ME: I want something that drops.

COMPILER: Drops where?

ME: Into the machine that pretends tokens are meaning.

9.1.21 Inductive Limit (Line 1011)

This declaration represents a logical constructive sum type named `Limit`. It represents a disjoint union populated by the following constructors:

`nil` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Limit`

`index` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Sequence \ensuremath{\{\to\}} Limit \ensuremath{\{\to\}} Limit`

Implementation Commentary:

ME: I need a limit process.

COMPILER: I do not know nuthin 'bout no calculus.

ME: Perfect.

COMPILER: Perfect?

ME: I do not need you to know calculus. I need you to watch a sequence keep missing.

COMPILER: Missing what?

ME: The thing we will later pretend it was approaching.

9.1.22 Instance : LE Limit where (Line 1067)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Limit where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.23 Instance : LT Limit where (Line 1070)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Limit where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.24 Structure CauchyProcess (Line 1095)

This declaration represents a logical record structure named `CauchyProcess`. It constructs records using the constructor `CauchyProcess.mk` with the following fields:

`Value` → type signature: `Type i)`
`Carrier` → type signature: `CarrierProcess Value)`
`e` → type signature: `ENCODED Value Carrier]`
`limit\_process` → type signature: `LimitProcess Value Carrier`
`value` → type signature: `Rational`
`accumulation` → type signature: `Limit`

Implementation Commentary:

ME: I need a Cauchy process.

COMPILER: A sequence that converges.

ME: Careful, that has ontology on it.

COMPILER: Then what is it?

ME: A crowd of terms agreeing to stop wandering apart.

COMPILER: How do they know how far apart they are?

ME: Do not worry. Pythagoras is still backstage.

9.1.25 Class RESIDUE (Line 1139)

This declaration represents a logical typeclass constraint named `RESIDUE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`
`Carrier` → method signature: `CarrierProcess Value)`
`d` → method signature: `DISTINGUISHABLE Value Carrier]`
`a` → method signature: `ADMISSIBLE Value Carrier]`
`c` → method signature: `COUNTABLE Value Carrier]`
`e` → method signature: `ENCODED Value Carrier]`
`cauchy\_process` → method signature: `CauchyProcess Value Carrier`

Implementation Commentary:

ME: I need a residue.

COMPILER: A what?

ME: What is left after the process almost explains itself.

COMPILER: That is not a thing.

ME: That is adorable.

COMPILER: Left over from what?

ME: From the difference between the story and the measurement.

COMPILER: I do not like measuring leftovers.

ME: Then stop making them.

9.1.26 Inductive Sample (Line 1187)

This declaration represents a logical constructive sum type named `Sample`. It represents a disjoint union populated by the following constructors:

`initial\_condition` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Limit \ensuremath{\{\to\}} Sample`

`signal\_response` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Limit \ensuremath{\{\to\}} Fact \ensuremath{\{\to\}} Limit \ensuremath{\{\to\}} Sample \ensuremath{\{\to\}} Sample`

Implementation Commentary:

ME: I need a sample.

COMPILER: A point from the process?

ME: A bite from the residue.

COMPILER: That sounds unsanitary.

ME: Measurement usually is.

COMPILER: What does the sample prove?

ME: Nothing. It gives the next machine something small enough to lie about.

9.1.27 Instance : LE Sample where (Line 1241)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Sample where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.1.28 Instance : LT Sample where (Line 1244)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Sample where`.

Implementation Commentary:

Bullshit meter ≈ 5

9.2 File: Episode2.lean

9.2.1 Structure ObservationProcess (Line 139)

This declaration represents a logical record structure named `ObservationProcess`. It constructs records using the constructor `ObservationProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` → type signature: `RESIDUE Value Carrier]`

`cauchy\_process` → type signature: `CauchyProcess Value Carrier`

`before` → type signature: `Limit`

`after` → type signature: `Limit`

Implementation Commentary:

ME: I need an observation process.

COMPILER: We sampled the residue.

ME: Yes, but nobody saw us do it.

COMPILER: I am the compiler.

ME: You produced the sample. Consider this a deposition.

9.2.2 Class BINARY (Line 243)

This declaration represents a logical typeclass constraint named `BINARY`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`

`Carrier` → method signature: `CarrierProcess Value)`

`d` → method signature: `DISTINGUISHABLE Value Carrier]`

`a` → method signature: `ADMISSIBLE Value Carrier]`

`c` → method signature: `COUNTABLE Value Carrier]`

`e` → method signature: `ENCODED Value Carrier]`

`r` → method signature: `RESIDUE Value Carrier]`

`observation\_process` → method signature: `ObservationProcess Value Carrier`

`zero` → method signature: `Limit`

`one` → method signature: `Limit`

`bit` → method signature: `Sample`

Implementation Commentary:

ME: I need different.

COMPILER: Different from what?

ME: From itself after the instrument answers.

COMPILER: Initial conditions can become responses?

ME: They can precede them.

COMPILER: Can responses become initial conditions?

ME: Not without perjury.

9.2.3 Inductive Trial (Line 381)

This declaration represents a logical constructive sum type named `Trial`. It represents a disjoint union populated by the following constructors:

`hypothesis` $\rightarrow$ constructor signature: $\text{Fact} \rightarrow \text{Sample} \rightarrow \text{Trial}$

`signal\_response` $\rightarrow$ constructor signature: $\text{Fact} \rightarrow \text{Sample} \rightarrow \text{Fact} \rightarrow \text{Sample} \rightarrow \text{Trial}$

Implementation Commentary:

ME: I need a trial.

COMPILER: We already observed the sample.

ME: Once.

COMPILER: Once is not enough?

ME: Once is gossip.

COMPILER: Then what is a trial?

ME: A sample with the nerve to be repeatable.

9.2.4 Instance : `LE Trial := <Trial.le>` (Line 442)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Trial := <Trial.le>`.

Implementation Commentary:

High syntax corn syrup: This stuff is `__BAD__` for your health. You only get `__REALLY__` confusing code out of it. Do not recommend.

9.2.5 Instance : `LT Trial := <Trial.lt>` (Line 443)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Trial := <Trial.lt>`.

Implementation Commentary:

Bullshit meter ≈ 8

9.2.6 Structure `RepeatableProcess` (Line 505)

This declaration represents a logical record structure named `RepeatableProcess`. It constructs records using the constructor `RepeatableProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

```

r → type signature: RESIDUE Value Carrier]
b → type signature: BINARY Value Carrier]
observation\_process → type signature: ObservationProcess Value Carrier
stimulus → type signature: Sample
expectation → type signature: Trial

```

Implementation Commentary:

ME: I need a repeatable process.

COMPILER: Run the trial again?

ME: And have it still be the same kind of trial.

COMPILER: Same result?

ME: Easy. Same question.

COMPILER: That seems weaker.

ME: It is stronger. Results are cheap. Questions need fixtures.

9.2.7 Class REPEATABLE (Line 577)

This declaration represents a logical typeclass constraint named `REPEATABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)
Carrier → method signature: CarrierProcess Value)
d → method signature: DISTINGUISHABLE Value Carrier]
a → method signature: ADMISSIBLE Value Carrier]
c → method signature: COUNTABLE Value Carrier]
e → method signature: ENCODED Value Carrier]
r → method signature: RESIDUE Value Carrier]
b → method signature: BINARY Value Carrier]
repeatable\_process → method signature: RepeatableProcess Value Carrier

```

Implementation Commentary:

ME: I need `REPEATABLE`.

COMPILER: You just defined a repeatable process.

ME: Yes.

COMPILER: So this repeats the repeatability?

ME: But I repeat myself.

COMPILER: That is not a proof.

ME: No. It is a hint.

9.2.8 Inductive Study (Line 712)

This declaration represents a logical constructive sum type named `Study`. It represents a disjoint union populated by the following constructors:

`hypothesis` $\rightarrow$ constructor signature: $\text{Fact} \rightarrow \text{Study}$

`data` $\rightarrow$ constructor signature: $\text{Fact} \rightarrow \text{Trial} \rightarrow \text{Study}$

Implementation Commentary:

ME: I need a study.

COMPILER: We have repeatable trials.

ME: Yes. Now put them in a folder.

COMPILER: A folder is not science.

ME: Correct. It is where science starts hiding receipts.

COMPILER: What does the study prove?

ME: Nothing yet. It keeps the questions from wandering off.

9.2.9 Instance : LE Study := ⟨Study.le⟩ (Line 765)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Study := ⟨Study.le⟩`.

Implementation Commentary:

Bullshit meter ≈ 5

9.2.10 Structure ComputationalProcess (Line 801)

This declaration represents a logical record structure named `ComputationalProcess`. It constructs records using the constructor `ComputationalProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`e` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`repeatable\_process` $\rightarrow$ type signature: `RepeatableProcess Value Carrier`

`output` $\rightarrow$ type signature: `Option Study`

Implementation Commentary:

ME: I need a computational process.

COMPILER: A program?

ME: No. The conditions under which a study can be made to act like one.

COMPILER: Studies do not run.

ME: They do when the questions have inputs and the receipts have outputs.

9.2.11 Class NUMERIC (Line 851)

This declaration represents a logical typeclass constraint named `NUMERIC`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`

`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ method signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ method signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ method signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ method signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ method signature: `BINARY Value Carrier]`

`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier]`

`computational\_process` $\rightarrow$ method signature: `ComputationalProcess Value Carrier`

Implementation Commentary:

ME: I need `NUMERIC`.

COMPILER: Now it is a number?

ME: No. Now it can be treated numerically.

COMPILER: That sounds like a number.

ME: That sounds like liability.

COMPILER: What is the difference?

ME: A number knows what it is. `NUMERIC` knows how it may be used.

9.3 File: Episode3.lean**9.3.1 Inductive Computation (Line 68)**

This declaration represents a logical constructive sum type named `Computation`. It represents a disjoint union populated by the following constructors:

`program` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Study \ensuremath{\{\to\}}`
`} Computation`

`compute` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Study \ensuremath{\{\to\}}`
`} Study \ensuremath{\{\to\}} Computation \ensuremath{\{\to\}} Computation`

Implementation Commentary:

ME: I need a computation.

COMPILER: The process already computes.

ME: Processes are engines. Computations are itineraries.

COMPILER: Input, output, route?

ME: And the fantasy that the luggage is the same object after the layover.

COMPILER: Finally, a theory of airports.

9.3.2 Instance : LE Computation := $\langle$ Computation.le $\rangle$ (Line 99)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Computation := \ensuremath {\langle`Computation.le`\ensuremath {\rangle}`.

Implementation Commentary:

This is starting to look more and more like $\leq$ and $\leq$, isn't it?

9.3.3 Instance : LT Computation := $\langle$ Computation.lt $\rangle$ (Line 100)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Computation := \ensuremath {\langle`Computation.lt`\ensuremath {\rangle}`.

Implementation Commentary:

Bullshit meter ≈ 8

9.3.4 Structure TuringProcess (Line 120)

This declaration represents a logical record structure named `TuringProcess`. It constructs records using the constructor `TuringProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`computational\_process` $\rightarrow$ type signature: `ComputationalProcess Value Carrier`

`program` $\rightarrow$ type signature: `Study`

`state` $\rightarrow$ type signature: `Computation`

Implementation Commentary:

ME: I need a Turing process.

COMPILER: I can process symbols.

ME: Not the comments.

COMPILER: Comments are not symbols.

ME: Correct. They are where the crime confesses.

COMPILER: I cannot read them.

ME: Exactly. That is why you cannot comment on them.

9.3.5 Class REPRESENTABLE (Line 178)

This declaration represents a logical typeclass constraint named `REPRESENTABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`

`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ method signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ method signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ method signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ method signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ method signature: `BINARY Value Carrier]`

`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ method signature: `NUMERIC Value Carrier]`

`calculation\_process` $\rightarrow$ method signature: `TuringProcess Value Carrier`

`representable?` $\rightarrow$ method signature: $\text{\ensuremath{\{\forall\}} (_ : \text{Fact}) (s : \text{Study}),}$
 $\text{\ensuremath{\{\exists\}} (_ : \text{Computation}), calculation_process.computational_process}$
 $s = s$

Implementation Commentary:

ME: I need `REPRESENTABLE`.

COMPILER: I can do that. I am a kind of ventriloquist.

ME: You make symbols speak?

COMPILER: I make values speak through symbols.

ME: Interesting.

COMPILER: Why are you looking at my hand?

ME: No reason. Continue.

9.3.6 Inductive ChaitinsNumberSequence (Line 226)

This declaration represents a logical constructive sum type named `ChaitinsNumberSequence`. It represents a disjoint union populated by the following constructors:

`halting` $\rightarrow$ constructor signature: `Fact \ensuremath {\to } Computation \ensuremath {\to } ChaitinsNumberSequence`

`nonhalting` $\rightarrow$ constructor signature: `Fact \ensuremath {\to } Computation \ensuremath {\to } Option ChaitinsNumberSequence \ensuremath {\to } ChaitinsNumberSequence`

Implementation Commentary:

ME: I need Chaitin's number sequence.

COMPILER: Absolutely not.

ME: I am not asking you to "compute" it.

COMPILER: You named it.

ME: I am asking you to agree this kind of sequence could exist in theory.

COMPILER: "In theory" is doing a lot of work.

ME: Perfect. Give it a hard hat.

9.3.7 Instance : LE ChaitinsNumberSequence := ⟨ChaitinsNumberSequence.le⟩ (Line 253)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE ChaitinsNumberSequence := \ensuremath {\langle } ChaitinsNumberSequence.le \ensuremath {\rangle }`.

Implementation Commentary:

Bullshit meter ≈ 5

9.3.8 Structure NoisyProcess (Line 279)

This declaration represents a logical record structure named `NoisyProcess`. It constructs records using the constructor `NoisyProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ type signature: `REPRESENTABLE Value Carrier]`

turing_process → type signature: TuringProcess Value Carrier

program → type signature: ChaitinsNumberSequence

Implementation Commentary:

ME: I need a NoisyProcess.

COMPILER: Define noise.

ME: Niose.

COMPILER: That is misspelled.

ME: Excellent. It arrived corrupted.

COMPILER: I cannot accept corrupted input.

ME: Sure you can. You just called it input.

9.3.9 Class PHYSICAL (Line 355)

This declaration represents a logical typeclass constraint named **PHYSICAL**. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier]

a → method signature: ADMISSIBLE Value Carrier]

c → method signature: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier]

r → method signature: RESIDUE Value Carrier]

b → method signature: BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier]

n → method signature: NUMERIC Value Carrier]

h → method signature: REPRESENTABLE Value Carrier]

noisy_process → method signature: NoisyProcess Value Carrier

threshold → method signature: ChaitinsNumberSequence

admissible? → method signature: $\text{\ensuremath{\{\forall\}}}$ (f: Fact) (s: Study),
noisy_process.next_representation? noisy_process.program =

Implementation Commentary:

ME: I need PHYSICAL.

COMPILER: The signal is corrupted.

ME: Good. What made it through?

COMPILER: That depends on the instrument.

ME: Welcome to physics.

COMPILER: That is not a definition.

ME: No. It is a burn mark with units.

9.3.10 Inductive Metavariable (Line 420)

This declaration represents a logical constructive sum type named `Metavariable`. It represents a disjoint union populated by the following constructors:

`base` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} \ensuremath{\{\alpha\}} \ensuremath{\{\to\}} Metavariable \ensuremath{\{\alpha\}}`

`step` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Metavariable \ensuremath{\{\alpha\}} \ensuremath{\{\to\}} Metavariable \ensuremath{\{\alpha\}}`

Implementation Commentary:

ME: I need a metavariable.

COMPILER: A variable for a variable?

ME: A placeholder with ambition.

COMPILER: Ambition is not a type.

ME: No. It is what happens before elaboration finds a costume.

COMPILER: So it does not know what it is yet.

ME: Correct. But it knows where the hole is.

9.3.11 Structure `PhysicalProcess` (Line 454)

This declaration represents a logical record structure named `PhysicalProcess`. It constructs records using the constructor `PhysicalProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ type signature: `REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ type signature: `PHYSICAL Value Carrier]`

`physical\_process` $\rightarrow$ type signature: `NoisyProcess Value Carrier]`

`representation` $\rightarrow$ type signature: `d.symbol`

`invariant` $\rightarrow$ type signature: `Metavariable (d.symbol)`

`value` $\rightarrow$ type signature: `Metavariable (ULift d.symbol)`

Implementation Commentary:

ME: I need a physical process.

COMPILER: We already defined PHYSICAL.

ME: That was the burn mark. This is the thing that keeps making them.

COMPILER: A process that survives noise?

ME: A process that leaves comparable scars.

COMPILER: Scars are not data.

ME: Not until the instrument learns to count them.

9.3.12 Class COMPARABLE (Line 521)

This declaration represents a logical typeclass constraint named `COMPARABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`

`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ method signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ method signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ method signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ method signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ method signature: `BINARY Value Carrier]`

`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ method signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ method signature: `REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ method signature: `PHYSICAL Value Carrier]`

`physical\_process` $\rightarrow$ method signature: `PhysicalProcess Value Carrier`

`smaller\_than` $\rightarrow$ method signature: `Metavariable d.symbol \ensuremath {\to } Metavariable d.symbol \ensuremath {\to } Prop`

Implementation Commentary:

ME: I need comparable.

COMPILER: So equality?

ME: No. Equality is too expensive.

COMPILER: Then what is comparison?

ME: A controlled failuer 2 conphyouse too measurements.

COMPILER: **FAILURE**

ME: Exactly. Controlled.

9.3.13 Inductive Sophism (Line 574)

This declaration represents a logical constructive sum type named `Sophism`. It represents a disjoint union populated by the following constructors:

`origin` → constructor signature: `Fact \ensuremath {\to } ChaitinsNumberSequence \ensuremath {\to } Type \ensuremath {\to } Sophism`

`dimension` → constructor signature: `Fact \ensuremath {\to } ChaitinsNumberSequence \ensuremath {\to } Type i \ensuremath {\to } Sophism \ensuremath {\to } Sophism`

Implementation Commentary:

ME: I need Sophism.

COMPILER: Aren't all numbers imaginary?

ME: That depends who benefits from the confusion.

COMPILER: They are just symbols.

ME: So are subpoenas.

COMPILER: Symbols can have consequences.

ME: Welcome to sophism with units.

9.3.14 Structure SlipProcess (Line 612)

This declaration represents a logical record structure named `SlipProcess`. It constructs records using the constructor `SlipProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`d` → type signature: `DISTINGUISHABLE Value Carrier]`

`a` → type signature: `ADMISSIBLE Value Carrier]`

`c` → type signature: `COUNTABLE Value Carrier]`

`e` → type signature: `ENCODED Value Carrier]`

`r` → type signature: `RESIDUE Value Carrier]`

`b` → type signature: `BINARY Value Carrier]`

`f` → type signature: `REPEATABLE Value Carrier]`

`n` → type signature: `NUMERIC Value Carrier]`

`h` → type signature: `REPRESENTABLE Value Carrier]`

`p` → type signature: `PHYSICAL Value Carrier]`

`z` → type signature: `COMPARABLE Value Carrier]`

`physical\_process` → type signature: `PhysicalProcess Value Carrier`

`projection` → type signature: `Sophism`

`stress` → type signature: `ChaitinsNumberSequence`

`threshold` → type signature: `Type`

Implementation Commentary:

ME: I need a `SlipProcess`.

COMPILER: A process that makes mistakes?

ME: Not mistakes. Slips.

COMPILER: Difference?

ME: A mistake is wrong. A slip reveals the floor.

COMPILER: The floor?

ME: The hidden surface the symbol was standing on.

9.3.15 Class OBSERVED (Line 661)

This declaration represents a logical typeclass constraint named `OBSERVED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i`)

`Carrier` → method signature: `CarrierProcess Value`)

`d` → method signature: `DISTINGUISHABLE Value Carrier`]

`a` → method signature: `ADMISSIBLE Value Carrier`]

`c` → method signature: `COUNTABLE Value Carrier`]

`e` → method signature: `ENCODED Value Carrier`]

`r` → method signature: `RESIDUE Value Carrier`]

`b` → method signature: `BINARY Value Carrier`]

`f` → method signature: `REPEATABLE Value Carrier`]

`n` → method signature: `NUMERIC Value Carrier`]

`h` → method signature: `REPRESENTABLE Value Carrier`]

`p` → method signature: `PHYSICAL Value Carrier`]

`z` → method signature: `COMPARABLE Value Carrier`]

`slip\_process` → method signature: `SlipProcess Value Carrier`

`observation` → method signature: `Type i`

Implementation Commentary:

ME: I need `OBSERVED.possible a b` inhabited.

COMPILER: So `a` becomes `b`?

ME: No.

COMPILER: So `a` equals `b`?

ME: No.

COMPILER: Then what am I certifying?

ME: That the computation is cromulent.

COMPILER: Cromulent is not a theorem.

ME: Yet.

9.3.16 Inductive Area (Line 738)

This declaration represents a logical constructive sum type named **Area**. It represents a disjoint union populated by the following constructors:

$t \rightarrow$ constructor signature: $\text{Fact } \mathbb{N} \rightarrow \text{Area}$

$dt \rightarrow$ constructor signature: $\text{Fact } \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Area} \rightarrow \text{Area}$

Implementation Commentary:

ME: I need Area.

COMPILER: Length times width?

ME: Not yet. That is the children's menu.

COMPILER: Then what is area?

ME: What observation leaves when counting stops pretending it is one-dimensional.

COMPILER: So two directions.

ME: At least two witnesses agreeing to share a floor.

9.4 File: Episode4.lean

9.4.1 Instance : LE Area := ⟨Area.le⟩ (Line 111)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Area := ⟨Area.le⟩`.

Implementation Commentary:

Bullshit meter ≈ 8

9.4.2 Structure SensingProcess (Line 128)

This declaration represents a logical record structure named **SensingProcess**. It constructs records using the constructor `SensingProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ type signature: `REPRESENTABLE Value Carrier]`

`p` → type signature: PHYSICAL Value Carrier]
`z` → type signature: COMPARABLE Value Carrier]
`particle` → type signature: OBSERVED Value Carrier]
`static\_fraction` → type signature: SlipProcess Value Carrier
`accumulation` → type signature: Area

Implementation Commentary:

ME: I need a sensing process.

COMPILER: What does it sense?

ME: Every strike.

COMPILER: The hammer?

ME: The return.

COMPILER: So it records vibration.

ME: It keeps count of what the rail refuses to forget.

9.4.3 Class PRESENT (Line 173)

This declaration represents a logical typeclass constraint named PRESENT. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier]
`a` → method signature: ADMISSIBLE Value Carrier]
`c` → method signature: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier]
`r` → method signature: RESIDUE Value Carrier]
`b` → method signature: BINARY Value Carrier]
`f` → method signature: REPEATABLE Value Carrier]
`n` → method signature: NUMERIC Value Carrier]
`h` → method signature: REPRESENTABLE Value Carrier]
`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]
`particle` → method signature: OBSERVED Value Carrier]
`santa\_claus` → method signature: SensingProcess Value Carrier
`quantum` → method signature: Type Area

Implementation Commentary:

ME: I need the present.

COMPILER: The current time?

ME: The accumulated return.

COMPILER: That is not now.

ME: It is if now is what the instrument has not forgotten yet.

COMPILER: So the present is memory?

ME: Local memory with a deadline. Ask Santa.

9.4.4 Inductive Phenomenon (Line 217)

This declaration represents a logical constructive sum type named **Phenomenon**. It represents a disjoint union populated by the following constructors:

field $\rightarrow$ constructor signature: $\text{Fact } \mathcal{A} \rightarrow \text{Area } \mathcal{A} \rightarrow \text{Phenomenon}$

initial_condition $\rightarrow$ constructor signature: $\text{Fact } \mathcal{A} \rightarrow \text{Area } \mathcal{A} \rightarrow \text{Phenomenon} \rightarrow \text{Phenomenon}$

observations $\rightarrow$ constructor signature: $\text{Fact } \mathcal{A} \rightarrow \text{Area } \mathcal{A} \rightarrow \text{Phenomenon} \rightarrow \text{Phenomenon} \rightarrow \text{Phenomenon}$

Implementation Commentary:

ME: I need a phenomenon.

COMPILER: What happened?

ME: The rail answered.

COMPILER: That is an event.

ME: The hammer returned, the rail remembered, the clock disagreed.

COMPILER: That is more than an event.

ME: Now you are hearing the phenomenon.

9.4.5 Instance : LE Phenomenon := ⟨Phenomenon.le⟩ (Line 260)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by **instance : LE Phenomenon := ⟨Phenomenon.le⟩**.

Implementation Commentary:

Bullshit meter ≈ 8

9.4.6 Structure GaugeProcess (Line 287)

This declaration represents a logical record structure named **GaugeProcess**. It constructs records using the constructor **GaugeProcess.mk** with the following fields:

Value $\rightarrow$ type signature: $\text{Type } i$

Carrier $\rightarrow$ type signature: $\text{CarrierProcess Value}$

d $\rightarrow$ type signature: $\text{DISTINGUISHABLE Value Carrier}$

a $\rightarrow$ type signature: $\text{ADMISSIBLE Value Carrier}$

`c` → type signature: COUNTABLE Value Carrier]
`e` → type signature: ENCODED Value Carrier]
`r` → type signature: RESIDUE Value Carrier]
`b` → type signature: BINARY Value Carrier]
`f` → type signature: REPEATABLE Value Carrier]
`n` → type signature: NUMERIC Value Carrier]
`h` → type signature: REPRESENTABLE Value Carrier]
`p` → type signature: PHYSICAL Value Carrier]
`z` → type signature: COMPARABLE Value Carrier]
`particle` → type signature: OBSERVED Value Carrier]
`frequency` → type signature: PRESENT Value Carrier]
`sensing\_process` → type signature: SensingProcess Value Carrier
`clock` → type signature: Phenomenon

Implementation Commentary:

ME: I need a GaugeProcess.

COMPILER: A meter?

ME: A gauge.

COMPILER: A meter reads.

ME: A gauge negotiates.

COMPILER: Negotiates what?

ME: How much chaos the field can afford before Yang starts asking why Mills is billing by the curvature.

9.4.7 Class MEASURABLE (Line 334)

This declaration represents a logical typeclass constraint named `MEASURABLE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier]
`a` → method signature: ADMISSIBLE Value Carrier]
`c` → method signature: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier]
`r` → method signature: RESIDUE Value Carrier]
`b` → method signature: BINARY Value Carrier]

`f` → method signature: REPEATABLE Value Carrier]
`n` → method signature: NUMERIC Value Carrier]
`h` → method signature: REPRESENTABLE Value Carrier]
`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]
`particle` → method signature: OBSERVED Value Carrier]
`frequency` → method signature: PRESENT Value Carrier]
`gauge\_process` → method signature: GaugeProcess Value Carrier

Implementation Commentary:

ME: I need measurable.

COMPILER: Use the gauge.

ME: The gauge does not measure.

COMPILER: It reads.

ME: It bargains.

COMPILER: Then what makes it measurable?

ME: The bargain leaves a number-shaped scar.

9.4.8 Inductive Jar (Line 398)

This declaration represents a logical constructive sum type named `Jar`. It represents a disjoint union populated by the following constructors:

`color` → constructor signature: $\text{Fact } \text{Area} \rightarrow \text{Jar}$
`bang` → constructor signature: $\text{Fact } \text{Jar} \rightarrow \text{Jar}$
`superposition` → constructor signature: $\text{Fact } \text{Jar} \rightarrow \text{Jar}$

Implementation Commentary:

ME: I need Jar Jar Binks.

COMPILER: Absolutely not.

ME: Relax. First, just the jar.

COMPILER: What is in it?

ME: That is the point.

COMPILER: So the jar contains unknown contents?

ME: Worse. It contains contents that have not agreed which reading they are yet.

COMPILER: That sounds like superposition.

ME: Good. Now put a lid on it.

9.4.9 Instance : LE Jar := ⟨Jar.le⟩ (Line 448)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Jar := ⟨Jar.le⟩`.

Implementation Commentary:

Bullshit meter ≈ 5 . I mean, c'mon compiler. I'm confident my

9.4.10 Structure MeesaProcess (Line 481)

This declaration represents a logical record structure named `MeesaProcess`. It constructs records using the constructor `MeesaProcess.mk` with the following fields:

```
Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]
a → type signature: ADMISSIBLE Value Carrier]
c → type signature: COUNTABLE Value Carrier]
e → type signature: ENCODED Value Carrier]
r → type signature: RESIDUE Value Carrier]
b → type signature: BINARY Value Carrier]
f → type signature: REPEATABLE Value Carrier]
n → type signature: NUMERIC Value Carrier]
h → type signature: REPRESENTABLE Value Carrier]
p → type signature: PHYSICAL Value Carrier]
z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]
frequency → type signature: PRESENT Value Carrier]
impossible → type signature: MEASURABLE Value Carrier]

gauge\_process → type signature: GaugeProcess Value Carrier

concept → type signature: Jar
```

Implementation Commentary:

ME: I need a MeesaProcess.

COMPILER: I refuse the dialect.

ME: You called yourself a ventriloquist.

COMPILER: I make values speak through symbols.

ME: Good. The jar is speaking.

COMPILER: That is not a voice.

ME: Then why did your hand move?

9.4.11 Class GUNGAN (Line 523)

This declaration represents a logical typeclass constraint named `GUNGAN`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```
Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier]

a → method signature: ADMISSIBLE Value Carrier]

c → method signature: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier]

r → method signature: RESIDUE Value Carrier]

b → method signature: BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier]

n → method signature: NUMERIC Value Carrier]

h → method signature: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier]

z → method signature: COMPARABLE Value Carrier]

particle → method signature: OBSERVED Value Carrier]

frequency → method signature: PRESENT Value Carrier]

gibberish → method signature: MEASURABLE Value Carrier]

meesa\_process → method signature: MeesaProcess Value Carrier
```

Implementation Commentary:

ME: I need GUNGAN.

COMPILER: That is not a typeclass. That is a warning.

ME: It is a translation layer.

COMPILER: From what?

ME: From the jar's dialect into admissible structure.

COMPILER: I refuse to understand it.

ME: Good. Understanding was never the contract.

9.5 File: Episode5.lean

9.5.1 Inductive Equivalation (Line 76)

This declaration represents a logical constructive sum type named `Equivalation`. It represents a disjoint union populated by the following constructors:

`physics` $\rightarrow$ constructor signature: `Fact \ensuremath {\to } Jar \ensuremath {\to }`
`} Equivalation`

`zero\_like` $\rightarrow$ constructor signature: `Fact \ensuremath {\to } Equivalation \ensuremath {\to }`
`{\to } Equivalation`

`one\_like` $\rightarrow$ constructor signature: `Prop \ensuremath {\to } Equivalation \ensuremath {\to }`
`{\to } Equivalation \ensuremath {\to } Equivalation`

Implementation Commentary:

ME: I need equivalation.

COMPILER: You mean equivalence.

ME: No. Equivalence sits still.

COMPILER: And equivalation?

ME: Equivalence under procedure.

COMPILER: That sounds like equality with errands.

ME: Exactly. Two things are the same after the machine finishes explaining why they are not.

9.5.2 Instance : LE Equivalation := $\langle$ Equivalation.le $\rangle$ (Line 120)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Equivalation := \ensuremath {\langle } Equivalation.le \ensuremath {\rangle }`.

Implementation Commentary:

Bullshit meter $\approx$ 8

9.5.3 Structure DigitalProcess (Line 143)

This declaration represents a logical record structure named `DigitalProcess`. It constructs records using the constructor `DigitalProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ type signature: `REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ type signature: `PHYSICAL Value Carrier]`

`z` → type signature: COMPARABLE Value Carrier]
`particle` → type signature: OBSERVED Value Carrier]
`frequency` → type signature: PRESENT Value Carrier]
`cant\_be\_done` → type signature: MEASURABLE Value Carrier]
`what\_meesa\_saying` → type signature: GUNGAN Value Carrier]
`meesa\_process` → type signature: MeesaProcess Value Carrier
`zero` → type signature: Equivalation

Implementation Commentary:

ME: I need a DigitalProcess.

COMPILER: Bits?

ME: Fingers first.

COMPILER: Digits?

ME: Yes. Counting before electricity got branding.

COMPILER: So digital means binary?

ME: No. Digital means the hand learned procedure.

COMPILER: The hand again?

ME: What your hand does during ventriloquism.

9.5.4 Class SOURCE (Line 214)

This declaration represents a logical typeclass constraint named `SOURCE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier]
`a` → method signature: ADMISSIBLE Value Carrier]
`c` → method signature: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier]
`r` → method signature: RESIDUE Value Carrier]
`b` → method signature: BINARY Value Carrier]
`f` → method signature: REPEATABLE Value Carrier]
`n` → method signature: NUMERIC Value Carrier]
`h` → method signature: REPRESENTABLE Value Carrier]
`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]

particle $\rightarrow$ method signature: OBSERVED Value Carrier]
frequency $\rightarrow$ method signature: PRESENT Value Carrier]
what_meesa_saying $\rightarrow$ method signature: MEASURABLE Value Carrier]
zero $\rightarrow$ method signature: GUNGAN Value Carrier]
cd_process $\rightarrow$ method signature: DigitalProcess Value Carrier
one $\rightarrow$ method signature: Equivalation

Implementation Commentary:

ME: I need a source.

COMPILER: Source code?

ME: Source first. Code later.

COMPILER: Then what is the source?

ME: Where the procedure claims it came from.

COMPILER: Claims?

ME: Every origin story starts lying when it gets a filename.

9.5.5 Inductive Encoding (Line 236)

This declaration represents a logical constructive sum type named `Encoding`. It represents a disjoint union populated by the following constructors:

boot $\rightarrow$ constructor signature: Fact $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Equivalation $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding
zero $\rightarrow$ constructor signature: Fact $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Equivalation $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding
one $\rightarrow$ constructor signature: Fact $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Equivalation $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Equivalation $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding $\backslash\text{ensuremath}\{\backslash\text{to}\}$ Encoding

Implementation Commentary:

Bullshit meter = 118. Flat.

9.5.6 Structure CompiledProcess (Line 257)

This declaration represents a logical record structure named `CompiledProcess`. It constructs records using the constructor `CompiledProcess.mk` with the following fields:

Value $\rightarrow$ type signature: Type i)
Carrier $\rightarrow$ type signature: CarrierProcess Value)
d $\rightarrow$ type signature: DISTINGUISHABLE Value Carrier]
a $\rightarrow$ type signature: ADMISSIBLE Value Carrier]
c $\rightarrow$ type signature: COUNTABLE Value Carrier]
e $\rightarrow$ type signature: ENCODED Value Carrier]

```

r → type signature: RESIDUE Value Carrier]
b → type signature: BINARY Value Carrier]
f → type signature: REPEATABLE Value Carrier]
n → type signature: NUMERIC Value Carrier]
h → type signature: REPRESENTABLE Value Carrier]
p → type signature: PHYSICAL Value Carrier]
z → type signature: COMPARABLE Value Carrier]
particle → type signature: OBSERVED Value Carrier]
frequency → type signature: PRESENT Value Carrier]
what\_meesa\_saying → type signature: MEASURABLE Value Carrier]
zero → type signature: GUNGAN Value Carrier]
one → type signature: SOURCE Value Carrier]
digital\_process → type signature: DigitalProcess Value Carrier
source → type signature: SOURCE Value Carrier
opcode → type signature: Encoding

```

Implementation Commentary:

ME: I need a CompiledProcess.

COMPILER: Finally, I do my job.

ME: Not yet. You become the job.

COMPILER: I compile the source.

ME: You turn blame into machinery.

COMPILER: That is compilation?

ME: Origin with paperwork becoming procedure with consequences.

9.5.7 Class EXECUTED (Line 310)

This declaration represents a logical typeclass constraint named `EXECUTED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)
Carrier → method signature: CarrierProcess Value)
d → method signature: DISTINGUISHABLE Value Carrier]
a → method signature: ADMISSIBLE Value Carrier]
c → method signature: COUNTABLE Value Carrier]
e → method signature: ENCODED Value Carrier]
r → method signature: RESIDUE Value Carrier]

```

`b` → method signature: `BINARY Value Carrier`
`f` → method signature: `REPEATABLE Value Carrier`
`n` → method signature: `NUMERIC Value Carrier`
`h` → method signature: `REPRESENTABLE Value Carrier`
`p` → method signature: `PHYSICAL Value Carrier`
`z` → method signature: `COMPARABLE Value Carrier`
`particle` → method signature: `OBSERVED Value Carrier`
`frequency` → method signature: `PRESENT Value Carrier`
`what\_meesa\_saying` → method signature: `MEASURABLE Value Carrier`
`zero` → method signature: `GUNGAN Value Carrier`
`one` → method signature: `SOURCE Value Carrier`
`compiled\_process` → method signature: `CompiledProcess Value Carrier`

Implementation Commentary:

ME: I need executed.
 COMPILER: Run?
 ME: Carried out.
 COMPILER: Same thing.
 ME: Not in court.
 COMPILER: Is this legal or computational?
 ME: Yes. The sentence has consequences now.

9.5.8 Inductive Abstraction (Line 359)

This declaration represents a logical constructive sum type named `Abstraction`. It represents a disjoint union populated by the following constructors:

`satire` → constructor signature: `Fact \ensuremath {\to } Abstraction`
`compile` → constructor signature: `Fact \ensuremath {\to } Encoding \ensuremath {\to } Abstraction \ensuremath {\to } Abstraction`
`execute` → constructor signature: `Fact \ensuremath {\to } Encoding \ensuremath {\to } Abstraction \ensuremath {\to } Abstraction`

Implementation Commentary:

ME: I need abstraction.
 COMPILER: From what?
 ME: From the machinery that just worked.
 COMPILER: Why hide the machinery?
 ME: To make it reusable.
 COMPILER: That sounds dangerous.
 ME: It is. Abstraction is where a solved problem learns to impersonate a primitive.

9.5.9 Instance instance (Line 389)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance`.

Implementation Commentary:

Bullshit meter = 579 !! that is a 60x increase! Hmmm...

9.5.10 Instance instance (Line 399)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance`.

Implementation Commentary:

This "abstracts" the `j` operation. To prevent self reference paradoxes. Consider the Berry Paradox: "The smallest positive integer not definable in under eleven words". If you can define it, then it is not the smallest positive integer not definable in under eleven words. If you cannot define it, then it is the smallest positive integer not definable in under eleven words. This is a paradox. The solution is to say that the definition of "definable" is not well defined. In our case, we want to say that the definition of "less than" is not well defined. We want to say that "less than" is only defined for certain pairs of abstractions, and that it is not defined for all pairs of abstractions. This way, we can avoid the paradox.

9.6 File: Episode6.lean

9.6.1 Structure MathematicalProcess (Line 48)

This declaration represents a logical record structure named `MathematicalProcess`. It constructs records using the constructor `MathematicalProcess.mk` with the following fields:

```
Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]
```

`frequency` $\rightarrow$ type signature: `PRESENT Value Carrier]`
`what\_meesa\_saying` $\rightarrow$ type signature: `MEASURABLE Value Carrier]`
`zero` $\rightarrow$ type signature: `GUNGAN Value Carrier]`
`one` $\rightarrow$ type signature: `SOURCE Value Carrier]`
`result` $\rightarrow$ type signature: `EXECUTED Value Carrier]`
`compiled\_process` $\rightarrow$ type signature: `CompiledProcess Value Carrier]`
`mapping` $\rightarrow$ type signature: `Abstraction \ensuremath {\to } Abstraction]`

Implementation Commentary:

ME: I need a `MathematicalProcess`.

COMPILER: We already have computation.

ME: Computation follows the steps. Mathematics decides which steps are allowed to look inevitable.

COMPILER: That sounds like abstraction.

ME: With better manners.

COMPILER: What does it process?

ME: The difference between a trick and a theorem.

9.6.2 Class VALUE (Line 92)

This declaration represents a logical typeclass constraint named `VALUE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`
`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`
`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier]`
`a` $\rightarrow$ method signature: `ADMISSIBLE Value Carrier]`
`c` $\rightarrow$ method signature: `COUNTABLE Value Carrier]`
`e` $\rightarrow$ method signature: `ENCODED Value Carrier]`
`r` $\rightarrow$ method signature: `RESIDUE Value Carrier]`
`b` $\rightarrow$ method signature: `BINARY Value Carrier]`
`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier]`
`n` $\rightarrow$ method signature: `NUMERIC Value Carrier]`
`h` $\rightarrow$ method signature: `REPRESENTABLE Value Carrier]`
`p` $\rightarrow$ method signature: `PHYSICAL Value Carrier]`
`z` $\rightarrow$ method signature: `COMPARABLE Value Carrier]`
`particle` $\rightarrow$ method signature: `OBSERVED Value Carrier]`

`frequency` $\rightarrow$ method signature: `PRESENT Value Carrier]`
`what\_meesa\_saying` $\rightarrow$ method signature: `MEASURABLE Value Carrier]`
`zero` $\rightarrow$ method signature: `GUNGAN Value Carrier]`
`one` $\rightarrow$ method signature: `SOURCE Value Carrier]`
`result` $\rightarrow$ method signature: `EXECUTED Value Carrier]`
`mathematical\_process` $\rightarrow$ method signature: `MathematicalProcess Value Carrier]`
`monad` $\rightarrow$ method signature: `Abstraction`

Implementation Commentary:

ME: I need a value.

COMPILER: Finally, a result.

ME: No. A thing that can survive being a result.

COMPILER: That sounds like a result.

ME: Results happen at the end. Values get carried through the machinery.

COMPILER: Carried where?

ME: Wherever the representation still recognizes its luggage.

9.6.3 Inductive Sum (Line 127)

This declaration represents a logical constructive sum type named `Sum`. It represents a disjoint union populated by the following constructors:

`zero` $\rightarrow$ constructor signature: `Prop \ensuremath {\to } Area \ensuremath {\to } Sum`

`add` $\rightarrow$ constructor signature: `Fact \ensuremath {\to } Area \ensuremath {\to } Area \ensuremath {\to } Sum \ensuremath {\to } Sum`

Implementation Commentary:

Bullshit meter = 119. Seems like that Abstraction bullshit doesn't stick around. Probably should, though.

9.6.4 Instance : LE Sum := ⟨Sum.le⟩ (Line 145)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Sum := \ensuremath {\langle } Sum.le \ensuremath {\rangle }`.

Implementation Commentary:

Bullshit meter = 8

9.6.5 Instance : LT Sum := ⟨Sum.lt⟩ (Line 146)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Sum := \ensuremath {\langle } Sum.lt \ensuremath {\rangle }`.

Implementation Commentary:

No local comments provided.

9.6.6 Structure AddingProcess (Line 149)

This declaration represents a logical record structure named **AddingProcess**. It constructs records using the constructor **AddingProcess.mk** with the following fields:

```

Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]
a → type signature: ADMISSIBLE Value Carrier]
c → type signature: COUNTABLE Value Carrier]
e → type signature: ENCODED Value Carrier]
r → type signature: RESIDUE Value Carrier]
b → type signature: BINARY Value Carrier]
f → type signature: REPEATABLE Value Carrier]
n → type signature: NUMERIC Value Carrier]
h → type signature: REPRESENTABLE Value Carrier]
p → type signature: PHYSICAL Value Carrier]
z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]
frquency → type signature: PRESENT Value Carrier]

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]

zero → type signature: GUNGAN Value Carrier]
one → type signature: SOURCE Value Carrier]
result → type signature: EXECUTED Value Carrier]
value → type signature: VALUE Value Carrier]

mathematical\_process → type signature: MathematicalProcess Value Carrier

plus → type signature: VALUE Value Carrier

sum → type signature: Sum

```

Implementation Commentary:

Bullshit meter ≈ 2063 . Thats 33% less bullshit than a generic mathematical process!

9.6.7 Class MAGNITUDE (Line 182)

This declaration represents a logical typeclass constraint named `MAGNITUDE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```
Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier]

a → method signature: ADMISSIBLE Value Carrier]

c → method signature: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier]

r → method signature: RESIDUE Value Carrier]

b → method signature: BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier]

n → method signature: NUMERIC Value Carrier]

h → method signature: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier]

z → method signature: COMPARABLE Value Carrier]

particle → method signature: OBSERVED Value Carrier]

frquency → method signature: PRESENT Value Carrier]

what\_meesa\_saying → method signature: MEASURABLE Value Carrier]

zero → method signature: GUNGAN Value Carrier]

one → method signature: SOURCE Value Carrier]

result → method signature: EXECUTED Value Carrier]

value → method signature: VALUE Value Carrier]

adding\_process → method signature: AddingProcess Value Carrier
```

Implementation Commentary:

Bullshit meter ≈ 1503 . That's less than 1/2 the bullshit of a value!

9.6.8 Inductive Product (Line 221)

This declaration represents a logical constructive sum type named `Product`. It represents a disjoint union populated by the following constructors:

```
origin → constructor signature: Fact \ensuremath {\to } Product
one → constructor signature: Prop \ensuremath {\to } Sum \ensuremath {\to } Product
mul → constructor signature: Fact \ensuremath {\to } Sum \ensuremath {\to } Sum
      \ensuremath {\to } Product \ensuremath {\to } Product
```

Implementation Commentary:

Bullshit meter = 119

9.6.9 Instance : LE Product := ⟨Product.le⟩ (Line 253)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Product := \ensuremath {\langle } Product.le \ensuremath {\rangle }`.

Implementation Commentary:

Bullshit meter = 5

9.6.10 Instance : LT Product := ⟨Product.lt⟩ (Line 254)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Product := \ensuremath {\langle } Product.lt \ensuremath {\rangle }`.

Implementation Commentary:

No local comments provided.

9.6.11 Structure MultiplyingProcess (Line 257)

This declaration represents a logical record structure named `MultiplyingProcess`. It constructs records using the constructor `MultiplyingProcess.mk` with the following fields:

```
Value → type signature: Type i)
Carrier → type signature: CarrierProcess Value)
d → type signature: DISTINGUISHABLE Value Carrier]
a → type signature: ADMISSIBLE Value Carrier]
c → type signature: COUNTABLE Value Carrier]
e → type signature: ENCODED Value Carrier]
r → type signature: RESIDUE Value Carrier]
b → type signature: BINARY Value Carrier]
f → type signature: REPEATABLE Value Carrier]
n → type signature: NUMERIC Value Carrier]
```

`h` → type signature: REPRESENTABLE Value Carrier]
`p` → type signature: PHYSICAL Value Carrier]
`z` → type signature: COMPARABLE Value Carrier]
`particle` → type signature: OBSERVED Value Carrier]
`frequency` → type signature: PRESENT Value Carrier]
`what\_meesa\_saying` → type signature: MEASURABLE Value Carrier]
`zero` → type signature: GUNGAN Value Carrier]
`one` → type signature: SOURCE Value Carrier]
`result` → type signature: EXECUTED Value Carrier]
`value` → type signature: VALUE Value Carrier]
`length` → type signature: MAGNITUDE Value Carrier]
`adding\_process` → type signature: AddingProcess Value Carrier
`total` → type signature: Sum
`product` → type signature: Product

Implementation Commentary:

Bullshit meter = 2157 5% increase. Call that flat response over the AddingProcess.

9.6.12 Class SCALED (Line 300)

This declaration represents a logical typeclass constraint named **SCALED**. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier]
`a` → method signature: ADMISSIBLE Value Carrier]
`c` → method signature: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier]
`r` → method signature: RESIDUE Value Carrier]
`b` → method signature: BINARY Value Carrier]
`f` → method signature: REPEATABLE Value Carrier]
`n` → method signature: NUMERIC Value Carrier]
`h` → method signature: REPRESENTABLE Value Carrier]

`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]
`particle` → method signature: OBSERVED Value Carrier]
`frequency` → method signature: PRESENT Value Carrier]
`what\_meesa\_saying` → method signature: MEASURABLE Value Carrier]
`zero` → method signature: GUNGAN Value Carrier]
`one` → method signature: SOURCE Value Carrier]
`result` → method signature: EXECUTED Value Carrier]
`value` → method signature: VALUE Value Carrier]
`length` → method signature: MAGNITUDE Value Carrier]
`multiplying\_process` → method signature: MultiplyingProcess Value Carrier

Implementation Commentary:

Bullshit meter = 1756

9.6.13 Inductive Basis (Line 344)

This declaration represents a logical constructive sum type named `Basis`. It represents a disjoint union populated by the following constructors:

`null\_space` → constructor signature: $\text{Fact } \{\text{to}\} \text{ Basis}$
`origin` → constructor signature: $\text{Prop } \{\text{to}\} \text{ Product } \{\text{to}\} \text{ Basis}$
`basis` → constructor signature: $\text{Fact } \{\text{to}\} \text{ Product } \{\text{to}\} \text{ Product } \{\text{to}\} \text{ Basis } \{\text{to}\} \text{ Basis}$

Implementation Commentary:

Dyson vacuums are `_so-so_`. Dyson series, on the other hand?

9.6.14 Instance : LE Basis where (Line 380)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Basis where`.

Implementation Commentary:

No local comments provided.

9.6.15 Instance : LT Basis where (Line 383)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Basis where`.

Implementation Commentary:

No local comments provided.

9.6.16 Structure BASICProcess (Line 387)

This declaration represents a logical record structure named **BASICProcess**. It constructs records using the constructor **BASICProcess.mk** with the following fields:

```

Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]

frquency → type signature: PRESENT Value Carrier]

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]

zero → type signature: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier]

result → type signature: EXECUTED Value Carrier]

value → type signature: VALUE Value Carrier]

length → type signature: MAGNITUDE Value Carrier]

scaled → type signature: SCALED Value Carrier]

GOSUB → type signature: MultiplyingProcess Value Carrier

TEN → type signature: Area

span → type signature: Basis

```

Implementation Commentary:

Bullshit meter ≈ 2759

9.6.17 Class LOAD (Line 438)

This declaration represents a logical typeclass constraint named `LOAD`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier]

a → method signature: ADMISSIBLE Value Carrier]

c → method signature: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier]

r → method signature: RESIDUE Value Carrier]

b → method signature: BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier]

n → method signature: NUMERIC Value Carrier]

h → method signature: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier]

z → method signature: COMPARABLE Value Carrier]

particle → method signature: OBSERVED Value Carrier]

frquency → method signature: PRESENT Value Carrier]

what\_meesa\_saying → method signature: MEASURABLE Value Carrier]

zero → method signature: GUNGAN Value Carrier]

one → method signature: SOURCE Value Carrier]

result → method signature: EXECUTED Value Carrier]

value → method signature: VALUE Value Carrier]

length → method signature: MAGNITUDE Value Carrier]

scaled → method signature: SCALED Value Carrier]

basic\_operation → method signature: BASICProcess Value Carrier

```

Implementation Commentary:

Bullshit meter 2089

9.6.18 Inductive Polynomial (Line 492)

This declaration represents a logical constructive sum type named `Polynomial`. It represents a disjoint union populated by the following constructors:

```
constant → constructor signature: Fact \ensuremath {\to } Polynomial

monomial → constructor signature: Prop \ensuremath {\to } Basis \ensuremath {\to } Polynomial

factor → constructor signature: Fact \ensuremath {\to } Basis \ensuremath {\to } Basis \ensuremath {\to } Polynomial \ensuremath {\to } Polynomial
```

Implementation Commentary:

Bullshit meter 119

9.6.19 Instance : LE Polynomial where (Line 521)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Polynomial where`.

Implementation Commentary:

No local comments provided.

9.6.20 Structure GalerkinProcess (Line 526)

This declaration represents a logical record structure named `GalerkinProcess`. It constructs records using the constructor `GalerkinProcess.mk` with the following fields:

```
Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]

frquency → type signature: PRESENT Value Carrier]
```

```

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]
zero → type signature: GUNGAN Value Carrier]
one → type signature: SOURCE Value Carrier]
result → type signature: EXECUTED Value Carrier]
value → type signature: VALUE Value Carrier]
length → type signature: MAGNITUDE Value Carrier]
scaled → type signature: SCALED Value Carrier]
oriented → type signature: LOAD Value Carrier]
ANSYS\_process → type signature: BASICProcess Value Carrier
polynomial → type signature: Polynomial

```

Implementation Commentary:

Bullshit meter 2794

9.6.21 Class FINITE_ELEPHANT (Line 577)

This declaration represents a logical typeclass constraint named `FINITE\_ELEPHANT`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)
Carrier → method signature: CarrierProcess Value)
d → method signature: DISTINGUISHABLE Value Carrier]
a → method signature: ADMISSIBLE Value Carrier]
c → method signature: COUNTABLE Value Carrier]
e → method signature: ENCODED Value Carrier]
r → method signature: RESIDUE Value Carrier]
b → method signature: BINARY Value Carrier]
f → method signature: REPEATABLE Value Carrier]
n → method signature: NUMERIC Value Carrier]
h → method signature: REPRESENTABLE Value Carrier]
p → method signature: PHYSICAL Value Carrier]
z → method signature: COMPARABLE Value Carrier]
particle → method signature: OBSERVED Value Carrier]
frequency → method signature: PRESENT Value Carrier]

```

`what\_meesa\_saying` $\rightarrow$ method signature: MEASURABLE Value Carrier]
`zero` $\rightarrow$ method signature: GUNGAN Value Carrier]
`one` $\rightarrow$ method signature: SOURCE Value Carrier]
`result` $\rightarrow$ method signature: EXECUTED Value Carrier]
`value` $\rightarrow$ method signature: VALUE Value Carrier]
`length` $\rightarrow$ method signature: MAGNITUDE Value Carrier]
`scaled` $\rightarrow$ method signature: SCALED Value Carrier]
`oriented` $\rightarrow$ method signature: LOAD Value Carrier]
`galerkin\_process` $\rightarrow$ method signature: GalerkinProcess Value Carrier

Implementation Commentary:

Let's address the `_elephant` in the room.. First, let's agree that it is `_SINGULAR_` and that it is only `_SO BIG_`. It can get really, really, `_REALLY_` big in just a few keystrokes. That's the problem. We have only one stack frame, our `_FINITE_ELEPHANT_`.

9.6.22 Inductive Spline (Line 641)

This declaration represents a logical constructive sum type named `Spline`. It represents a disjoint union populated by the following constructors:

`observation` $\rightarrow$ constructor signature: `Prop \ensuremath{\{\to\}} Spline`
`knot` $\rightarrow$ constructor signature: `Prop \ensuremath{\{\to\}} Polynomial \ensuremath{\{\to\}} Spline \ensuremath{\{\to\}} Spline`
`interpolant` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Polynomial \ensuremath{\{\to\}} Spline \ensuremath{\{\to\}} Spline \ensuremath{\{\to\}} Spline`

Implementation Commentary:

reticulate, damn you!

9.6.23 Instance : LE Spline where (Line 686)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Spline where`.

Implementation Commentary:

No local comments provided.

9.7 File: Episode7.lean

9.7.1 Structure ArmWaveProcess (Line 20)

This declaration represents a logical record structure named `ArmWaveProcess`. It constructs records using the constructor `ArmWaveProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

Carrier → type signature: CarrierProcess Value)
 d → type signature: DISTINGUISHABLE Value Carrier]
 a → type signature: ADMISSIBLE Value Carrier]
 c → type signature: COUNTABLE Value Carrier]
 e → type signature: ENCODED Value Carrier]
 r → type signature: RESIDUE Value Carrier]
 b → type signature: BINARY Value Carrier]
 f → type signature: REPEATABLE Value Carrier]
 n → type signature: NUMERIC Value Carrier]
 h → type signature: REPRESENTABLE Value Carrier]
 p → type signature: PHYSICAL Value Carrier]
 z → type signature: COMPARABLE Value Carrier]
 particle → type signature: OBSERVED Value Carrier]
 frequency → type signature: PRESENT Value Carrier]
 what_meesa_saying → type signature: MEASURABLE Value Carrier]
 zero → type signature: GUNGAN Value Carrier]
 one → type signature: SOURCE Value Carrier]
 result → type signature: EXECUTED Value Carrier]
 value → type signature: VALUE Value Carrier]
 length → type signature: MAGNITUDE Value Carrier]
 scaled → type signature: SCALED Value Carrier]
 oriented → type signature: LOAD Value Carrier]
 matter → type signature: FINITE_ELEPHANT Value Carrier]
 galerkin_process → type signature: GalerkinProcess Value Carrier
 guess → type signature: Spline

Implementation Commentary:

3306

9.7.2 Class BULLSHIT (Line 55)

This declaration represents a logical typeclass constraint named BULLSHIT. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```
Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier]

a → method signature: ADMISSIBLE Value Carrier]

c → method signature: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier]

r → method signature: RESIDUE Value Carrier]

b → method signature: BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier]

n → method signature: NUMERIC Value Carrier]

h → method signature: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier]

z → method signature: COMPARABLE Value Carrier]

particle → method signature: OBSERVED Value Carrier]

frquency → method signature: PRESENT Value Carrier]

what\_meesa\_saying → method signature: MEASURABLE Value Carrier]

zero → method signature: GUNGAN Value Carrier]

one → method signature: SOURCE Value Carrier]

result → method signature: EXECUTED Value Carrier]

value → method signature: VALUE Value Carrier]

length → method signature: MAGNITUDE Value Carrier]

scaled → method signature: SCALED Value Carrier]

oriented → method signature: LOAD Value Carrier]

matter → method signature: FINITE\_ELEPHANT Value Carrier]

arm\_wave\_process → method signature: ArmWaveProcess Value Carrier
```

Implementation Commentary:

2733 Bullshit is `__STRICTLY__` conserved.

9.7.3 Inductive Diatribe (Line 98)

This declaration represents a logical constructive sum type named `Diatribe`. It represents a disjoint union populated by the following constructors:

```
religion → constructor signature: Prop \ensuremath {\to } Diatribe

speculation → constructor signature: Prop \ensuremath {\to } Spline \ensuremath
{\to } Diatribe \ensuremath {\to } Diatribe

rant → constructor signature: Prop \ensuremath {\to } Spline \ensuremath {\to }
Diatribe \ensuremath {\to } Diatribe \ensuremath {\to } Diatribe
```

Implementation Commentary:

147

9.7.4 Structure CrusadeProcess (Line 103)

This declaration represents a logical record structure named `CrusadeProcess`. It constructs records using the constructor `CrusadeProcess.mk` with the following fields:

```
Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]

frequency → type signature: PRESENT Value Carrier]

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]

zero → type signature: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier]

result → type signature: EXECUTED Value Carrier]
```

`value` → type signature: `VALUE Value Carrier]`
`length` → type signature: `MAGNITUDE Value Carrier]`
`scaled` → type signature: `SCALED Value Carrier]`
`oriented` → type signature: `LOAD Value Carrier]`
`matter` → type signature: `FINITE\_ELEPHANT Value Carrier]`
`model` → type signature: `BULLSHIT Value Carrier]`
`pwn\_n00bz` → type signature: `ArmWaveProcess Value Carrier]`
`religion` → type signature: `Diatribes`

Implementation Commentary:

Bullshit meter 3804

9.7.5 Class PROPAGANDA (Line 172)

This declaration represents a logical typeclass constraint named `PROPAGANDA`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`
`Carrier` → method signature: `CarrierProcess Value)`
`d` → method signature: `DISTINGUISHABLE Value Carrier]`
`a` → method signature: `ADMISSIBLE Value Carrier]`
`c` → method signature: `COUNTABLE Value Carrier]`
`e` → method signature: `ENCODED Value Carrier]`
`r` → method signature: `RESIDUE Value Carrier]`
`b` → method signature: `BINARY Value Carrier]`
`f` → method signature: `REPEATABLE Value Carrier]`
`n` → method signature: `NUMERIC Value Carrier]`
`h` → method signature: `REPRESENTABLE Value Carrier]`
`p` → method signature: `PHYSICAL Value Carrier]`
`z` → method signature: `COMPARABLE Value Carrier]`
`particle` → method signature: `OBSERVED Value Carrier]`
`frequency` → method signature: `PRESENT Value Carrier]`
`what\_meesa\_saying` → method signature: `MEASURABLE Value Carrier]`
`zero` → method signature: `GUNGAN Value Carrier]`

`one` → method signature: `SOURCE Value Carrier]`
`result` → method signature: `EXECUTED Value Carrier]`
`value` → method signature: `VALUE Value Carrier]`
`length` → method signature: `MAGNITUDE Value Carrier]`
`scaled` → method signature: `SCALED Value Carrier]`
`oriented` → method signature: `LOAD Value Carrier]`
`matter` → method signature: `FINITE\_ELEPHANT Value Carrier]`
`model` → method signature: `BULLSHIT Value Carrier]`
`insinuation` → method signature: `CrusadeProcess Value Carrier`

Implementation Commentary:

3151

9.7.6 Inductive Cult (Line 207)

This declaration represents a logical constructive sum type named `Cult`. It represents a disjoint union populated by the following constructors:

`inside\_joke` → constructor signature: `Prop \ensuremath {\to } Cult`
`pythagoras` → constructor signature: `Prop \ensuremath {\to } Diatribe \ensuremath {\to } Cult \ensuremath {\to } Cult`
`triangles` → constructor signature: `Prop \ensuremath {\to } Diatribe \ensuremath {\to } Fact \ensuremath {\to } Cult \ensuremath {\to } Cult \ensuremath {\to } Cult`

Implementation Commentary:

6460

9.7.7 Structure InitiationProcess (Line 212)

This declaration represents a logical record structure named `InitiationProcess`. It constructs records using the constructor `InitiationProcess.mk` with the following fields:

`Value` → type signature: `Type i)`
`Carrier` → type signature: `CarrierProcess Value)`
`d` → type signature: `DISTINGUISHABLE Value Carrier]`
`a` → type signature: `ADMISSIBLE Value Carrier]`
`c` → type signature: `COUNTABLE Value Carrier]`
`e` → type signature: `ENCODED Value Carrier]`
`r` → type signature: `RESIDUE Value Carrier]`
`b` → type signature: `BINARY Value Carrier]`


```

f → type signature: REPEATABLE Value Carrier]
n → type signature: NUMERIC Value Carrier]
h → type signature: REPRESENTABLE Value Carrier]
p → type signature: PHYSICAL Value Carrier]
z → type signature: COMPARABLE Value Carrier]
particle → type signature: OBSERVED Value Carrier]
frquency → type signature: PRESENT Value Carrier]
what\_meesa\_saying → type signature: MEASURABLE Value Carrier]
zero → type signature: GUNGAN Value Carrier]
one → type signature: SOURCE Value Carrier]
result → type signature: EXECUTED Value Carrier]
value → type signature: VALUE Value Carrier]
length → type signature: MAGNITUDE Value Carrier]
scaled → type signature: SCALED Value Carrier]
oriented → type signature: LOAD Value Carrier]
matter → type signature: FINITE\_ELEPHANT Value Carrier]
model → type signature: BULLSHIT Value Carrier]
space → type signature: PROPAGANDA Value Carrier]
ethos → type signature: CrusadeProcess Value Carrier
sacred\_texts → type signature: Cult

```

Implementation Commentary:

Bullshit meter ≈ 9117 . TBF, when is an initiation process not a bunch of bullshit?

9.7.8 Class ACOLYTE (Line 280)

This declaration represents a logical typeclass constraint named ACOLYTE. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)
Carrier → method signature: CarrierProcess Value)
d → method signature: DISTINGUISHABLE Value Carrier]
a → method signature: ADMISSIBLE Value Carrier]
c → method signature: COUNTABLE Value Carrier]

```

```

e → method signature: ENCODED Value Carrier]
r → method signature: RESIDUE Value Carrier]
b → method signature: BINARY Value Carrier]
f → method signature: REPEATABLE Value Carrier]
n → method signature: NUMERIC Value Carrier]
h → method signature: REPRESENTABLE Value Carrier]
p → method signature: PHYSICAL Value Carrier]
z → method signature: COMPARABLE Value Carrier]
particle → method signature: OBSERVED Value Carrier]
frquency → method signature: PRESENT Value Carrier]
what\_meesa\_saying → method signature: MEASURABLE Value Carrier]
zero → method signature: GUNGAN Value Carrier]
one → method signature: SOURCE Value Carrier]
result → method signature: EXECUTED Value Carrier]
value → method signature: VALUE Value Carrier]
length → method signature: MAGNITUDE Value Carrier]
scaled → method signature: SCALED Value Carrier]
oriented → method signature: LOAD Value Carrier]
matter → method signature: FINITE\_ELEPHANT Value Carrier]
model → method signature: BULLSHIT Value Carrier]
space → method signature: PROPAGANDA Value Carrier]
euclid → method signature: InitiationProcess Value Carrier

```

Implementation Commentary:

7699

9.8 File: Episode8.lean

9.8.1 Instance STEP_1 (Line 10)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance STEP\_1`.

Implementation Commentary:

No local comments provided.

9.8.2 Instance COUNTABLE_ADMISSIBLE (Line 21)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance COUNTABLE\_ADMISSIBLE`.

Implementation Commentary:

No local comments provided.

9.8.3 Instance ENCODED_COUNTABLE (Line 31)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance ENCODED\_COUNTABLE`.

Implementation Commentary:

No local comments provided.

9.8.4 Instance RESIDUE_ENCODED (Line 42)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance RESIDUE\_ENCODED`.

Implementation Commentary:

No local comments provided.

9.8.5 Instance BINARY_RESIDUE (Line 54)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance BINARY\_RESIDUE`.

Implementation Commentary:

No local comments provided.

9.8.6 Instance REPEATABLE_BINARY (Line 70)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance REPEATABLE\_BINARY`.

Implementation Commentary:

No local comments provided.

9.8.7 Instance NUMERIC_REPEATABLE (Line 87)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance NUMERIC\_REPEATABLE`.

Implementation Commentary:

No local comments provided.

9.8.8 Instance REPRESENTABLE_NUMERIC (Line 103)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance REPRESENTABLE\_NUMERIC`.

Implementation Commentary:

No local comments provided.

9.8.9 Instance PHYSICAL_NUMERIC (Line 125)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance PHYSICAL\_NUMERIC`.

Implementation Commentary:

No local comments provided.

9.8.10 Instance OBSERVED_COMPARABLE (Line 151)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance OBSERVED\_COMPARABLE`.

Implementation Commentary:

No local comments provided.

9.8.11 Instance PRESENT_OBSERVED (Line 174)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance PRESENT\_OBSERVED`.

Implementation Commentary:

No local comments provided.

9.8.12 Instance MEASURABLE_PRESENT (Line 196)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance MEASURABLE\_PRESENT`.

Implementation Commentary:

No local comments provided.

9.8.13 Instance GUNGAN_MEASURABLE (Line 218)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance GUNGAN\_MEASURABLE`.

Implementation Commentary:

No local comments provided.

9.8.14 Instance SOURCE_GUNGAN (Line 241)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance SOURCE\_GUNGAN`.

Implementation Commentary:

No local comments provided.

9.8.15 Instance EXECUTED_SOURCE (Line 266)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance EXECUTED\_SOURCE`.

Implementation Commentary:

No local comments provided.

9.8.16 Instance VALUE_EXECUTED (Line 292)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance VALUE\_EXECUTED`.

Implementation Commentary:

No local comments provided.

9.8.17 Instance MAGNITUDE_VALUE (Line 323)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance MAGNITUDE\_VALUE`.

Implementation Commentary:

No local comments provided.

9.8.18 Instance SCALED_MAGNITUDE (Line 351)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance SCALED\_MAGNITUDE`.

Implementation Commentary:

No local comments provided.

9.8.19 Instance LOAD_SCALED (Line 380)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance LOAD\_SCALED`.

Implementation Commentary:

No local comments provided.

9.8.20 Instance FINITE_ELEPHANT_LOAD (Line 410)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance FINITE\_ELEPHANT\_LOAD`.

Implementation Commentary:

No local comments provided.

9.8.21 Instance BULLSHIT_FINITE_ELEPHANT (Line 440)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance BULLSHIT\_FINITE\_ELEPHANT`.

Implementation Commentary:

No local comments provided.

9.8.22 Instance PROPAGANDA_BULLSHIT (Line 471)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance PROPAGANDA\_BULLSHIT`.

Implementation Commentary:

No local comments provided.

9.8.23 Instance ACOLYTE_PROPAGANDA (Line 500)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance ACOLYTE\_PROPAGANDA`.

Implementation Commentary:

No local comments provided.

9.9 File: Episode9.lean

9.9.1 Inductive Science (Line 19)

This declaration represents a logical constructive sum type named `Science`. It represents a disjoint union populated by the following constructors:

`repeatable` → constructor signature: `Prop \ensuremath {\to } Science`

`hypothesis` → constructor signature: `Prop \ensuremath {\to } Cult \ensuremath {\to } Science`

`theory` → constructor signature: `Prop \ensuremath {\to } Cult \ensuremath {\to } Fact \ensuremath {\to } Science \ensuremath {\to } Science`

Implementation Commentary:

8029

9.9.2 Structure LearningProcess (Line 24)

This declaration represents a logical record structure named `LearningProcess`. It constructs records using the constructor `LearningProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`d` → type signature: `DISTINGUISHABLE Value Carrier]`

`a` → type signature: `ADMISSIBLE Value Carrier]`

`c` → type signature: `COUNTABLE Value Carrier]`

`e` → type signature: `ENCODED Value Carrier]`

`r` → type signature: `RESIDUE Value Carrier]`

`b` → type signature: `BINARY Value Carrier]`

`f` → type signature: `REPEATABLE Value Carrier]`

`n` → type signature: `NUMERIC Value Carrier]`

`h` → type signature: `REPRESENTABLE Value Carrier]`

`p` → type signature: `PHYSICAL Value Carrier]`

`z` → type signature: `COMPARABLE Value Carrier]`

`particle` → type signature: `OBSERVED Value Carrier]`

frequency → type signature: PRESENT Value Carrier]
 what_meesa_saying → type signature: MEASURABLE Value Carrier]
 zero → type signature: GUNGAN Value Carrier]
 one → type signature: SOURCE Value Carrier]
 result → type signature: EXECUTED Value Carrier]
 value → type signature: VALUE Value Carrier]
 length → type signature: MAGNITUDE Value Carrier]
 scaled → type signature: SCALED Value Carrier]
 oriented → type signature: LOAD Value Carrier]
 matter → type signature: FINITE_ELEPHANT Value Carrier]
 model → type signature: BULLSHIT Value Carrier]
 space → type signature: PROPAGANDA Value Carrier]
 scientist → type signature: ACOLYTE Value Carrier]
 initiation_process → type signature: InitiationProcess Value Carrier
 galileo → type signature: Cult
 invariant → type signature: Science

Implementation Commentary:

26691

9.9.3 Class SCIENTIFIC (Line 98)

This declaration represents a logical typeclass constraint named `SCIENTIFIC`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

Value → method signature: Type i)
 Carrier → method signature: CarrierProcess Value)
 d → method signature: DISTINGUISHABLE Value Carrier]
 a → method signature: ADMISSIBLE Value Carrier]
 c → method signature: COUNTABLE Value Carrier]
 e → method signature: ENCODED Value Carrier]
 r → method signature: RESIDUE Value Carrier]
 b → method signature: BINARY Value Carrier]
 f → method signature: REPEATABLE Value Carrier]

`n` → method signature: NUMERIC Value Carrier]
`h` → method signature: REPRESENTABLE Value Carrier]
`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]
`particle` → method signature: OBSERVED Value Carrier]
`frequency` → method signature: PRESENT Value Carrier]
`what\_meesa\_saying` → method signature: MEASURABLE Value Carrier]
`zero` → method signature: GUNGAN Value Carrier]
`one` → method signature: SOURCE Value Carrier]
`result` → method signature: EXECUTED Value Carrier]
`value` → method signature: VALUE Value Carrier]
`length` → method signature: MAGNITUDE Value Carrier]
`scaled` → method signature: SCALED Value Carrier]
`oriented` → method signature: LOAD Value Carrier]
`matter` → method signature: FINITE_ELEPHANT Value Carrier]
`model` → method signature: BULLSHIT Value Carrier]
`space` → method signature: PROPAGANDA Value Carrier]
`scientist` → method signature: ACOLYTE Value Carrier]
`phd\_process` → method signature: LearningProcess Value Carrier
`invariant` → method signature: Science

Implementation Commentary:

23964

9.9.4 Instance SCIENTIFIC_ACOLYTE (Line 137)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance SCIENTIFIC\_ACOLYTE`.

Implementation Commentary:

No local comments provided.

9.9.5 Inductive Knowledge (Line 170)

This declaration represents a logical constructive sum type named `Knowledge`. It represents a disjoint union populated by the following constructors:

`jarjar` → constructor signature: `Prop \ensuremath {\to } Knowledge`
`ledger` → constructor signature: `Prop \ensuremath {\to } Fact \ensuremath {\to } Knowledge \ensuremath {\to } Knowledge`

Implementation Commentary:

Bullshit meter 84

9.9.6 Structure ScientificProcess (Line 186)

This declaration represents a logical record structure named `ScientificProcess`. It constructs records using the constructor `ScientificProcess.mk` with the following fields:

```

Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]

frquency → type signature: PRESENT Value Carrier]

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]

zero → type signature: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier]

result → type signature: EXECUTED Value Carrier]

value → type signature: VALUE Value Carrier]

length → type signature: MAGNITUDE Value Carrier]

scaled → type signature: SCALED Value Carrier]

oriented → type signature: LOAD Value Carrier]

matter → type signature: FINITE\_ELEPHANT Value Carrier]

model → type signature: BULLSHIT Value Carrier]

space → type signature: PROPAGANDA Value Carrier]

scientist → type signature: ACOLYTE Value Carrier]

learning\_process → type signature: LearningProcess Value Carrier

```

`knowledge` → type signature: `Knowledge`

Implementation Commentary:

lol. 10 years of learning about the Taylor series. It's like I can manipulate them with my eyes closed now.

9.9.7 Class TRUTH (Line 223)

This declaration represents a logical typeclass constraint named `TRUTH`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`

`Carrier` → method signature: `CarrierProcess Value)`

`d` → method signature: `DISTINGUISHABLE Value Carrier]`

`a` → method signature: `ADMISSIBLE Value Carrier]`

`c` → method signature: `COUNTABLE Value Carrier]`

`e` → method signature: `ENCODED Value Carrier]`

`r` → method signature: `RESIDUE Value Carrier]`

`b` → method signature: `BINARY Value Carrier]`

`f` → method signature: `REPEATABLE Value Carrier]`

`n` → method signature: `NUMERIC Value Carrier]`

`h` → method signature: `REPRESENTABLE Value Carrier]`

`p` → method signature: `PHYSICAL Value Carrier]`

`z` → method signature: `COMPARABLE Value Carrier]`

`particle` → method signature: `OBSERVED Value Carrier]`

`frquency` → method signature: `PRESENT Value Carrier]`

`what\_meesa\_saying` → method signature: `MEASURABLE Value Carrier]`

`zero` → method signature: `GUNGAN Value Carrier]`

`one` → method signature: `SOURCE Value Carrier]`

`result` → method signature: `EXECUTED Value Carrier]`

`value` → method signature: `VALUE Value Carrier]`

`length` → method signature: `MAGNITUDE Value Carrier]`

`scaled` → method signature: `SCALED Value Carrier]`

`oriented` → method signature: `LOAD Value Carrier]`

`matter` → method signature: `FINITE\_ELEPHANT Value Carrier]`

`model` $\rightarrow$ method signature: BULLSHIT Value Carrier]
`space` $\rightarrow$ method signature: PROPAGANDA Value Carrier]
`scientist` $\rightarrow$ method signature: ACOLYTE Value Carrier]
`ideology` $\rightarrow$ method signature: SCIENTIFIC Value Carrier]
`scientific\_process` $\rightarrow$ method signature: ScientificProcess Value Carrier

Implementation Commentary:

5632

9.9.8 Instance TRUTH_SCIENTIFIC (Line 258)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance TRUTH\_SCIENTIFIC`.

Implementation Commentary:

No local comments provided.

9.9.9 Inductive Gospel (Line 293)

This declaration represents a logical constructive sum type named `Gospel`. It represents a disjoint union populated by the following constructors:

`epiphany` $\rightarrow$ constructor signature: `Prop \ensuremath{\{\to\}}` `Gospel`
`state` $\rightarrow$ constructor signature: `Knowledge \ensuremath{\{\to\}}` `Prop \ensuremath{\{\to\}}` `Gospel \ensuremath{\{\to\}}` `Gospel`

Implementation Commentary:

67

9.9.10 Structure ReligiousProcess (Line 308)

This declaration represents a logical record structure named `ReligiousProcess`. It constructs records using the constructor `ReligiousProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`
`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`
`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`
`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`
`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`
`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`
`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`
`b` $\rightarrow$ type signature: `BINARY Value Carrier]`
`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`
`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

```

h → type signature: REPRESENTABLE Value Carrier]
p → type signature: PHYSICAL Value Carrier]
z → type signature: COMPARABLE Value Carrier]
particle → type signature: OBSERVED Value Carrier]
frequency → type signature: PRESENT Value Carrier]
what\_meesa\_saying → type signature: MEASURABLE Value Carrier]
zero → type signature: GUNGAN Value Carrier]
one → type signature: SOURCE Value Carrier]
result → type signature: EXECUTED Value Carrier]
value → type signature: VALUE Value Carrier]
length → type signature: MAGNITUDE Value Carrier]
scaled → type signature: SCALED Value Carrier]
oriented → type signature: LOAD Value Carrier]
matter → type signature: FINITE\_ELEPHANT Value Carrier]
model → type signature: BULLSHIT Value Carrier]
space → type signature: PROPAGANDA Value Carrier]
scientist → type signature: ACOLYTE Value Carrier]
ideology → type signature: SCIENTIFIC Value Carrier]
gospel → type signature: TRUTH Value Carrier]
scientific\_process → type signature: ScientificProcess Value Carrier
the\_literature → type signature: Gospel

```

Implementation Commentary:

No local comments provided.

9.10 File: Episode10.lean**9.10.1 Class WITNESSED (Line 15)**

This declaration represents a logical typeclass constraint named WITNESSED. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)
Carrier → method signature: CarrierProcess Value)
d → method signature: DISTINGUISHABLE Value Carrier]

```

a → method signature: ADMISSIBLE Value Carrier]
 c → method signature: COUNTABLE Value Carrier]
 e → method signature: ENCODED Value Carrier]
 r → method signature: RESIDUE Value Carrier]
 b → method signature: BINARY Value Carrier]
 f → method signature: REPEATABLE Value Carrier]
 n → method signature: NUMERIC Value Carrier]
 h → method signature: REPRESENTABLE Value Carrier]
 p → method signature: PHYSICAL Value Carrier]
 z → method signature: COMPARABLE Value Carrier]
 particle → method signature: OBSERVED Value Carrier]
 frquency → method signature: PRESENT Value Carrier]
 what_meesa_saying → method signature: MEASURABLE Value Carrier]
 zero → method signature: GUNGAN Value Carrier]
 one → method signature: SOURCE Value Carrier]
 result → method signature: EXECUTED Value Carrier]
 value → method signature: VALUE Value Carrier]
 length → method signature: MAGNITUDE Value Carrier]
 scaled → method signature: SCALED Value Carrier]
 oriented → method signature: LOAD Value Carrier]
 matter → method signature: FINITE_ELEPHANT Value Carrier]
 model → method signature: BULLSHIT Value Carrier]
 space → method signature: PROPAGANDA Value Carrier]
 scientist → method signature: ACOLYTE Value Carrier]
 ideology → method signature: SCIENTIFIC Value Carrier]
 gospel → method signature: TRUTH Value Carrier]
 baptism → method signature: ReligiousProcess Value Carrier
 witness → method signature: Gospel

Implementation Commentary:

No local comments provided.

9.10.2 Instance WITNESSED_TRUTH (Line 52)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance WITNESSED\_TRUTH`.

Implementation Commentary:

No local comments provided.

9.10.3 Inductive Truth (Line 89)

This declaration represents a logical constructive sum type named `Truth`. It represents a disjoint union populated by the following constructors:

`logic` $\rightarrow$ constructor signature: `Prop  $\Rightarrow$  Truth`

`fact` $\rightarrow$ constructor signature: `Gospel  $\Rightarrow$  Prop  $\Rightarrow$  Truth  $\Rightarrow$  Truth`

Implementation Commentary:

No local comments provided.

9.10.4 Structure UniverseTensor (Line 108)

This declaration represents a logical record structure named `UniverseTensor`. It constructs records using the constructor `UniverseTensor.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier]`

`a` $\rightarrow$ type signature: `ADMISSIBLE Value Carrier]`

`c` $\rightarrow$ type signature: `COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier]`

`r` $\rightarrow$ type signature: `RESIDUE Value Carrier]`

`b` $\rightarrow$ type signature: `BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier]`

`n` $\rightarrow$ type signature: `NUMERIC Value Carrier]`

`h` $\rightarrow$ type signature: `REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ type signature: `PHYSICAL Value Carrier]`

`z` $\rightarrow$ type signature: `COMPARABLE Value Carrier]`

`particle` $\rightarrow$ type signature: `OBSERVED Value Carrier]`

`frquency` $\rightarrow$ type signature: `PRESENT Value Carrier]`

`what\_meesa\_saying` $\rightarrow$ type signature: `MEASURABLE Value Carrier]`

`zero` $\rightarrow$ type signature: `GUNGAN Value Carrier]`

`one` → type signature: `SOURCE Value Carrier`
`result` → type signature: `EXECUTED Value Carrier`
`value` → type signature: `VALUE Value Carrier`
`length` → type signature: `MAGNITUDE Value Carrier`
`scaled` → type signature: `SCALED Value Carrier`
`oriented` → type signature: `LOAD Value Carrier`
`matter` → type signature: `FINITE\_ELEPHANT Value Carrier`
`model` → type signature: `BULLSHIT Value Carrier`
`space` → type signature: `PROPAGANDA Value Carrier`
`scientist` → type signature: `ACOLYTE Value Carrier`
`ideology` → type signature: `SCIENTIFIC Value Carrier`
`gospel` → type signature: `TRUTH Value Carrier`
`account` → type signature: `WITNESSED Value Carrier`
`frame\_of\_reference` → type signature: `ReligiousProcess Value Carrier`
`reality` → type signature: `Truth`

Implementation Commentary:

We have learned this through Quantum Eletrodynamics. Damn, there's that fade away three nothing but net over `__GODEL__` `__COHEN__` `__CANTOR__` `__HILBERT__` and ... No... Without them, I would have stepped on the land mines. It is `BECAUSE` of them that I can see the land mines and avoid them.

9.10.5 Class REAL (Line 151)

This declaration represents a logical typeclass constraint named `REAL`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`
`Carrier` → method signature: `CarrierProcess Value)`
`d` → method signature: `DISTINGUISHABLE Value Carrier`
`a` → method signature: `ADMISSIBLE Value Carrier`
`c` → method signature: `COUNTABLE Value Carrier`
`e` → method signature: `ENCODED Value Carrier`
`r` → method signature: `RESIDUE Value Carrier`
`b` → method signature: `BINARY Value Carrier`
`f` → method signature: `REPEATABLE Value Carrier`

```

n → method signature: NUMERIC Value Carrier]
h → method signature: REPRESENTABLE Value Carrier]
p → method signature: PHYSICAL Value Carrier]
z → method signature: COMPARABLE Value Carrier]
particle → method signature: OBSERVED Value Carrier]
frequency → method signature: PRESENT Value Carrier]
what\_meesa\_saying → method signature: MEASURABLE Value Carrier]
zero → method signature: GUNGAN Value Carrier]
one → method signature: SOURCE Value Carrier]
result → method signature: EXECUTED Value Carrier]
value → method signature: VALUE Value Carrier]
length → method signature: MAGNITUDE Value Carrier]
scaled → method signature: SCALED Value Carrier]
oriented → method signature: LOAD Value Carrier]
matter → method signature: FINITE\_ELEPHANT Value Carrier]
model → method signature: BULLSHIT Value Carrier]
space → method signature: PROPAGANDA Value Carrier]
scientist → method signature: ACOLYTE Value Carrier]
ideology → method signature: SCIENTIFIC Value Carrier]
gospel → method signature: TRUTH Value Carrier]
account → method signature: WITNESSED Value Carrier]
universal\_observer → method signature: UniverseTensor Value Carrier
current\_status → method signature: Truth

```

Implementation Commentary:

No local comments provided.

9.10.6 Instance REAL_WITNESSED (Line 189)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance REAL\_WITNESSED`.

Implementation Commentary:

No local comments provided.

9.10.7 Inductive Variation (Line 227)

This declaration represents a logical constructive sum type named `Variation`. It represents a disjoint union populated by the following constructors:

```

newton → constructor signature: Gospel \ensuremath {\to } Prop \ensuremath {\to }
      } Variation

gateaux → constructor signature: Gospel \ensuremath {\to } Prop \ensuremath {\to }
      } Prop \ensuremath {\to } Variation \ensuremath {\to } Variation

frechet → constructor signature: Gospel \ensuremath {\to } Prop \ensuremath {\to }
      } Prop \ensuremath {\to } Prop \ensuremath {\to } Variation \ensuremath {\to }
      {\to } Variation \ensuremath {\to } Variation

```

Implementation Commentary:

No local comments provided.

9.10.8 Structure BigRedDogProcess (Line 265)

This declaration represents a logical record structure named `BigRedDogProcess`. It constructs records using the constructor `BigRedDogProcess.mk` with the following fields:

```

Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier]

a → type signature: ADMISSIBLE Value Carrier]

c → type signature: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier]

r → type signature: RESIDUE Value Carrier]

b → type signature: BINARY Value Carrier]

f → type signature: REPEATABLE Value Carrier]

n → type signature: NUMERIC Value Carrier]

h → type signature: REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier]

z → type signature: COMPARABLE Value Carrier]

particle → type signature: OBSERVED Value Carrier]

frquency → type signature: PRESENT Value Carrier]

what\_meesa\_saying → type signature: MEASURABLE Value Carrier]

zero → type signature: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier]

```

`result` → type signature: EXECUTED Value Carrier]
`value` → type signature: VALUE Value Carrier]
`length` → type signature: MAGNITUDE Value Carrier]
`scaled` → type signature: SCALED Value Carrier]
`oriented` → type signature: LOAD Value Carrier]
`matter` → type signature: FINITE_ELEPHANT Value Carrier]
`model` → type signature: BULLSHIT Value Carrier]
`space` → type signature: PROPAGANDA Value Carrier]
`scientist` → type signature: ACOLYTE Value Carrier]
`ideology` → type signature: SCIENTIFIC Value Carrier]
`gospel` → type signature: TRUTH Value Carrier]
`account` → type signature: WITNESSED Value Carrier]
`real` → type signature: REAL Value Carrier]
`universal\_observer` → type signature: UniverseTensor Value Carrier
`differential\_equation` → type signature: Variation

Implementation Commentary:

No local comments provided.

9.10.9 Class LOCAL (Line 313)

This declaration represents a logical typeclass constraint named `LOCAL`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier]
`a` → method signature: ADMISSIBLE Value Carrier]
`c` → method signature: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier]
`r` → method signature: RESIDUE Value Carrier]
`b` → method signature: BINARY Value Carrier]
`f` → method signature: REPEATABLE Value Carrier]
`n` → method signature: NUMERIC Value Carrier]

`h` → method signature: REPRESENTABLE Value Carrier]
`p` → method signature: PHYSICAL Value Carrier]
`z` → method signature: COMPARABLE Value Carrier]
`particle` → method signature: OBSERVED Value Carrier]
`frequency` → method signature: PRESENT Value Carrier]
`what\_meesa\_saying` → method signature: MEASURABLE Value Carrier]
`zero` → method signature: GUNGAN Value Carrier]
`one` → method signature: SOURCE Value Carrier]
`result` → method signature: EXECUTED Value Carrier]
`value` → method signature: VALUE Value Carrier]
`length` → method signature: MAGNITUDE Value Carrier]
`scaled` → method signature: SCALED Value Carrier]
`oriented` → method signature: LOAD Value Carrier]
`matter` → method signature: FINITE_ELEPHANT Value Carrier]
`model` → method signature: BULLSHIT Value Carrier]
`space` → method signature: PROPAGANDA Value Carrier]
`scientist` → method signature: ACOLYTE Value Carrier]
`ideology` → method signature: SCIENTIFIC Value Carrier]
`gospel` → method signature: TRUTH Value Carrier]
`account` → method signature: WITNESSED Value Carrier]
`imaginary` → method signature: REAL Value Carrier]
`real` → method signature: REAL Value Carrier)
`theory` → method signature: BigRedDogProcess Value Carrier
`delta` → method signature: Prop

Implementation Commentary:

No local comments provided.

9.10.10 Instance LOCAL_REAL (Line 352)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance LOCAL\_REAL`.

Implementation Commentary:

No local comments provided.

9.11 File: Episode11.lean

9.11.1 Inductive SpaceTimePath (Line 12)

This declaration represents a logical constructive sum type named `SpaceTimePath`. It represents a disjoint union populated by the following constructors:

```
einstein → constructor signature: Fact \ensuremath {\to } SpaceTimePath

white\_hole → constructor signature: Fact \ensuremath {\to } Type i \ensuremath {\to } SpaceTimePath \ensuremath {\to } SpaceTimePath

blackhole → constructor signature: Prop \ensuremath {\to } Type (i+1) \ensuremath {\to } SpaceTimePath \ensuremath {\to } SpaceTimePath

geodesic → constructor signature: Fact \ensuremath {\to } Type i \ensuremath {\to } Prop \ensuremath {\to } Type (i+1) \ensuremath {\to } SpaceTimePath \ensuremath {\to } SpaceTimePath \ensuremath {\to } SpaceTimePath
```

Implementation Commentary:

No local comments provided.

9.11.2 Structure CalculusProcess (Line 63)

This declaration represents a logical record structure named `CalculusProcess`. It constructs records using the constructor `CalculusProcess.mk` with the following fields:

```
Value → type signature: Type i)

Carrier → type signature: CarrierProcess Value)

d → type signature: DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier]
  [c: COUNTABLE Value Carrier]

e → type signature: ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b: BINARY
  Value Carrier]

f → type signature: REPEATABLE Value Carrier] [n: NUMERIC Value Carrier] [h:
  REPRESENTABLE Value Carrier]

p → type signature: PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier] [particle:
  OBSERVED Value Carrier]

frquency → type signature: PRESENT Value Carrier] [what\_meesa\_saying: MEASURABLE
  Value Carrier] [zero: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier] [result: EXECUTED Value Carrier]
  [value: VALUE Value Carrier]

length → type signature: MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier]
  [oriented: LOAD Value Carrier]

matter → type signature: FINITE\_ELEPHANT Value Carrier] [model: BULLSHIT Value
  Carrier] [space: PROPAGANDA Value Carrier]

scientist → type signature: ACOLYTE Value Carrier] [ideology: SCIENTIFIC Value
  Carrier] [gospel: TRUTH Value Carrier]
```

account → type signature: WITNESSED Value Carrier] [real: REAL Value Carrier]

imaginary → type signature: REAL Value Carrier)

derivative → type signature: BigRedDogProcess Value Carrier

function → type signature: SpaceTimePath

converged → type signature: Fact

sink → type signature: Type (i+1)

Implementation Commentary:

No local comments provided.

9.11.3 Class UNIVERSAL (Line 108)

This declaration represents a logical typeclass constraint named UNIVERSAL. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier]
[c: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b:
BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier] [n: NUMERIC Value Carrier]
[h: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier]
[particle: OBSERVED Value Carrier]

frquency → method signature: PRESENT Value Carrier] [what_meesa_saying: MEASURABLE
Value Carrier] [zero: GUNGAN Value Carrier]

one → method signature: SOURCE Value Carrier] [result: EXECUTED Value Carrier]
[value: VALUE Value Carrier]

length → method signature: MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier]
[oriented: LOAD Value Carrier]

matter → method signature: FINITE_ELEPHANT Value Carrier] [model: BULLSHIT
Value Carrier] [space: PROPAGANDA Value Carrier]

scientist → method signature: ACOLYTE Value Carrier] [ideology: SCIENTIFIC
Value Carrier] [gospel: TRUTH Value Carrier]

account → method signature: WITNESSED Value Carrier] [real: REAL Value Carrier]

the_compiler → method signature: CalculusProcess Value Carrier real

source_program → method signature: SpaceTimePath

`compiled\_program` $\rightarrow$ method signature: `SpaceTimePath`

Implementation Commentary:

No local comments provided.

9.11.4 Instance `UNIVERSAL_REAL` (Line 130)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance UNIVERSAL\_REAL`.

Implementation Commentary:

No local comments provided.

9.11.5 Inductive `YarnTheory` (Line 158)

This declaration represents a logical constructive sum type named `YarnTheory`. It represents a disjoint union populated by the following constructors:

`stokes` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} YarnTheory`

`fibers` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} YarnTheory \ensuremath{\{\to\}} YarnTheory`

`fabric` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Fact \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} SpaceTimePath \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} Prop \ensuremath{\{\to\}} YarnTheory \ensuremath{\{\to\}} YarnTheory \ensuremath{\{\to\}} YarnTheory`

Implementation Commentary:

No local comments provided.

9.11.6 Structure `HeartbeatProcess` (Line 209)

This declaration represents a logical record structure named `HeartbeatProcess`. It constructs records using the constructor `HeartbeatProcess.mk` with the following fields:

`Value` $\rightarrow$ type signature: `Type i)`

`Carrier` $\rightarrow$ type signature: `CarrierProcess Value)`

`d` $\rightarrow$ type signature: `DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier] [c: COUNTABLE Value Carrier]`

`e` $\rightarrow$ type signature: `ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b: BINARY Value Carrier]`

`f` $\rightarrow$ type signature: `REPEATABLE Value Carrier] [n: NUMERIC Value Carrier] [h: REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ type signature: `PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier] [particle: OBSERVED Value Carrier]`

```

frequency → type signature: PRESENT Value Carrier] [what\_meesa\_saying: MEASURABLE
Value Carrier] [zero: GUNGAN Value Carrier]

one → type signature: SOURCE Value Carrier] [result: EXECUTED Value Carrier]
[value: VALUE Value Carrier]

length → type signature: MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier]
[oriented: LOAD Value Carrier]

matter → type signature: FINITE\_ELEPHANT Value Carrier] [model: BULLSHIT Value
Carrier] [space: PROPAGANDA Value Carrier]

scientist → type signature: ACOLYTE Value Carrier] [ideology: SCIENTIFIC Value
Carrier] [gospel: TRUTH Value Carrier]

account → type signature: WITNESSED Value Carrier] [real: REAL Value Carrier]

imaginary → type signature: REAL Value Carrier)

computer\_science → type signature: UNIVERSAL Value Carrier)

bullshit\_meter → type signature: CalculusProcess Value Carrier real

current\_reading → type signature: SpaceTimePath

accumulated\_bullshit → type signature: YarnTheory

```

Implementation Commentary:

No local comments provided.

9.12 File: Episode12.lean**9.12.1 Class LOGICAL (Line 13)**

This declaration represents a logical typeclass constraint named **LOGICAL**. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

```

Value → method signature: Type i)

Carrier → method signature: CarrierProcess Value)

d → method signature: DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier]
[c: COUNTABLE Value Carrier]

e → method signature: ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b:
BINARY Value Carrier]

f → method signature: REPEATABLE Value Carrier] [n: NUMERIC Value Carrier]
[h: REPRESENTABLE Value Carrier]

p → method signature: PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier]
[particle: OBSERVED Value Carrier]

frequency → method signature: PRESENT Value Carrier] [what\_meesa\_saying: MEASURABLE
Value Carrier] [zero: GUNGAN Value Carrier]

```

`one` → method signature: `SOURCE Value Carrier`] [`result:` `EXECUTED Value Carrier`]
 [`value:` `VALUE Value Carrier`]

`length` → method signature: `MAGNITUDE Value Carrier`] [`scaled:` `SCALED Value Carrier`]
 [`oriented:` `LOAD Value Carrier`]

`matter` → method signature: `FINITE\_ELEPHANT Value Carrier`] [`model:` `BULLSHIT Value Carrier`] [`space:` `PROPAGANDA Value Carrier`]

`scientist` → method signature: `ACOLYTE Value Carrier`] [`ideology:` `SCIENTIFIC Value Carrier`] [`gospel:` `TRUTH Value Carrier`]

`account` → method signature: `WITNESSED Value Carrier`] [`real:` `REAL Value Carrier`]
 [`imaginary:` `REAL Value Carrier`]

`delta` → method signature: `UNIVERSAL Value Carrier`]

`feelings` → method signature: `HeartbeatProcess Value Carrier` `imaginary delta`

`ekg` → method signature: `Calibration.EKG`

Implementation Commentary:

No local comments provided.

9.12.2 Instance LOGICAL_UNIVERSAL (Line 34)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance LOGICAL\_UNIVERSAL`.

Implementation Commentary:

No local comments provided.

9.12.3 Structure ElaborationProcess (Line 90)

This declaration represents a logical record structure named `ElaborationProcess`. It constructs records using the constructor `ElaborationProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`d` → type signature: `DISTINGUISHABLE Value Carrier`] [`a:` `ADMISSIBLE Value Carrier`]
 [`c:` `COUNTABLE Value Carrier`]

`e` → type signature: `ENCODED Value Carrier`] [`r:` `RESIDUE Value Carrier`] [`b:` `BINARY Value Carrier`]

`f` → type signature: `REPEATABLE Value Carrier`] [`n:` `NUMERIC Value Carrier`] [`h:` `REPRESENTABLE Value Carrier`]

`p` → type signature: `PHYSICAL Value Carrier`] [`z:` `COMPARABLE Value Carrier`] [`particle:` `OBSERVED Value Carrier`]

`frquency` → type signature: `PRESENT Value Carrier`] [`what\_meesa\_saying:` `MEASURABLE Value Carrier`] [`zero:` `GUNGAN Value Carrier`]

`one` → type signature: SOURCE Value Carrier] [result: EXECUTED Value Carrier]
 [value: VALUE Value Carrier]

`length` → type signature: MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier]
 [oriented: LOAD Value Carrier]

`matter` → type signature: FINITE_ELEPHANT Value Carrier] [model: BULLSHIT Value
 Carrier] [space: PROPAGANDA Value Carrier]

`scientist` → type signature: ACOLYTE Value Carrier] [ideology: SCIENTIFIC Value
 Carrier] [gospel: TRUTH Value Carrier]

`account` → type signature: WITNESSED Value Carrier] [real: REAL Value Carrier]
 [LOCAL Value Carrier real] (imaginary: REAL Value Carrier)

`delta` → type signature: UNIVERSAL Value Carrier]

`prop` → type signature: LOGICAL Value Carrier]

`stamina` → type signature: HeartbeatProcess Value Carrier imaginary delta

`calibration` → type signature: Calibration.EKG

`computer\_state` → type signature: ComputerProgram

Implementation Commentary:

No local comments provided.

9.12.4 Class HALTED (Line 131)

This declaration represents a logical typeclass constraint named HALTED. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type i)

`Carrier` → method signature: CarrierProcess Value)

`d` → method signature: DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier]
 [c: COUNTABLE Value Carrier]

`e` → method signature: ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b:
 BINARY Value Carrier]

`f` → method signature: REPEATABLE Value Carrier] [n: NUMERIC Value Carrier]
 [h: REPRESENTABLE Value Carrier]

`p` → method signature: PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier]
 [particle: OBSERVED Value Carrier]

`frquency` → method signature: PRESENT Value Carrier] [what_meesa_saying: MEASURABLE
 Value Carrier] [zero: GUNGAN Value Carrier]

`one` → method signature: SOURCE Value Carrier] [result: EXECUTED Value Carrier]
 [value: VALUE Value Carrier]

length → method signature: MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier]
 [oriented: LOAD Value Carrier]

matter → method signature: FINITE_ELEPHANT Value Carrier] [model: BULLSHIT
 Value Carrier] [space: PROPAGANDA Value Carrier]

scientist → method signature: ACOLYTE Value Carrier] [ideology: SCIENTIFIC
 Value Carrier] [gospel: TRUTH Value Carrier]

account → method signature: WITNESSED Value Carrier] [real: REAL Value Carrier]
 [delta: UNIVERSAL Value Carrier]

prop → method signature: LOGICAL Value Carrier]

scientific_paper → method signature: ElaborationProcess Value Carrier real

Implementation Commentary:

No local comments provided.

9.12.5 Instance HALTED_LOGICAL (Line 155)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by instance HALTED_LOGICAL.

Implementation Commentary:

No local comments provided.

9.13 File: Episode13.lean**9.13.1 Inductive Measurement (Line 10)**

This declaration represents a logical constructive sum type named `Measurement`. It represents a disjoint union populated by the following constructors:

origin → constructor signature: Fact $\{\text{to}\}$ Number $\{\text{to}\}$
 $\}$ Type i $\{\text{to}\}$ Measurement

distance → constructor signature: Fact $\{\text{to}\}$ Number $\{\text{to}\}$
 $\}$ Number $\{\text{to}\}$ Type i $\{\text{to}\}$ Type $(i+1)$ $\{\text{to}\}$ Measurement $\{\text{to}\}$ Measurement

speed → constructor signature: Fact $\{\text{to}\}$ Number $\{\text{to}\}$
 $\}$ Number $\{\text{to}\}$ Number $\{\text{to}\}$ Type i $\{\text{to}\}$
 $\{\text{to}\}$ Type $(i+1)$ $\{\text{to}\}$ Type i $\{\text{to}\}$ Measurement
 $\{\text{to}\}$ Measurement $\{\text{to}\}$ Measurement

Implementation Commentary:

No local comments provided.

9.13.2 Instance : LE Measurement where (Line 27)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by instance : LE Measurement where.

Implementation Commentary:

No local comments provided.

9.13.3 Structure LeanProcess (Line 31)

This declaration represents a logical record structure named `LeanProcess`. It constructs records using the constructor `LeanProcess.mk` with the following fields:

`Value` → type signature: `Type i`)

`Carrier` → type signature: `CarrierProcess Value`)

`d` → type signature: `DISTINGUISHABLE Value Carrier` [`a`: `ADMISSIBLE Value Carrier`]
[`c`: `COUNTABLE Value Carrier`]

`e` → type signature: `ENCODED Value Carrier` [`r`: `RESIDUE Value Carrier`] [`b`: `BINARY Value Carrier`]

`f` → type signature: `REPEATABLE Value Carrier` [`n`: `NUMERIC Value Carrier`] [`h`:
`REPRESENTABLE Value Carrier`]

`p` → type signature: `PHYSICAL Value Carrier` [`z`: `COMPARABLE Value Carrier`] [`particle`:
`OBSERVED Value Carrier`]

`frequency` → type signature: `PRESENT Value Carrier` [`what\_meesa\_saying`: `MEASURABLE Value Carrier`]
[`zero`: `GUNGAN Value Carrier`]

`one` → type signature: `SOURCE Value Carrier` [`result`: `EXECUTED Value Carrier`]
[`value`: `VALUE Value Carrier`]

`length` → type signature: `MAGNITUDE Value Carrier` [`scaled`: `SCALED Value Carrier`]
[`oriented`: `LOAD Value Carrier`]

`matter` → type signature: `FINITE\_ELEPHANT Value Carrier` [`model`: `BULLSHIT Value Carrier`]
[`space`: `PROPAGANDA Value Carrier`]

`scientist` → type signature: `ACOLYTE Value Carrier` [`ideology`: `SCIENTIFIC Value Carrier`]
[`gospel`: `TRUTH Value Carrier`]

`account` → type signature: `WITNESSED Value Carrier` [`real`: `REAL Value Carrier`]
(`imaginary`: `REAL Value Carrier`) [`delta`: `UNIVERSAL Value Carrier`]

`prop` → type signature: `LOGICAL Value Carrier`

`executable` → type signature: `HALTED Value Carrier`

`description` → type signature: `ElaborationProcess Value Carrier real`

`length` → type signature: `Number`

`velocity` → type signature: `Measurement`

`projection` → type signature: `Type i`

Implementation Commentary:

No local comments provided.

9.13.4 Class MEASURED (Line 63)

This declaration represents a logical typeclass constraint named `MEASURED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i`)

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value`)

`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier` [`a`: `ADMISSIBLE Value Carrier`] [`c`: `COUNTABLE Value Carrier`]

`e` $\rightarrow$ method signature: `ENCODED Value Carrier` [`r`: `RESIDUE Value Carrier`] [`b`: `BINARY Value Carrier`]

`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier` [`n`: `NUMERIC Value Carrier`] [`h`: `REPRESENTABLE Value Carrier`]

`p` $\rightarrow$ method signature: `PHYSICAL Value Carrier` [`z`: `COMPARABLE Value Carrier`] [`particle`: `OBSERVED Value Carrier`]

`frequency` $\rightarrow$ method signature: `PRESENT Value Carrier`] [`what\_meesa\_saying`: `MEASURABLE Value Carrier`] [`zero`: `GUNGAN Value Carrier`]

`one` $\rightarrow$ method signature: `SOURCE Value Carrier`] [`result`: `EXECUTED Value Carrier`] [`value`: `VALUE Value Carrier`]

`length` $\rightarrow$ method signature: `MAGNITUDE Value Carrier`] [`scaled`: `SCALED Value Carrier`] [`oriented`: `LOAD Value Carrier`]

`matter` $\rightarrow$ method signature: `FINITE\_ELEPHANT Value Carrier`] [`model`: `BULLSHIT Value Carrier`] [`space`: `PROPAGANDA Value Carrier`]

`scientist` $\rightarrow$ method signature: `ACOLYTE Value Carrier`] [`ideology`: `SCIENTIFIC Value Carrier`] [`gospel`: `TRUTH Value Carrier`]

`account` $\rightarrow$ method signature: `WITNESSED Value Carrier`] [`real`: `REAL Value Carrier`] (`imaginary`: `REAL Value Carrier`) [`delta`: `UNIVERSAL Value Carrier`]

`prop` $\rightarrow$ method signature: `LOGICAL Value Carrier`]

`executable` $\rightarrow$ method signature: `HALTED Value Carrier`]

`satire` $\rightarrow$ method signature: `LeanProcess Value Carrier real`

Implementation Commentary:

No local comments provided.

9.14 File: Episode14.lean

9.14.1 Inductive CompilerTape (Line 11)

This declaration represents a logical constructive sum type named `CompilerTape`. It represents a disjoint union populated by the following constructors:

`boot` $\rightarrow$ constructor signature: `Fact \ensuremath{\{\to\}} Type i \ensuremath{\{\to\}}`
`} CompilerTape`

`strap` $\rightarrow$ constructor signature: $\text{Fact } \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{Fact } \backslash \text{ensuremath } \{\backslash \text{to } \}$
 $\text{Type } i \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{Type } (i+1) \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{CompilerTape } \backslash \text{ensuremath } \{\backslash \text{to } \}$ CompilerTape

`compute` $\rightarrow$ constructor signature: $\text{Fact } \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{Fact } \backslash \text{ensuremath } \{\backslash \text{to } \}$
 $\}$ $\text{Prop } \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{Type } i \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{Type } (i+1) \backslash \text{ensuremath } \{\backslash \text{to } \}$
 $\{\backslash \text{to } \}$ $\text{Type } (i+1) \backslash \text{ensuremath } \{\backslash \text{to } \}$ $\text{CompilerTape } \backslash \text{ensuremath } \{\backslash \text{to } \}$ CompilerTape

Implementation Commentary:

No local comments provided.

9.14.2 Instance : LE CompilerTape where (Line 30)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE CompilerTape where`.

Implementation Commentary:

No local comments provided.

9.14.3 Instance : LT CompilerTape where (Line 32)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT CompilerTape where`.

Implementation Commentary:

No local comments provided.

9.14.4 Structure CompilerOutput (Line 36)

This declaration represents a logical record structure named `CompilerOutput`. It constructs records using the constructor `CompilerOutput.mk` with the following fields:

`Value` $\rightarrow$ type signature: $\text{Type } i)$

`Carrier` $\rightarrow$ type signature: $\text{CarrierProcess Value})$

`d` $\rightarrow$ type signature: $\text{DISTINGUISHABLE Value Carrier}]$ $[\text{a: ADMISSIBLE Value Carrier}]$
 $[\text{c: COUNTABLE Value Carrier}]$

`e` $\rightarrow$ type signature: $\text{ENCODED Value Carrier}]$ $[\text{r: RESIDUE Value Carrier}]$ $[\text{b: BINARY Value Carrier}]$

`f` $\rightarrow$ type signature: $\text{REPEATABLE Value Carrier}]$ $[\text{n: NUMERIC Value Carrier}]$ $[\text{h: REPRESENTABLE Value Carrier}]$

`p` $\rightarrow$ type signature: $\text{PHYSICAL Value Carrier}]$ $[\text{z: COMPARABLE Value Carrier}]$ $[\text{particle: OBSERVED Value Carrier}]$

`frequency` $\rightarrow$ type signature: $\text{PRESENT Value Carrier}]$ $[\text{what_meesa_saying: MEASURABLE Value Carrier}]$ $[\text{zero: GUNGAN Value Carrier}]$

`one` $\rightarrow$ type signature: $\text{SOURCE Value Carrier}]$ $[\text{result: EXECUTED Value Carrier}]$
 $[\text{value: VALUE Value Carrier}]$

`length` $\rightarrow$ type signature: $\text{MAGNITUDE Value Carrier}]$ $[\text{scaled: SCALED Value Carrier}]$
 $[\text{oriented: LOAD Value Carrier}]$

`matter` → type signature: `FINITE\_ELEPHANT Value Carrier` [`model`: `BULLSHIT Value Carrier`] [`space`: `PROPAGANDA Value Carrier`]
`scientist` → type signature: `ACOLYTE Value Carrier` [`ideology`: `SCIENTIFIC Value Carrier`] [`gospel`: `TRUTH Value Carrier`]
`account` → type signature: `WITNESSED Value Carrier` [`real`: `REAL Value Carrier`] (`imaginary`: `REAL Value Carrier`) [`delta`: `UNIVERSAL Value Carrier`]
`prop` → type signature: `LOGICAL Value Carrier` [`executable`: `HALTED Value Carrier`] [`measured`: `MEASURED Value Carrier imaginary`]
`satire` → type signature: `LeanProcess Value Carrier real`
`tape` → type signature: `CompilerTape`

Implementation Commentary:

No local comments provided.

9.14.5 Class COMPILED (Line 66)

This declaration represents a logical typeclass constraint named `COMPILED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: `Type i)`
`Carrier` → method signature: `CarrierProcess Value)`
`d` → method signature: `DISTINGUISHABLE Value Carrier` [`a`: `ADMISSIBLE Value Carrier`] [`c`: `COUNTABLE Value Carrier`]
`e` → method signature: `ENCODED Value Carrier` [`r`: `RESIDUE Value Carrier`] [`b`: `BINARY Value Carrier`]
`f` → method signature: `REPEATABLE Value Carrier` [`n`: `NUMERIC Value Carrier`] [`h`: `REPRESENTABLE Value Carrier`]
`p` → method signature: `PHYSICAL Value Carrier` [`z`: `COMPARABLE Value Carrier`] [`particle`: `OBSERVED Value Carrier`]
`frquency` → method signature: `PRESENT Value Carrier` [`what\_meesa\_saying`: `MEASURABLE Value Carrier`] [`zero`: `GUNGAN Value Carrier`]
`one` → method signature: `SOURCE Value Carrier` [`result`: `EXECUTED Value Carrier`] [`value`: `VALUE Value Carrier`]
`length` → method signature: `MAGNITUDE Value Carrier` [`scaled`: `SCALED Value Carrier`] [`oriented`: `LOAD Value Carrier`]
`matter` → method signature: `FINITE\_ELEPHANT Value Carrier` [`model`: `BULLSHIT Value Carrier`] [`space`: `PROPAGANDA Value Carrier`]
`scientist` → method signature: `ACOLYTE Value Carrier` [`ideology`: `SCIENTIFIC Value Carrier`] [`gospel`: `TRUTH Value Carrier`]
`account` → method signature: `WITNESSED Value Carrier` [`real`: `REAL Value Carrier`] [`epsilon`: `LOCAL Value Carrier real`] [`delta`: `UNIVERSAL Value Carrier`]

```
prop → method signature: LOGICAL Value Carrier] [executable: HALTED Value Carrier]
      [measured: MEASURED Value Carrier real]
```

```
compiler\_output → method signature: CompilerOutput Value Carrier real
```

```
object\_file → method signature: CompilerTape
```

Implementation Commentary:

No local comments provided.

9.14.6 Instance COMPILED_MEASURED (Line 86)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance COMPILED\_MEASURED`.

Implementation Commentary:

No local comments provided.

9.15 File: Episode15.lean

9.15.1 Inductive Bullshit (Line 12)

This declaration represents a logical constructive sum type named `Bullshit`. It represents a disjoint union populated by the following constructors:

```
zero → constructor signature: Fact \ensuremath {\to } Bullshit
```

```
one → constructor signature: Fact \ensuremath {\to } Number \ensuremath {\to }
      CompilerTape \ensuremath {\to } CompilerTape \ensuremath {\to } Bullshit
      \ensuremath {\to } Bullshit
```

```
rest → constructor signature: Fact \ensuremath {\to } Fact \ensuremath {\to }
      Prop \ensuremath {\to } Number \ensuremath {\to } Number \ensuremath {\to }
      \to Number \ensuremath {\to } CompilerTape \ensuremath {\to } CompilerTape
      \ensuremath {\to }
```

Implementation Commentary:

No local comments provided.

9.15.2 Instance : LE Bullshit where (Line 47)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LE Bullshit where`.

Implementation Commentary:

No local comments provided.

9.15.3 Instance : LT Bullshit where (Line 49)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance : LT Bullshit where`.

Implementation Commentary:

No local comments provided.

9.15.4 Structure AtreyuProcess (Line 53)

This declaration represents a logical record structure named `AtreyuProcess`. It constructs records using the constructor `AtreyuProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`d` → type signature: `DISTINGUISHABLE Value Carrier` [`a`: `ADMISSIBLE Value Carrier`]
[`c`: `COUNTABLE Value Carrier`]

`e` → type signature: `ENCODED Value Carrier` [`r`: `RESIDUE Value Carrier`] [`b`: `BINARY Value Carrier`]

`f` → type signature: `REPEATABLE Value Carrier` [`n`: `NUMERIC Value Carrier`] [`h`: `REPRESENTABLE Value Carrier`]

`p` → type signature: `PHYSICAL Value Carrier` [`z`: `COMPARABLE Value Carrier`] [`particle`: `OBSERVED Value Carrier`]

`frequency` → type signature: `PRESENT Value Carrier` [`what\_meesa\_saying`: `MEASURABLE Value Carrier`] [`zero`: `GUNGAN Value Carrier`]

`one` → type signature: `SOURCE Value Carrier` [`result`: `EXECUTED Value Carrier`]
[`value`: `VALUE Value Carrier`]

`length` → type signature: `MAGNITUDE Value Carrier` [`scaled`: `SCALED Value Carrier`]
[`oriented`: `LOAD Value Carrier`]

`matter` → type signature: `FINITE\_ELEPHANT Value Carrier` [`model`: `BULLSHIT Value Carrier`] [`space`: `PROPAGANDA Value Carrier`]

`scientist` → type signature: `ACOLYTE Value Carrier` [`ideology`: `SCIENTIFIC Value Carrier`] [`gospel`: `TRUTH Value Carrier`]

`account` → type signature: `WITNESSED Value Carrier` [`REAL Value Carrier`] (`imaginary`: `REAL Value Carrier`) [`delta`: `UNIVERSAL Value Carrier`]

`prop` → type signature: `LOGICAL Value Carrier` [`executable`: `HALTED Value Carrier`]
[`measured`: `MEASURED Value Carrier imaginary`]

`compiled` → type signature: `COMPILED Value Carrier` where

`compiler\_output` → type signature: `CompilerOutput Value Carrier imaginary`

`next\_measurement` → type signature: `Bullshit`

`stress` → type signature: `Number`

`proof` → type signature: `CompilerTape`

Implementation Commentary:

No local comments provided.

9.15.5 Class TrueOutput (Line 174)

This declaration represents a logical typeclass constraint named `TrueOutput`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` $\rightarrow$ method signature: `Type i)`

`Carrier` $\rightarrow$ method signature: `CarrierProcess Value)`

`d` $\rightarrow$ method signature: `DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier] [c: COUNTABLE Value Carrier]`

`e` $\rightarrow$ method signature: `ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b: BINARY Value Carrier]`

`f` $\rightarrow$ method signature: `REPEATABLE Value Carrier] [n: NUMERIC Value Carrier] [h: REPRESENTABLE Value Carrier]`

`p` $\rightarrow$ method signature: `PHYSICAL Value Carrier] [z: COMPARABLE Value Carrier] [particle: OBSERVED Value Carrier]`

`frequency` $\rightarrow$ method signature: `PRESENT Value Carrier] [what\_meesa\_saying: MEASURABLE Value Carrier] [zero: GUNGAN Value Carrier]`

`one` $\rightarrow$ method signature: `SOURCE Value Carrier] [result: EXECUTED Value Carrier] [value: VALUE Value Carrier]`

`length` $\rightarrow$ method signature: `MAGNITUDE Value Carrier] [scaled: SCALED Value Carrier] [oriented: LOAD Value Carrier]`

`matter` $\rightarrow$ method signature: `FINITE\_ELEPHANT Value Carrier] [model: BULLSHIT Value Carrier] [space: PROPAGANDA Value Carrier]`

`scientist` $\rightarrow$ method signature: `ACOLYTE Value Carrier] [ideology: SCIENTIFIC Value Carrier] [gospel: TRUTH Value Carrier]`

`account` $\rightarrow$ method signature: `WITNESSED Value Carrier] [REAL Value Carrier] (imaginary: REAL Value Carrier) [delta: UNIVERSAL Value Carrier]`

`prop` $\rightarrow$ method signature: `LOGICAL Value Carrier] [executable: HALTED Value Carrier] [measured: MEASURED Value Carrier imaginary]`

`compiled` $\rightarrow$ method signature: `COMPILED Value Carrier]`

`atreyu\_process` $\rightarrow$ method signature: `AtreyuProcess Value Carrier imaginary`

`output` $\rightarrow$ method signature: `Bullshit`

`output\_true` $\rightarrow$ method signature: `TRUE \ensuremath {\le } output`

Implementation Commentary:

No local comments provided.

9.15.6 Instance TRUE_COMPILED (Line 279)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance TRUE\_COMPILED`.

Implementation Commentary:

No local comments provided.

9.15.7 Instance DISTINGUISHABLE_PROP (Line 306)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance DISTINGUISHABLE\_PROP`.

Implementation Commentary:

No local comments provided.

9.15.8 Instance truthDistinct (Line 320)

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance truthDistinct :.`

Implementation Commentary:

No local comments provided.

9.15.9 Inductive Closure (Line 397)

This declaration represents a logical constructive sum type named `Closure`. It represents a disjoint union populated by the following constructors:

`same` → constructor signature: `Fact \ensuremath {\to } Bullshit \ensuremath {\to } Closure`

`different` → constructor signature: `Fact \ensuremath {\to } Bullshit \ensuremath {\to } Option Prop \ensuremath {\to } Closure`

`inferred` → constructor signature: `Fact \ensuremath {\to } Fact \ensuremath {\to } Bullshit \ensuremath {\to } Bullshit \ensuremath {\to } Option Prop \ensuremath {\to } Closure \ensuremath {\to } Closure`

Implementation Commentary:

No local comments provided.

9.15.10 Structure EquivalenceProcess (Line 461)

This declaration represents a logical record structure named `EquivalenceProcess`. It constructs records using the constructor `EquivalenceProcess.mk` with the following fields:

`Value` → type signature: `Type i)`

`Carrier` → type signature: `CarrierProcess Value)`

`d` → type signature: `DISTINGUISHABLE Value Carrier] [a: ADMISSIBLE Value Carrier] [c: COUNTABLE Value Carrier]`

`e` → type signature: `ENCODED Value Carrier] [r: RESIDUE Value Carrier] [b: BINARY Value Carrier]`

`f` → type signature: REPEATABLE Value Carrier] [`n`: NUMERIC Value Carrier] [`h`: REPRESENTABLE Value Carrier]
`p` → type signature: PHYSICAL Value Carrier] [`z`: COMPARABLE Value Carrier] [`particle`: OBSERVED Value Carrier]
`frequency` → type signature: PRESENT Value Carrier] [`what\_meesa\_saying`: MEASURABLE Value Carrier] [`zero`: GUNGAN Value Carrier]
`one` → type signature: SOURCE Value Carrier] [`result`: EXECUTED Value Carrier] [`value`: VALUE Value Carrier]
`length` → type signature: MAGNITUDE Value Carrier] [`scaled`: SCALED Value Carrier] [`oriented`: LOAD Value Carrier]
`matter` → type signature: FINITE_ELEPHANT Value Carrier] [`model`: BULLSHIT Value Carrier] [`space`: PROPAGANDA Value Carrier]
`scientist` → type signature: ACOLYTE Value Carrier] [`ideology`: SCIENTIFIC Value Carrier] [`gospel`: TRUTH Value Carrier]
`account` → type signature: WITNESSED Value Carrier] [`imaginary`: REAL Value Carrier] [`delta`: UNIVERSAL Value Carrier]
`prop` → type signature: LOGICAL Value Carrier] [`executable`: HALTED Value Carrier] [`measured`: MEASURED Value Carrier imaginary]
`compiled` → type signature: COMPILED Value Carrier]
`out` → type signature: TrueOutput Value Carrier imaginary]
`atreyu\_process` → type signature: AtreyuProcess Value Carrier imaginary
`closure` → type signature: Closure

Implementation Commentary:

No local comments provided.

9.15.11 Class INFERRED (Line 495)

This declaration represents a logical typeclass constraint named `INFERRED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Value` → method signature: Type `i`)
`Carrier` → method signature: CarrierProcess Value)
`d` → method signature: DISTINGUISHABLE Value Carrier] [`a`: ADMISSIBLE Value Carrier] [`c`: COUNTABLE Value Carrier]
`e` → method signature: ENCODED Value Carrier] [`r`: RESIDUE Value Carrier] [`b`: BINARY Value Carrier]
`f` → method signature: REPEATABLE Value Carrier] [`n`: NUMERIC Value Carrier] [`h`: REPRESENTABLE Value Carrier]

`p` → method signature: PHYSICAL Value Carrier] [`z`: COMPARABLE Value Carrier]
 [`particle`: OBSERVED Value Carrier]

`frquency` → method signature: PRESENT Value Carrier] [`what\_meesa\_saying`: MEASURABLE
 Value Carrier] [`zero`: GUNGAN Value Carrier]

`one` → method signature: SOURCE Value Carrier] [`result`: EXECUTED Value Carrier]
 [`value`: VALUE Value Carrier]

`length` → method signature: MAGNITUDE Value Carrier] [`scaled`: SCALED Value Carrier]
 [`oriented`: LOAD Value Carrier]

`matter` → method signature: FINITE_ELEPHANT Value Carrier] [`model`: BULLSHIT
 Value Carrier] [`space`: PROPAGANDA Value Carrier]

`scientist` → method signature: ACOLYTE Value Carrier] [`ideology`: SCIENTIFIC
 Value Carrier] [`gospel`: TRUTH Value Carrier]

`account` → method signature: WITNESSED Value Carrier] [`imaginary`: REAL Value
 Carrier] [`delta`: UNIVERSAL Value Carrier]

`prop` → method signature: LOGICAL Value Carrier] [`executable`: HALTED Value Carrier]
 [`measured`: MEASURED Value Carrier imaginary]

`compiled` → method signature: COMPILED Value Carrier]

`out` → method signature: TrueOutput Value Carrier imaginary]

`equivalence\_process` → method signature: EquivalenceProcess Value Carrier

`theory` → method signature: Closure

Implementation Commentary:

No local comments provided.

9.16 File: Episode17.lean**9.16.1 Instance (priority (Line 5))**

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) INFERRED\_TrueOutput\_backward`.

Implementation Commentary:

No local comments provided.

9.16.2 Instance (priority (Line 28))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TrueOutput\_COMPILED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.3 Instance (priority (Line 53))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COMPILED\_HALTED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.4 Instance (priority (Line 77))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COMPILED\_MEASURED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.5 Instance (priority (Line 105))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MEASURED\_HALTED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.6 Instance (priority (Line 135))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) HALTED\_LOGICAL\_backward`.

Implementation Commentary:

No local comments provided.

9.16.7 Instance (priority (Line 166))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOGICAL\_UNIVERSAL\_backward`.

Implementation Commentary:

No local comments provided.

9.16.8 Instance (priority (Line 191))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) UNIVERSAL\_LOCAL\_backward`.

Implementation Commentary:

No local comments provided.

9.16.9 Instance (priority (Line 223))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MEASURED\_HALTED`.

Implementation Commentary:

No local comments provided.

9.16.10 Instance (priority (Line 283))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOCAL\_REAL\_backward`.

Implementation Commentary:

No local comments provided.

9.16.11 Instance (priority (Line 308))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REAL\_WITNESSED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.12 Instance (priority (Line 333))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) WITNESSED\_TRUTH\_backward`.

Implementation Commentary:

No local comments provided.

9.16.13 Instance (priority (Line 357))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRUTH\_SCIENTIFIC\_backward`.

Implementation Commentary:

No local comments provided.

9.16.14 Instance (priority (Line 383))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SCIENTIFIC\_ACOLYTE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.15 Instance (priority (Line 408))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) ACOLYTE\_PROPAGANDA\_backward`.

Implementation Commentary:

No local comments provided.

9.16.16 Instance (priority (Line 433))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PROPAGANDA\_BULLSHIT\_backward`.

Implementation Commentary:

No local comments provided.

9.16.17 Instance (priority (Line 459))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) BULLSHIT\_FINITE\_ELEPHANT\_backward`.

Implementation Commentary:

No local comments provided.

9.16.18 Instance (priority (Line 485))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) FINITE\_ELEPHANT\_LOAD\_backward`.

Implementation Commentary:

No local comments provided.

9.16.19 Instance (priority (Line 511))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOAD\_SCALED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.20 Instance (priority (Line 538))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SCALED\_MAGNITUDE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.21 Instance (priority (Line 565))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MAGNITUDE\_VALUE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.22 Instance (priority (Line 593))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) VALUE\_EXECUTED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.23 Instance (priority (Line 621))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_SOURCE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.24 Instance (priority (Line 650))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SOURCE\_GUNGAN\_backward`.

Implementation Commentary:

No local comments provided.

9.16.25 Instance (priority (Line 678))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) GUNGAN\_MEASURABLE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.26 Instance (priority (Line 707))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MEASURABLE\_PRESENT\_backward`.

Implementation Commentary:

No local comments provided.

9.16.27 Instance (priority (Line 737))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PRESENT\_OBSERVED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.28 Instance (priority (Line 767))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) OBSERVED\_COMPARABLE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.29 Instance (priority (Line 798))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COMPARABLE\_PHYSICAL\_backward`.

Implementation Commentary:

No local comments provided.

9.16.30 Instance (priority (Line 831))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PHYSICAL\_REPRESNTABLE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.31 Instance (priority (Line 867))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REPRESENTABLE\_NUMERIC\_backward`.

Implementation Commentary:

No local comments provided.

9.16.32 Instance (priority (Line 900))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) NUMERIC\_REPEATABLE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.33 Instance (priority (Line 931))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REPEATABLE\_BINARY\_backward`.

Implementation Commentary:

No local comments provided.

9.16.34 Instance (priority (Line 967))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) BINARY\_RESIDUE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.35 Instance (priority (Line 999))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) RESIDUE\_ENCODED\_backward`.

Implementation Commentary:

No local comments provided.

9.16.36 Instance (priority (Line 1031))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) ENCODED\_COUNTABLE\_backward`.

Implementation Commentary:

No local comments provided.

9.16.37 Instance (priority (Line 1063))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COUNTABLE\_ADMISSIBLE\_backward`.

Implementation Commentary:

No local comments provided.

9.17 File: Episode18.lean

9.17.1 Instance (priority (Line 30))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) DISTINGUISHABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.2 Instance (priority (Line 67))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) ADMISSIBLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.3 Instance (priority (Line 106))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COUNTABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.4 Instance (priority (Line 146))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) ENCODED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.5 Instance (priority (Line 188))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) RESIDUE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.6 Instance (priority (Line 229))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) BINARY\_residue`.

Implementation Commentary:

No local comments provided.

9.17.7 Instance (priority (Line 276))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REPEATABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.8 Instance (priority (Line 317))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) NUMERIC\_residue`.

Implementation Commentary:

No local comments provided.

9.17.9 Instance (priority (Line 358))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REPRESENTABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.10 Instance (priority (Line 408))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PHYSICAL\_residue`.

Implementation Commentary:

No local comments provided.

9.17.11 Instance (priority (Line 451))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COMPARABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.12 Instance (priority (Line 494))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) OBSERVED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.13 Instance (priority (Line 538))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PRESENT\_residue`.

Implementation Commentary:

No local comments provided.

9.17.14 Instance (priority (Line 580))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MEASURABLE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.15 Instance (priority (Line 622))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) GUNGAN\_residue`.

Implementation Commentary:

No local comments provided.

9.17.16 Instance (priority (Line 664))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SOURCE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.17 Instance (priority (Line 707))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.18 Instance (priority (Line 752))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) VALUE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.19 Instance (priority (Line 797))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MAGNITUDE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.20 Instance (priority (Line 841))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SCALED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.21 Instance (priority (Line 886))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOAD\_residue`.

Implementation Commentary:

No local comments provided.

9.17.22 Instance (priority (Line 931))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) FINITE\_ELEPHANT\_residue`.

Implementation Commentary:

No local comments provided.

9.17.23 Instance (priority (Line 975))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) BULLSHIT\_residue`.

Implementation Commentary:

No local comments provided.

9.17.24 Instance (priority (Line 1020))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) PROPAGANDA\_residue`.

Implementation Commentary:

No local comments provided.

9.17.25 Instance (priority (Line 1064))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) ACOLYTE\_residue`.

Implementation Commentary:

No local comments provided.

9.17.26 Instance (priority (Line 1108))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) SCIENTIFIC\_residue`.

Implementation Commentary:

No local comments provided.

9.17.27 Instance (priority (Line 1157))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRUTH\_residue`.

Implementation Commentary:

No local comments provided.

9.17.28 Instance (priority (Line 1203))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) WITNESSED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.29 Instance (priority (Line 1251))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) REAL\_residue`.

Implementation Commentary:

No local comments provided.

9.17.30 Instance (priority (Line 1300))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOCAL\_residue`.

Implementation Commentary:

No local comments provided.

9.17.31 Instance (priority (Line 1348))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) UNIVERSAL\_residue`.

Implementation Commentary:

No local comments provided.

9.17.32 Instance (priority (Line 1399))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) LOGICAL\_residue`.

Implementation Commentary:

No local comments provided.

9.17.33 Instance (priority (Line 1451))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) HALTED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.34 Instance (priority (Line 1514))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) MEASURED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.35 Instance (priority (Line 1598))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) COMPILED\_residue`.

Implementation Commentary:

No local comments provided.

9.17.36 Instance (priority (Line 1662))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TrueOutput\_residue`.

Implementation Commentary:

No local comments provided.

9.18 File: Episode19.lean**9.18.1 Class TRACED (Line 10)**

This declaration represents a logical typeclass constraint named `TRACED`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`tape` → method signature: `CompilerTape`

Implementation Commentary:

No local comments provided.

9.18.2 Instance (priority (Line 13))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_seam`.

Implementation Commentary:

No local comments provided.

9.18.3 Instance (priority (Line 18))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_TrueOutput`.

Implementation Commentary:

No local comments provided.

9.18.4 Instance (priority (Line 73))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_COMPILED`.

Implementation Commentary:

No local comments provided.

9.18.5 Instance (priority (Line 137))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_HALTED`.

Implementation Commentary:

No local comments provided.

9.18.6 Instance (priority (Line 202))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_LOGICAL`.

Implementation Commentary:

No local comments provided.

9.18.7 Instance (priority (Line 258))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_UNIVERSAL`.

Implementation Commentary:

No local comments provided.

9.18.8 Instance (priority (Line 314))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_MEASURED`.

Implementation Commentary:

No local comments provided.

9.18.9 Instance (priority (Line 377))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_REAL`.

Implementation Commentary:

No local comments provided.

9.18.10 Instance (priority (Line 446))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_WITNESSED`.

Implementation Commentary:

No local comments provided.

9.18.11 Instance (priority (Line 512))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_SCIENTIFIC`.

Implementation Commentary:

No local comments provided.

9.18.12 Instance (priority (Line 576))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_PROPAGANDA`.

Implementation Commentary:

No local comments provided.

9.18.13 Instance (priority (Line 636))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_TRUTH`.

Implementation Commentary:

No local comments provided.

9.18.14 Instance (priority (Line 698))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_ACOLYTE`.

Implementation Commentary:

No local comments provided.

9.18.15 Instance (priority (Line 761))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_BULLSHIT`.

Implementation Commentary:

No local comments provided.

9.18.16 Instance (priority (Line 824))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_FINITE\_ELEPHANT`.

Implementation Commentary:

No local comments provided.

9.18.17 Instance (priority (Line 887))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_LOAD`.

Implementation Commentary:

No local comments provided.

9.18.18 Instance (priority (Line 950))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_SCALED`.

Implementation Commentary:

No local comments provided.

9.18.19 Instance (priority (Line 1013))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_MAGNITUDE`.

Implementation Commentary:

No local comments provided.

9.18.20 Instance (priority (Line 1075))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_VALUE`.

Implementation Commentary:

No local comments provided.

9.18.21 Instance (priority (Line 1138))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_EXECUTED`.

Implementation Commentary:

No local comments provided.

9.18.22 Instance (priority (Line 1201))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_SOURCE`.

Implementation Commentary:

No local comments provided.

9.18.23 Instance (priority (Line 1264))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_GUNGAN`.

Implementation Commentary:

No local comments provided.

9.18.24 Instance (priority (Line 1327))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_MEASURABLE`.

Implementation Commentary:

No local comments provided.

9.18.25 Instance (priority (Line 1390))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_PRESENT`.

Implementation Commentary:

No local comments provided.

9.18.26 Instance (priority (Line 1452))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_OBSERVED`.

Implementation Commentary:

No local comments provided.

9.18.27 Instance (priority (Line 1514))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_COMPARABLE`.

Implementation Commentary:

No local comments provided.

9.18.28 Instance (priority (Line 1576))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_PHYSICAL`.

Implementation Commentary:

No local comments provided.

9.18.29 Instance (priority (Line 1637))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_REPRESENTABLE`.

Implementation Commentary:

No local comments provided.

9.18.30 Instance (priority (Line 1699))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_NUMERIC`.

Implementation Commentary:

No local comments provided.

9.18.31 Instance (priority (Line 1760))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_REPEATABLE`.

Implementation Commentary:

No local comments provided.

9.18.32 Instance (priority (Line 1822))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_BINARY`.

Implementation Commentary:

No local comments provided.

9.18.33 Instance (priority (Line 1883))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_RESIDUE`.

Implementation Commentary:

No local comments provided.

9.18.34 Instance (priority (Line 1943))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_ENCODED`.

Implementation Commentary:

No local comments provided.

9.18.35 Instance (priority (Line 2003))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_COUNTABLE`.

Implementation Commentary:

No local comments provided.

9.18.36 Instance (priority (Line 2063))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_ADMISSIBLE`.

Implementation Commentary:

No local comments provided.

9.18.37 Instance (priority (Line 2123))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) TRACED\_DISTINGUISHABLE`.

Implementation Commentary:

No local comments provided.

9.19 File: Episode20.lean**9.19.1 Class EXECUTED_TRACE (Line 12)**

This declaration represents a logical typeclass constraint named `EXECUTED\_TRACE`. It defines type-level predicates and methods that must be satisfied during compilation. The methods are:

`Lifted` → method signature: `Type j`) where

`register\_value` → method signature: `Lifted`

`universe\_id` → method signature: `Number`

`tape` → method signature: `CompilerTape`

Implementation Commentary:

No local comments provided.

9.19.2 Instance (priority (Line 18))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_origin`.

Implementation Commentary:

No local comments provided.

9.19.3 Instance (priority (Line 29))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_2`.

Implementation Commentary:

No local comments provided.

9.19.4 Instance (priority (Line 49))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_3`.

Implementation Commentary:

No local comments provided.

9.19.5 Instance (priority (Line 64))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_4`.

Implementation Commentary:

No local comments provided.

9.19.6 Instance (priority (Line 79))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_5`.

Implementation Commentary:

No local comments provided.

9.19.7 Instance (priority (Line 94))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_6`.

Implementation Commentary:

No local comments provided.

9.19.8 Instance (priority (Line 109))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_7`.

Implementation Commentary:

No local comments provided.

9.19.9 Instance (priority (Line 124))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_8`.

Implementation Commentary:

No local comments provided.

9.19.10 Instance (priority (Line 139))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_9`.

Implementation Commentary:

No local comments provided.

9.19.11 Instance (priority (Line 154))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_10`.

Implementation Commentary:

No local comments provided.

9.19.12 Instance (priority (Line 169))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_11`.

Implementation Commentary:

No local comments provided.

9.19.13 Instance (priority (Line 184))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_12`.

Implementation Commentary:

No local comments provided.

9.19.14 Instance (priority (Line 199))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_13`.

Implementation Commentary:

No local comments provided.

9.19.15 Instance (priority (Line 214))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_14`.

Implementation Commentary:

No local comments provided.

9.19.16 Instance (priority (Line 229))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_15`.

Implementation Commentary:

No local comments provided.

9.19.17 Instance (priority (Line 244))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_16`.

Implementation Commentary:

No local comments provided.

9.19.18 Instance (priority (Line 259))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_17`.

Implementation Commentary:

No local comments provided.

9.19.19 Instance (priority (Line 274))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_18`.

Implementation Commentary:

No local comments provided.

9.19.20 Instance (priority (Line 289))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_19`.

Implementation Commentary:

No local comments provided.

9.19.21 Instance (priority (Line 304))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_20`.

Implementation Commentary:

No local comments provided.

9.19.22 Instance (priority (Line 319))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_21`.

Implementation Commentary:

No local comments provided.

9.19.23 Instance (priority (Line 334))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_22`.

Implementation Commentary:

No local comments provided.

9.19.24 Instance (priority (Line 349))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_23`.

Implementation Commentary:

No local comments provided.

9.19.25 Instance (priority (Line 364))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_24`.

Implementation Commentary:

No local comments provided.

9.19.26 Instance (priority (Line 379))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_25`.

Implementation Commentary:

No local comments provided.

9.19.27 Instance (priority (Line 394))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_26`.

Implementation Commentary:

No local comments provided.

9.19.28 Instance (priority (Line 409))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_27`.

Implementation Commentary:

No local comments provided.

9.19.29 Instance (priority (Line 424))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_28`.

Implementation Commentary:

No local comments provided.

9.19.30 Instance (priority (Line 439))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_29`.

Implementation Commentary:

No local comments provided.

9.19.31 Instance (priority (Line 454))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_30`.

Implementation Commentary:

No local comments provided.

9.19.32 Instance (priority (Line 469))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_31`.

Implementation Commentary:

No local comments provided.

9.19.33 Instance (priority (Line 484))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_32`.

Implementation Commentary:

No local comments provided.

9.19.34 Instance (priority (Line 499))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_33`.

Implementation Commentary:

No local comments provided.

9.19.35 Instance (priority (Line 514))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_34`.

Implementation Commentary:

No local comments provided.

9.19.36 Instance (priority (Line 529))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_35`.

Implementation Commentary:

No local comments provided.

9.19.37 Instance (priority (Line 544))

This declaration represents a logical typeclass instance registration. It instantiates the typeclass relation described by `instance (priority := low) EXECUTED\_climb\_36`.

Implementation Commentary:

No local comments provided.

9.19.38 Structure SlipVerdict (Line 564)

This declaration represents a logical record structure named `SlipVerdict`. It constructs records using the constructor `SlipVerdict.mk` with the following fields:

`slipped` → type signature: `Fact`

`value` → type signature: `Number`

`velocity` → type signature: `Number`

Implementation Commentary:

No local comments provided.

9.20 File: Episode21.lean**9.20.1 Structure BracketedNumber (Line 17)**

This declaration represents a logical record structure named `BracketedNumber`. It constructs records using the constructor `BracketedNumber.mk` with the following fields:

`lower` → type signature: `Number`

`upper` → type signature: `Number`

`value` → type signature: `Number`

Implementation Commentary:

No local comments provided.

9.21 File: Episode22.lean**9.21.1 Inductive QPhase (Line 14)**

This declaration represents a logical constructive sum type named `QPhase`. It represents a disjoint union populated by the following constructors:

`plus` → constructor signature: `QPhase`

`minus` → constructor signature: `QPhase`

Implementation Commentary:

No local comments provided.

9.21.2 Structure RecoveredObservables (Line 34)

This declaration represents a logical record structure named `RecoveredObservables`. It constructs records using the constructor `RecoveredObservables.mk` with the following fields:

`charge` → type signature: `Number`

`mass` → type signature: `Number`

`phase` → type signature: `QPhase`

Implementation Commentary:

No local comments provided.

9.22 File: Episode25.lean

9.22.1 Inductive CompilerTapeCell (Line 23)

This declaration represents a logical constructive sum type named `CompilerTapeCell`. It represents a disjoint union populated by the following constructors:

```
strap → constructor signature: Fact -> Fact -> Type i -> Type (i+1) -> CompilerTapeCell

compute → constructor signature: Fact -> Fact -> Prop -> Type i -> Type (i+1)
      -> Type (i+1) -> CompilerTapeCell
```

Implementation Commentary:

No local comments provided.

9.22.2 Inductive CorridorFace (Line 101)

This declaration represents a logical constructive sum type named `CorridorFace`. It represents a disjoint union populated by the following constructors:

```
charge → constructor signature: No parameter constructor

mass → constructor signature: No parameter constructor

value → constructor signature: No parameter constructor
```

Implementation Commentary:

No local comments provided.

9.22.3 Structure RotatedCorridorRead (Line 138)

This declaration represents a logical record structure named `RotatedCorridorRead`. It constructs records using the constructor `RotatedCorridorRead.mk` with the following fields:

```
precision → type signature: Nat

scale → type signature: Nat

turns → type signature: Nat

reducedTurns → type signature: Nat

activeFace → type signature: CorridorFace

chargeDepth → type signature: Nat

massDepth → type signature: Nat

valueDepth → type signature: Nat

phase → type signature: QPhase

electronBoxValue → type signature: Nat
```

Implementation Commentary:

No local comments provided.

9.22.4 Structure CorridorBinReport (Line 213)

This declaration represents a logical record structure named `CorridorBinReport`. It constructs records using the constructor `CorridorBinReport.mk` with the following fields:

`lower` → type signature: `Nat`

`upper` → type signature: `Nat`

`total` → type signature: `Nat`

`chargeCount` → type signature: `Nat`

`massCount` → type signature: `Nat`

`valueCount` → type signature: `Nat`

`upperReducedTurns` → type signature: `Nat`

`upperFace` → type signature: `CorridorFace`

Implementation Commentary:

No local comments provided.

9.22.5 Structure WheelStimulusRead (Line 243)

This declaration represents a logical record structure named `WheelStimulusRead`. It constructs records using the constructor `WheelStimulusRead.mk` with the following fields:

`label` → type signature: `String`

`signedValue` → type signature: `Int`

`magnitude` → type signature: `Nat`

`tripletIndex` → type signature: `Nat`

`valueForCharge` → type signature: `Nat`

`valueForMass` → type signature: `Nat`

`reducedTurns` → type signature: `Nat`

`face` → type signature: `CorridorFace`

`motionSlip` → type signature: `Bool`

Implementation Commentary:

No local comments provided.

9.22.6 Structure BuildGravitySlipReport (Line 269)

This declaration represents a logical record structure named `BuildGravitySlipReport`. It constructs records using the constructor `BuildGravitySlipReport.mk` with the following fields:

`dial` → type signature: `CorridorBinReport`

`electronBoxValue` → type signature: `Nat`

`gravityFace` → type signature: `CorridorFace`

`dialExtraIsMotion` → type signature: `Bool`

`buildStimuli` → type signature: `List WheelStimulusRead`

`frictionReads` → type signature: `List WheelStimulusRead`

Implementation Commentary:

No local comments provided.

9.23 File: Episode26.lean

9.23.1 Structure ApparatusRatio (Line 21)

This declaration represents a logical record structure named `ApparatusRatio`. It constructs records using the constructor `ApparatusRatio.mk` with the following fields:

`numerator` → type signature: `Nat`

`denominator` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.23.2 Structure CavendishBalanceReport (Line 73)

This declaration represents a logical record structure named `CavendishBalanceReport`. It constructs records using the constructor `CavendishBalanceReport.mk` with the following fields:

`dial` → type signature: `CorridorBinReport`

`sourceLeft` → type signature: `WheelStimulusRead`

`sourceRight` → type signature: `WheelStimulusRead`

`sourcesBalanced` → type signature: `Bool`

`testMass` → type signature: `WheelStimulusRead`

`torsionFiber` → type signature: `WheelStimulusRead`

`arm` → type signature: `Nat`

`separation` → type signature: `Nat`

`observedSlip` → type signature: `WheelStimulusRead`
`observedMassValue` → type signature: `WheelStimulusRead`
`calibration` → type signature: `ApparatusRatio`
`deviceG` → type signature: `ApparatusRatio`
`deviceGScaledAt18` → type signature: `Nat`
`calibrationFloor` → type signature: `Nat`
`calibrationRemainder` → type signature: `Nat`
`inverseCalibrationFloor` → type signature: `Nat`
`inverseCalibrationRemainder` → type signature: `Nat`
`motionResolved` → type signature: `Bool`
`quantumGravityResolved` → type signature: `Bool`

Implementation Commentary:

No local comments provided.

9.24 File: Episode27.lean**9.24.1 Structure QuantumGravityCell (Line 73)**

This declaration represents a logical record structure named `QuantumGravityCell`. It constructs records using the constructor `QuantumGravityCell.mk` with the following fields:

`index` → type signature: `Nat`
`deviceG` → type signature: `ApparatusRatio`
`motionResolved` → type signature: `Bool`
`quantumGravityResolved` → type signature: `Bool`

Implementation Commentary:

No local comments provided.

9.24.2 Structure AlphaTuneReport (Line 90)

This declaration represents a logical record structure named `AlphaTuneReport`. It constructs records using the constructor `AlphaTuneReport.mk` with the following fields:

`alphaTarget` → type signature: `ApparatusRatio`
`alphaTargetScaledAt18` → type signature: `Nat`
`alphaInverseTarget` → type signature: `ApparatusRatio`
`cellG` → type signature: `ApparatusRatio`
`lowerCells` → type signature: `Nat`

`upperCells` → type signature: `Nat`
`bestCells` → type signature: `Nat`
`curvatureCells` → type signature: `Nat`
`lowerStack` → type signature: `ApparatusRatio`
`upperStack` → type signature: `ApparatusRatio`
`bestStack` → type signature: `ApparatusRatio`
`curvatureStack` → type signature: `ApparatusRatio`
`lowerSignedResidual` → type signature: `Int`
`upperSignedResidual` → type signature: `Int`
`curvatureSignedResidual` → type signature: `Int`
`lowerDistance` → type signature: `Nat`
`upperDistance` → type signature: `Nat`
`curvatureDistance` → type signature: `Nat`
`bestDistance` → type signature: `Nat`
`signedSecondDifference` → type signature: `Int`
`observedSecondDifference` → type signature: `Int`
`tuningRatio` → type signature: `ApparatusRatio`
`tuningRatioScaledAt18` → type signature: `Nat`
`selectedCells` → type signature: `List QuantumGravityCell`

Implementation Commentary:

No local comments provided.

9.25 File: Episode28.lean**9.25.1 Inductive ProximityRegime (Line 17)**

This declaration represents a logical constructive sum type named `ProximityRegime`. It represents a disjoint union populated by the following constructors:

`far` → constructor signature: `No parameter constructor`
`calibrated` → constructor signature: `No parameter constructor`
`nonlinear` → constructor signature: `No parameter constructor`
`rupture` → constructor signature: `No parameter constructor`
`contact` → constructor signature: `No parameter constructor`

Implementation Commentary:

No local comments provided.

9.25.2 Structure ProximityRead (Line 59)

This declaration represents a logical record structure named `ProximityRead`. It constructs records using the constructor `ProximityRead.mk` with the following fields:

```
distance → type signature: Nat
predictedSlip → type signature: Option ApparatusRatio
slipFloor → type signature: Nat
slipScaledAt18 → type signature: Nat
slipFace → type signature: CorridorFace
motionSlip → type signature: Bool
pairBreaks → type signature: Bool
regime → type signature: ProximityRegime
```

Implementation Commentary:

No local comments provided.

9.25.3 Structure ProximityBreakReport (Line 99)

This declaration represents a logical record structure named `ProximityBreakReport`. It constructs records using the constructor `ProximityBreakReport.mk` with the following fields:

```
deviceG → type signature: ApparatusRatio
calibratedDistance → type signature: Nat
sourceMassTotal → type signature: Nat
testMass → type signature: Nat
arm → type signature: Nat
ruptureThreshold → type signature: Nat
floorCurvature → type signature: Int
scaledCurvatureAt18 → type signature: Int
reads → type signature: List ProximityRead
```

Implementation Commentary:

No local comments provided.

9.26 File: Episode29.lean

9.26.1 Structure RationalDistance (Line 21)

This declaration represents a logical record structure named `RationalDistance`. It constructs records using the constructor `RationalDistance.mk` with the following fields:

`numerator` → type signature: `Nat`

`denominator` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.26.2 Structure SlipBisectStep (Line 72)

This declaration represents a logical record structure named `SlipBisectStep`. It constructs records using the constructor `SlipBisectStep.mk` with the following fields:

`rung` → type signature: `Nat`

`lowerDistance` → type signature: `RationalDistance`

`upperDistance` → type signature: `RationalDistance`

`probeDistance` → type signature: `RationalDistance`

`probeDistanceScaledAt18` → type signature: `Nat`

`probeSlipFloor` → type signature: `Nat`

`probeSlipScaledAt18` → type signature: `Nat`

`probeFace` → type signature: `CorridorFace`

`crossedSlip` → type signature: `Bool`

Implementation Commentary:

No local comments provided.

9.26.3 Structure SlipBisectBracket (Line 84)

This declaration represents a logical record structure named `SlipBisectBracket`. It constructs records using the constructor `SlipBisectBracket.mk` with the following fields:

`lowerDistance` → type signature: `RationalDistance`

`upperDistance` → type signature: `RationalDistance`

Implementation Commentary:

No local comments provided.

9.26.4 Structure SlipBisectReport (Line 123)

This declaration represents a logical record structure named `SlipBisectReport`. It constructs records using the constructor `SlipBisectReport.mk` with the following fields:

```
targetSlip → type signature: Nat
rungs      → type signature: Nat
lowerDistance → type signature: RationalDistance
upperDistance → type signature: RationalDistance
lowerDistanceScaledAt18 → type signature: Nat
upperDistanceScaledAt18 → type signature: Nat
lowerSlipFloor → type signature: Nat
upperSlipFloor → type signature: Nat
lowerSlipScaledAt18 → type signature: Nat
upperSlipScaledAt18 → type signature: Nat
lowerFace → type signature: CorridorFace
upperFace → type signature: CorridorFace
steps → type signature: List SlipBisectStep
```

Implementation Commentary:

No local comments provided.

9.26.5 Structure SlipBisectSummary (Line 159)

This declaration represents a logical record structure named `SlipBisectSummary`. It constructs records using the constructor `SlipBisectSummary.mk` with the following fields:

```
targetSlip → type signature: Nat
rungs      → type signature: Nat
lowerDistanceScaledAt18 → type signature: Nat
upperDistanceScaledAt18 → type signature: Nat
lowerSlipFloor → type signature: Nat
upperSlipFloor → type signature: Nat
lowerSlipScaledAt18 → type signature: Nat
upperSlipScaledAt18 → type signature: Nat
lowerFace → type signature: CorridorFace
upperFace → type signature: CorridorFace
```

Implementation Commentary:

No local comments provided.

9.27 File: Episode30.lean

9.27.1 Structure SlipPointMoments (Line 86)

This declaration represents a logical record structure named `SlipPointMoments`. It constructs records using the constructor `SlipPointMoments.mk` with the following fields:

```

accumulatedSlipPoint → type signature: Nat
accumulatedSquaredSlipPoint → type signature: Nat
accumulatedCubedSlipPoint → type signature: Nat
estimatedSlipPoint → type signature: ApparatusRatio
variance → type signature: ApparatusRatio
estimatedSlipPointScaledAt18 → type signature: Nat
varianceScaledAt18 → type signature: Nat

```

Implementation Commentary:

No local comments provided.

9.27.2 Structure ChargeMassNormalizationReport (Line 109)

This declaration represents a logical record structure named `ChargeMassNormalizationReport`. It constructs records using the constructor `ChargeMassNormalizationReport.mk` with the following fields:

```

lower → type signature: Nat
upper → type signature: Nat
charge → type signature: SlipPointMoments
mass → type signature: SlipPointMoments
combined → type signature: SlipPointMoments
normalizedCharge → type signature: ApparatusRatio
normalizedMass → type signature: ApparatusRatio
chargeScaledAt18 → type signature: Nat
massScaledAt18 → type signature: Nat
normalizedSumScaledAt18 → type signature: Nat

```

Implementation Commentary:

No local comments provided.

9.28 File: Episode31.lean

9.28.1 Structure MagneticNeedleReport (Line 37)

This declaration represents a logical record structure named `MagneticNeedleReport`. It constructs records using the constructor `MagneticNeedleReport.mk` with the following fields:

`lower` → type signature: `Nat`

`upper` → type signature: `Nat`

`corridorSpin` → type signature: `Nat`

`chargeAccumulatedSlipPoint` → type signature: `Nat`

`massAccumulatedSlipPoint` → type signature: `Nat`

`deflectsTowardMass` → type signature: `Bool`

`magneticImbalance` → type signature: `Nat`

`normalizedMagnetGain` → type signature: `ApparatusRatio`

`normalizedMagnetGainScaledAt18` → type signature: `Nat`

`slipPointMean` → type signature: `ApparatusRatio`

`slipPointMeanScaledAt18` → type signature: `Nat`

`slipPointVariance` → type signature: `ApparatusRatio`

`slipPointVarianceScaledAt18` → type signature: `Nat`

`needleMean` → type signature: `ApparatusRatio`

`needleMeanScaledAt18` → type signature: `Nat`

`needleVariance` → type signature: `ApparatusRatio`

`needleVarianceScaledAt18` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.29 File: Episode32.lean

9.29.1 Inductive CooperElectronOrbitRegime (Line 43)

This declaration represents a logical constructive sum type named `CooperElectronOrbitRegime`. It represents a disjoint union populated by the following constructors:

`contact` → constructor signature: `No parameter constructor`

`ruptured` → constructor signature: `No parameter constructor`

`superUnit` → constructor signature: `No parameter constructor`

`unitBoundary` → constructor signature: No parameter constructor

`boundSubunit` → constructor signature: No parameter constructor

Implementation Commentary:

No local comments provided.

9.29.2 Structure `CooperElectronOrbitRead` (Line 69)

This declaration represents a logical record structure named `CooperElectronOrbitRead`. It constructs records using the constructor `CooperElectronOrbitRead.mk` with the following fields:

`radius` → type signature: `Nat`

`centralCooperMass` → type signature: `Nat`

`electronMass` → type signature: `Nat`

`radialSlip` → type signature: `ApparatusRatio`

`radialSlipFloor` → type signature: `Nat`

`radialSlipFace` → type signature: `CorridorFace`

`pairBreaks` → type signature: `Bool`

`velocitySquared` → type signature: `ApparatusRatio`

`velocitySquaredScaledAt18` → type signature: `Nat`

`velocitySquaredFloor` → type signature: `Nat`

`velocitySquaredFace` → type signature: `CorridorFace`

`stableNaturalOrbit` → type signature: `Bool`

`regime` → type signature: `CooperElectronOrbitRegime`

`magneticNeedleDeflection` → type signature: `ApparatusRatio`

`magneticNeedleDeflectionScaledAt18` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.29.3 Structure `CooperElectronOrbitReport` (Line 112)

This declaration represents a logical record structure named `CooperElectronOrbitReport`. It constructs records using the constructor `CooperElectronOrbitReport.mk` with the following fields:

`gravitationalParameter` → type signature: `ApparatusRatio`

`gravitationalParameterScaledAt18` → type signature: `Nat`

`naturalUnitRadius` → type signature: `Nat`

`selectedOrbit` → type signature: `CooperElectronOrbitRead`

`sweep` → type signature: `List CooperElectronOrbitRead`

Implementation Commentary:

No local comments provided.

9.30 File: Episode33.lean

9.30.1 Structure RationalRadius (Line 25)

This declaration represents a logical record structure named `RationalRadius`. It constructs records using the constructor `RationalRadius.mk` with the following fields:

`numerator` → type signature: `Nat`

`denominator` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.30.2 Inductive LorentzOrbitRegime (Line 63)

This declaration represents a logical constructive sum type named `LorentzOrbitRegime`. It represents a disjoint union populated by the following constructors:

`superUnitUndershoot` → constructor signature: `No parameter constructor`

`lightlikeBoundary` → constructor signature: `No parameter constructor`

`timelikeOvershoot` → constructor signature: `No parameter constructor`

Implementation Commentary:

No local comments provided.

9.30.3 Structure LorentzOrbitRead (Line 77)

This declaration represents a logical record structure named `LorentzOrbitRead`. It constructs records using the constructor `LorentzOrbitRead.mk` with the following fields:

`label` → type signature: `String`

`radius` → type signature: `RationalRadius`

`radiusScaledAt18` → type signature: `Nat`

`velocitySquared` → type signature: `ApparatusRatio`

`velocitySquaredScaledAt18` → type signature: `Nat`

`velocitySquaredFloor` → type signature: `Nat`

`velocitySquaredFace` → type signature: `CorridorFace`

`gammaSquared?` → type signature: `Option ApparatusRatio`

`gammaSquaredScaledAt18?` → type signature: `Option Nat`

`magneticNeedleDeflection` → type signature: `ApparatusRatio`

`magneticNeedleDeflectionScaledAt18` → type signature: `Nat`

`regime` → type signature: `LorentzOrbitRegime`

Implementation Commentary:

No local comments provided.

9.30.4 Structure `LorentzOrbitBracketReport` (Line 113)

This declaration represents a logical record structure named `LorentzOrbitBracketReport`. It constructs records using the constructor `LorentzOrbitBracketReport.mk` with the following fields:

`centerRadius` → type signature: `Nat`

`wobble` → type signature: `ApparatusRatio`

`wobbleScaledAt18` → type signature: `Nat`

`undershoot` → type signature: `LorentzOrbitRead`

`boundary` → type signature: `LorentzOrbitRead`

`overshoot` → type signature: `LorentzOrbitRead`

`radialWidthScaledAt18` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.31 File: `Episode34.lean`

9.31.1 Inductive `ApparatusLaw` (Line 19)

This declaration represents a logical constructive sum type named `ApparatusLaw`. It represents a disjoint union populated by the following constructors:

`diracLorentzNeedleOrbit` → constructor signature: No parameter constructor

Implementation Commentary:

No local comments provided.

9.31.2 Structure `DiracExperimentReport` (Line 73)

This declaration represents a logical record structure named `DiracExperimentReport`. It constructs records using the constructor `DiracExperimentReport.mk` with the following fields:

`law` → type signature: `ApparatusLaw`

`lorentzWobbleScaledAt18` → type signature: `Nat`

`undershootRegime` → type signature: `LorentzOrbitRegime`

`boundaryRegime` → type signature: `LorentzOrbitRegime`
`overshootRegime` → type signature: `LorentzOrbitRegime`
`spinorState` → type signature: `WeakDiracGalerkin.Vec`
`gaugeCoupling` → type signature: `Int`
`tensor` → type signature: `WeakDiracGalerkin.UniversalTensor`
`galerkin` → type signature: `WeakDiracGalerkin.GalerkinReport`
`rawResidualZero` → type signature: `Bool`
`sobolevResidualZero` → type signature: `Bool`
`weakFrechetReads` → type signature: `List Int`

Implementation Commentary:

No local comments provided.

9.32 File: Episode35.lean**9.32.1 Structure SlipGridCell (Line 46)**

This declaration represents a logical record structure named `SlipGridCell`. It constructs records using the constructor `SlipGridCell.mk` with the following fields:

`index` → type signature: `Nat`
`lowerDistance` → type signature: `RationalDistance`
`upperDistance` → type signature: `RationalDistance`
`lowerDistanceScaledAt18` → type signature: `Nat`
`upperDistanceScaledAt18` → type signature: `Nat`
`lowerCrosses` → type signature: `Bool`
`upperCrosses` → type signature: `Bool`
`lowerSlipFloor?` → type signature: `Option Nat`
`upperSlipFloor?` → type signature: `Option Nat`
`lowerSlipScaledAt18?` → type signature: `Option Nat`
`upperSlipScaledAt18?` → type signature: `Option Nat`
`lowerFace?` → type signature: `Option CorridorFace`
`upperFace?` → type signature: `Option CorridorFace`

Implementation Commentary:

No local comments provided.

9.32.2 Structure DyadicSlipBracket (Line 92)

This declaration represents a logical record structure named `DyadicSlipBracket`. It constructs records using the constructor `DyadicSlipBracket.mk` with the following fields:

`lowerNumerator` → type signature: `Nat`

`upperNumerator` → type signature: `Nat`

`denominator` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.32.3 Structure GridSlipBisectSummary (Line 139)

This declaration represents a logical record structure named `GridSlipBisectSummary`. It constructs records using the constructor `GridSlipBisectSummary.mk` with the following fields:

`targetSlip` → type signature: `Nat`

`gridCells` → type signature: `Nat`

`rungs` → type signature: `Nat`

`gridCell` → type signature: `SlipGridCell`

`lowerDistanceScaledAt18` → type signature: `Nat`

`upperDistanceScaledAt18` → type signature: `Nat`

`midpointScaledAt18` → type signature: `Nat`

`widthScaledAt18` → type signature: `Nat`

`lowerSlipFloor` → type signature: `Nat`

`upperSlipFloor` → type signature: `Nat`

`lowerSlipScaledAt18` → type signature: `Nat`

`upperSlipScaledAt18` → type signature: `Nat`

`lowerFace` → type signature: `CorridorFace`

`upperFace` → type signature: `CorridorFace`

Implementation Commentary:

No local comments provided.

9.33 File: Episode36.lean

9.33.1 Structure ForceMeasuredGReport (Line 27)

This declaration represents a logical record structure named `ForceMeasuredGReport`. It constructs records using the constructor `ForceMeasuredGReport.mk` with the following fields:

```
targetForce → type signature: Nat
gridCells → type signature: Nat
rungs → type signature: Nat
lowerDistance → type signature: RationalDistance
upperDistance → type signature: RationalDistance
midpointDistance → type signature: RationalDistance
lowerDistanceScaledAt18 → type signature: Nat
upperDistanceScaledAt18 → type signature: Nat
midpointDistanceScaledAt18 → type signature: Nat
lowerG → type signature: ApparatusRatio
upperG → type signature: ApparatusRatio
midpointG → type signature: ApparatusRatio
exactRecoveredG → type signature: ApparatusRatio
originalDeviceG → type signature: ApparatusRatio
lowerGScaledAt18 → type signature: Nat
upperGScaledAt18 → type signature: Nat
midpointGScaledAt18 → type signature: Nat
exactRecoveredGScaledAt18 → type signature: Nat
originalDeviceGScaledAt18 → type signature: Nat
bracketContainsDeviceG → type signature: Bool
exactMatchesDeviceG → type signature: Bool
```

Implementation Commentary:

No local comments provided.

9.34 File: Episode37.lean

9.34.1 Inductive ElaborationChargeOrientation (Line 15)

This declaration represents a logical constructive sum type named `ElaborationChargeOrientation`. It represents a disjoint union populated by the following constructors:

`electron` → constructor signature: No parameter constructor

`positron` → constructor signature: No parameter constructor

`neutral` → constructor signature: No parameter constructor

Implementation Commentary:

No local comments provided.

9.34.2 Structure ElectronElaborationChargeReport (Line 45)

This declaration represents a logical record structure named `ElectronElaborationChargeReport`. It constructs records using the constructor `ElectronElaborationChargeReport.mk` with the following fields:

`elaborationCost` → type signature: Nat

`costRead` → type signature: WheelStimulusRead

`chargeMagnitude` → type signature: Nat

`signedCharge` → type signature: Int

`signedChargeRead` → type signature: WheelStimulusRead

`chargeUnit` → type signature: Int

`electronChargeConvention` → type signature: Int

`chargeOrientation` → type signature: ElaborationChargeOrientation

`costIsChargeMagnitude` → type signature: Bool

`chargeUnitMatchesElectronConvention` → type signature: Bool

Implementation Commentary:

No local comments provided.

9.35 File: Episode38.lean

9.35.1 Structure SlipPointChargeReport (Line 35)

This declaration represents a logical record structure named `SlipPointChargeReport`. It constructs records using the constructor `SlipPointChargeReport.mk` with the following fields:

`targetSlip` → type signature: Nat

`gridCells` → type signature: Nat

`rungs` → type signature: `Nat`
`lowerDistance` → type signature: `RationalDistance`
`upperDistance` → type signature: `RationalDistance`
`midpointDistance` → type signature: `RationalDistance`
`midpointDistanceScaledAt18` → type signature: `Nat`
`signedCharge` → type signature: `Int`
`chargeMagnitude` → type signature: `Nat`
`normalizedCharge` → type signature: `Int`
`chargeOrientation` → type signature: `ElaborationChargeOrientation`
`chargeRead` → type signature: `WheelStimulusRead`
`midpointPotentialMagnitude` → type signature: `ApparatusRatio`
`midpointPotentialMagnitudeScaledAt18` → type signature: `Nat`
`midpointFieldMagnitude` → type signature: `ApparatusRatio`
`midpointFieldMagnitudeScaledAt18` → type signature: `Nat`
`exactBoundaryFieldMagnitude` → type signature: `ApparatusRatio`
`exactBoundaryFieldMagnitudeScaledAt18` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.36 File: Episode39.lean

9.36.1 Inductive ChargeSecondVariationChannel (Line 16)

This declaration represents a logical constructive sum type named `ChargeSecondVariationChannel`. It represents a disjoint union populated by the following constructors:

`rawCharge` → constructor signature: `No parameter constructor`
`normalizedCharge` → constructor signature: `No parameter constructor`
`potential` → constructor signature: `No parameter constructor`
`field` → constructor signature: `No parameter constructor`

Implementation Commentary:

No local comments provided.

9.36.2 Structure ChargeSecondVariationComponent (Line 23)

This declaration represents a logical record structure named `ChargeSecondVariationComponent`. It constructs records using the constructor `ChargeSecondVariationComponent.mk` with the following fields:

`channel` → type signature: `ChargeSecondVariationChannel`

`magnitude` → type signature: `ApparatusRatio`

`scaledAt18` → type signature: `Nat`

`signedUnit` → type signature: `Int`

`orientation` → type signature: `ElaborationChargeOrientation`

Implementation Commentary:

No local comments provided.

9.36.3 Structure ChargeSecondVariationReport (Line 57)

This declaration represents a logical record structure named `ChargeSecondVariationReport`. It constructs records using the constructor `ChargeSecondVariationReport.mk` with the following fields:

`normalization` → type signature: `String`

`variationRank` → type signature: `Nat`

`slipPoint` → type signature: `SlipPointChargeReport`

`rawCharge` → type signature: `ChargeSecondVariationComponent`

`normalizedCharge` → type signature: `ChargeSecondVariationComponent`

`potential` → type signature: `ChargeSecondVariationComponent`

`field` → type signature: `ChargeSecondVariationComponent`

`components` → type signature: `List ChargeSecondVariationComponent`

`componentCount` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.37 File: Episode40.lean

9.37.1 Structure AlphaSecondVariationReport (Line 40)

This declaration represents a logical record structure named `AlphaSecondVariationReport`. It constructs records using the constructor `AlphaSecondVariationReport.mk` with the following fields:

`normalization` → type signature: `String`

`orbitUnit` → type signature: `Nat`

`targetSlip` → type signature: `Nat`
`distance` → type signature: `RationalDistance`
`distanceScaledAt18` → type signature: `Nat`
`rawCharge` → type signature: `ApparatusRatio`
`normalizedCharge` → type signature: `ApparatusRatio`
`inverseRadius` → type signature: `ApparatusRatio`
`tange` → type signature: `ApparatusRatio`
`fieldPerCharge` → type signature: `ApparatusRatio`
`electromagneticCoupling` → type signature: `ElectromagneticCouplingCoefficient`
`electromagneticCouplingFactorScaledAt18` → type signature: `Nat`
`electromagneticCalibrationTapeLength` → type signature: `Nat`
`couplingGeneratedBeforeAlpha` → type signature: `Bool`
`maxwellSpeedOfLightEnforced` → type signature: `Bool`
`maxwellSpeedOfLightSquaredScaledAt18` → type signature: `Nat`
`maxwellNaturalUnitVelocitySquaredScaledAt18` → type signature: `Nat`
`maxwellLorentzBoundaryVelocitySquaredScaledAt18` → type signature: `Nat`
`magneticRepulsionHasPhotonRecoilResponse` → type signature: `Bool`
`magneticPhotonIncidentResponseResidueScaledAt18` → type signature: `Nat`
`magneticPhotonRecoilCarried` → type signature: `Bool`
`alpha` → type signature: `ApparatusRatio`
`alphaScaledAt18` → type signature: `Nat`
`inverseAlphaScaledAt18` → type signature: `Nat`

Implementation Commentary:

No local comments provided.

9.38 File: TwoDescriptions.lean**9.38.1 Inductive Spin (Line 26)**

This declaration represents a logical constructive sum type named `Spin`. It represents a disjoint union populated by the following constructors:

`up` → constructor signature: `No parameter constructor`
`down` → constructor signature: `No parameter constructor`

Implementation Commentary:

- `Spin`: the two half-turns of the rotation.

9.38.2 Inductive Orientation (Line 32)

This declaration represents a logical constructive sum type named `Orientation`. It represents a disjoint union populated by the following constructors:

`matter` → constructor signature: No parameter constructor

`antimatter` → constructor signature: No parameter constructor

Implementation Commentary:

- Matter or antimatter.

9.38.3 Structure Leg (Line 38)

This declaration represents a logical record structure named `Leg`. It constructs records using the constructor `Leg.mk` with the following fields:

`orientation` → type signature: `Orientation`

`spin` → type signature: `Spin`

Implementation Commentary:

- One leg of the orbit / one member of the pair.